# 1. Python Revision Tour 1

**Python Character set-** A character set is the set of all valid characters that the Python language recognizes. The following characters are included in the python character set-

- Letters: A-Z, a-z
- Digits: 0-9
- Special symbols: + - * / \ ( ) [ ] { } = ! > < . , ' " ; : % & # @ _
- Whitespaces: space key, tab, carriage return, newline, formfeed
- Other characters - All other ASCII and Unicode characters are recognized a part of data only.

**Tokens -** Tokens are the smallest individual unit in a program. Python has the following tokens

a) Keywords
b) Identifiers
c) Literals
d) Punctuators/Delimiters
e) Operators

**a) Keywords-** Keywords are words having special meaning in a programming language. eg. True, False, if, elif etc.

**b) Identifiers -** Identifiers are names given to different parts of a program such as variables, objects, functions etc.

**Rules for naming identifiers-**

1) Keywords cannot be used
2) They can contain only letters, numbers and underscore.
3) They cannot begin with a number

Examples of valid identifiers - abc, abc123, abc_123, _abc

Examples of invalid identifiers - 9abc, abc$123, elif

c) **Literals** - Literals are data items having a fixed or constant value. Literals can be -

i. **String Literals**- a sequence of characters enclosed in single quotes, double quotes, triple single quotes or triple double quotes. Eg. 'abc', "hello", '''good morning'''
ii. **Numeric Literals** - Literals containing numeric data. They can be -
    a. **Integers** - Integers not containing decimal point. They can be written in the following ways-
        i. **Decimal form**- containing decimal symbols 0-9 and can be preceded by +/-
        ii. **Octal form** - containing octal symbols 0-7 and preceded by 0o/0O i.e. digit zero followed by either capital 'O' or small 'o'.
        iii. **Hexadecimal form** - containing hexadecimal symbols 0-9, a-f, A-F and preceded by 0x/0X i.e. digit zero followed by small 'x' or capital
    b. **Floating point literals -** They are numbers containing decimal point. They can be written in -
        i. **Fractional form -** eg -3.2, 4.5567
        ii. **Exponent form -** eg 0.5E2, 3.67e5. The E/e represents power of 10 and the number following E/e must be an integer.
iii. **Boolean Literals -** They have value either True or False.
iv. **None -** It is a special literal which is used to indicate absence of value

**d) Punctuators/Delimiters -** Punctuators are symbols that are used to organize sentences/lines in python. Eg. ', ", (, [ etc.

**e) Operators -** Operators are symbols that trigger some computation when applied to variables or other objects. The following are some kinds of operators-

a) Arithmetic operators: +, -, *, /, %, **, //
b) Assignment operator: =
c) Relational operators: >, <, >=, <=, ==, !=
d) Logical operators: not, and , or
e) identity operators, is , is not
f) membership operators- in , not in
g) arithmetic-assignment operators - +=, -=, *=, %=, **=, //=

**Precedence of Operators**

When an expression consisting of different operators is encountered, the expression is evaluated based on precedence of operator. Higher precedence operator is evaluated first before lower precedence operator.

| Operators | Description | |
|---|---|---|
| () | Parentheses | **Highest Precedence** |
| x ** y | Power (exponentiation) | |
| −x, +x | Negation, identity | |
| x / y, x // y, x % y, x * y | True Division; floor division; (Remainder/format); Multiplication; repetition; | |
| x – y, x + y | (Subtraction/set difference), (Addition, concatenation) | |
| x == y, x != y, x < y, x <= y, x > y, x >= y | Relational Operators | |
| not x | Logical negation | |
| x and y | Logical AND | |
| x or y | Logical OR | |
| = | Assignment operator | **Lowest Precedence** |

**Operator associativity**

Operator associativity determines the order of evaluation when they are of same precedence, and are not grouped by parenthesis. An operator may be Left-associative or Right–associative. In left associative, the operator falling on left side will be evaluated first, while in right associative operator falling on right will be evaluated first.

Usually all operators are left associative, except '**' which is right associative.

**Variables -** Variables are named storage location whose values can be manipulated during program run.

**Dynamic Typing -** In python the same variable name can be used to refer to one data type such as int and in the next statement it can be used to refer to some other data type such as a string. This facility of changing the data type of a variable during program execution is known as Dynamic Typing.

**Comments-**They are un-executable code that are ignored by the Python interpreter. Comments are used to describe the working of the program and are also used to describe the changes that have been made to the program over a period of time.

**Ways of writing comments:**

1. Single line/inline comment - Place a # in front of the statement/line
2. multi-line comments- Use triple single quotes or triple double quotes to write comments that span multiple lines

**Assignment Operator -** The = is the assignment operator in python. It is used to assign a variable on the left side of the =, a value or an expression that evaluates to a value that is present on the right side of the =. The left side of the = symbol must contain either a single variable or if more than one variable is present then they must be separated by comma. The right side of the = symbol contains values or expressions that evaluates to a value. The number of values on the right side must match the number of variables on the left side.

Examples of valid assignment statements:
a = 5
a, b = 5, 'hello'
Examples of invalid assignment statements:
5 = a
a, b, c = 5, 6

**Multiple Assignments to a Variable**

```
a = b = c = 3
print (a)
print (b)
print (c)

d, e, f = 4, 5.23, "hello"
print (d)
print (e)
print (f)
```

**o/p**

*3*
*3*
*3*
*4*
*5.23*
*hello*

Variables a, b and c are assigned the same value of 3, whereas in the second case d is assigned a value of 4, e is assigned a value of 5.23 and f is assigned a value of “hello”. When using the second method of initialization care must be taken that the number of variables on the left of the = must be the same as the number of values on the right of the =, otherwise python gives an error.

If the right side of the = symbol (rvalue) contains many expressions separated by commas then -

1. The entire rvalue i.e. everything after the =, is evaluated first from left to right in one go.
2. Next the variables on the left side of the =, are assigned the corresponding values from left to right.

Normally this works fine when the variables on the left side of = symbol (lvalue) are all different. But if the variables are repeated in lvalue then the rightmost rvalue is the final value of the variable.

eg.
*x=2*
*y, x, y = 3 , x + 4, x +5*
*print (x,y)*

*o/p:*
*6 7*

The expression
y, x, y = 3 , x + 4, x +5
is evaluated as:

1. First the entire rvalue is evaluated at one go i.e. the expression becomes
   y,x,y = 3, 2+4, 2+5
   i.e., y, x, y = 3, 6, 7
2. Then, the lvalue variables are assigned from left to right as
   y = 3
   x=6
   y=7
3. After the statement is executed the variables have final values: x = 6 and y = 7

**Implicit data type conversion-** The data type conversions done automatically by the python interpreter are known as implicit data type conversions.

E.g. *x = 5 + 2.5*
*print (x)*

In the above expression the 5 is int and the 2.5 is float. So the int 5 is converted to float implicitly by the python interpreter and the operation is carried out as float + float and the variable x has float 7.5 stored in it.

Usually expressions containing different kinds of numbers are implicitly converted as -
int -> float ->complex

**Explicit data type conversion-** When the programmer forces one data type to be converted to another data type by using some function such as int(), float(), str() etc , then it is known as explicit data type conversion

E.g. x=int(input('Enter your age:'))
print(x)

In the above expression the input is always accepted as a string. In order to accept it as an *int*, the programmer has explicitly converted the string to an int.

**input statement-**

The input statement is used to accept user input. The user input is always accepted as a string which can be converted to any other data type by using the appropriate type cast operator.

```
eg.  name = input('Enter your name:')
     age = int(input('Enter your age:'))
```

**print statement-**

The print statement is used to display output. The print can be used in the following ways:

a) print('hello') # displays hello

b) x=5
print (x) #used to display value contained in variable x

c) print("abc", "def", "ghi") # displays abc def ghi there is a space in between the words

d) print("jk", "lm", "no", sep='# ') # displays jk#lm#no i.e. separator '#' is added in between the words

e) print("pq" , "rs", sep="!", end="#") #end changes the default end line character from '\n' to '#'
print ("tu") # prints pq!rs#tu in a single line

f) print('hello\'s') #\ escape sequence \' displays single quote
o/p:
hello's

g) print("world\"s") #\" displays double quote
o/p:
world"s

h) print("line one\nline two") #\n for newline
o/p:
line one
line two

i) print('how are\t\tyou') #\t for tab
o/p:
how are you

j) print("""good
morning""") #multiple line strings can be enclosed in triple single or triple double quotes
o/p:
good
morning

k) print('It is \
raining today') #\ char at the end is used to remove the newline at end
o/p:
It is raining today

l) x = ("hello " "world") #string placed next to each other are concatenated especially useful to place on multi lines
print (x)
o/p:
hello world

m) print('good"day') #double quotes can be enclosed within single quotes
o/p:
good"day

n) print("rainy'day") #single quotes can be enclosed within double quotes

**o/p:**

rainy'day

o) print(r'hello \n how \t\tare \'you') #using raw strings

**o/p:**

hello \n how \t\tare \'you

**Raw strings:** Raw strings are preceded by the character 'r' or 'R' in front of the string. For raw strings python does not apply any formatting inside the string and the escape sequences are ignored.

p) #using format functions

```
x='abc'
y=12
z="Hello {}. You are {} years old".format(x,y)
print(z)
```

**o/p:**

Hello abc. You are 12 years old

**Format() function:**A string enclosed in single or double quotes can have empty { } inside it which act as placeholders for values to be received from variables. The variables are passed as parameters to the format function.

q) #using format-strings / f-strings

```
x='abc'
y=12
print(f'hello {x} you are {y+2} years old')
```

**o/p:**

hello abc you are 14 years old

**f-string / format strings:**

The f-string is an extension of the format function in which instead of the empty { } brackets inside the string, variables names are written inside the { }. During program run the value of the variables is substituted and then given to the print function. The f-string must have 'f' or 'F' written before them.

r) #c-style string formatting

```
x='lia'
y=12
z=25.8
s="%s has %i apples and the average weight is %f grams. We have %.2f grams in all" %(x,y,z, y*z)
print(s)
```

o/p:

lia has 12 apples and the average weight is 25.800000 grams. We have 309.60 grams in all

**c-style string formatting:**

This formatting is inspired from c-language style string formatting. Here inside the string the place holders are specified by %s – string data , %i or %d - integer data,

%f – floating point data (we can specify the precision of the decimal by using the decimal point and the number of digits we want to be displayed after the decimal point before the f symbol.

After the string a tuple object [ enclosed in parentheses () ] is passed after specifying the % symbol after the string for string replacement.

**CONDITIONAL STATEMENTS -** Conditional statements allow us to execute one/multiple statements only if some condition evaluates to True.

The following conditional statements are supported in Python

1. If statement
2. If ... else ... statement
3. Nested if statements
4. If-elif-else statement

**1. If statement**

The if statement evaluates a condition and if the condition is True then either a single statement of a set of statements can be executed.

```
x=int(input("Enter a number: "))
if ( x > 100 ) :
   print ("The value")
   print (" that you have entered, ")
   print(str(x) + " is greater than 100")
print ("Good bye!")
```

**2. If—else statement**

If we want that the program should execute one set of statements when the condition is True and another set of statements when the condition is false then in that case, we use the if-else statement.

```
x = int(input('Enter number 1 : '))
y = int(input('Enter number 2 : '))
if x > y:
   print(x, " is greater ")
   print (" than" , y)
else:
   print(x, " is less ")
   print("than ", y)
print("Good Bye!")
```

**NOTE:** In an if-else statement if there is only one statement in the if-part or the else part then that one statement can be written in the same line as the header after the colon :

e.g.

```
x = int(input('Enter a number : '))
if x > 0: print(x, " is greater than 0")
else: print(x, " is less than 0")
```

3. **Nested if-else statements-** The nested if-else contains one if statement inside another if statement either in the True block or the False block.

```
x = int(input('Enter the x co-ordinate: '))
y = int(input('Enter the y co-ordinate: '))
if x > 0:
    if y > 0:
        print("The point lies in Quadrant I")  # x > 0, y > 0
    else:
        print("The point lies in Quadrant IV")  # x > 0, y <= 0
else:
    if y > 0:
        print("The point lies in Quadrant II")  # x <= 0, y > 0
    else:
        print("The point lies in Quadrant III")  # x <= 0, y <= 0
```

4. i**f-elif-else statement**

The if-elif-else statements are used when we want to test a series of conditions one after the other and only one of the conditions can be True at a time. There can be only one else and the else must be written after all the elif statements. The else part is optional.

```
m= int(input("Enter your percentage: "))
if m>=90:
    print("A Grade")
elif  m>=75 and m<90:
    print("B Grade")
elif  m>=50 and m<75:
    print("C Grade")
else:
    print ("D Grade")
```

**pass statement** - The pass statement does nothing. It is used in exceptional cases where due to the syntax requirements we must have a statement but we do not have any valid statement to execute. Typically this is done to provide clarity to the code.

e.g.

```
m= int(input("Enter your marks: "))
if m>=0:
        pass
else:
        print ("You have entered invalid marks")
```

**Truth Values (True and False Values)**

- All objects have an inherent Boolean True or False value.
- Any nonzero number or nonempty object is True.
- Zero numbers, empty objects, and the special object None are evaluated as False.
- Comparisons and equality tests return True or False (custom versions of 1 and 0).

L**oops -** Loops / Iteration statements are used to repeatedly execute a block of code. Python provides two types of looping constructs:

1. for statement
2. while statement

**for statement**- for is an iterator and is used to iterate over a sequence of objects. The sequence of objects can be integers generated by the range() function, the sequence can be another object such as a string, list, tuple or a dictionary in which case the for loop iterates over all the elements contained in that sequence object.

**Using for statement along with the range() function-**

| | | | |
|---|---|---|---|
| for i in range(3):<br>  print(i)<br>print("abc") | for i in range(5, 8):<br>  print(i, i ** 2)<br>print('end of loop') | for i in range(5, 12, 2):<br>  print(i, i * 2)<br>print('end of loop')<br>print ("value of i is", i) | for i in range(9, 1, -2 ):<br>  print(i)<br>print('end of loop')<br>print ("value of i is", i) |
| o/p:<br>0<br>1<br>2<br>abc | o/p:<br>5 25<br>6 36<br>7 49<br>end of loop | o/p:<br>5 10<br>7 14<br>9 18<br>11 22<br>end of loop<br>value of i is 11 | o/p:<br>9<br>7<br>5<br>3<br>end of loop<br>value of i is 3 |

**range() function-** The range() function generates a sequence of integers. It comes in three forms-

1. **range(x)** - x must be a positive integer. It generates numbers starting from 0 till x-1.
2. **range(start, stop)** - start, stop must be integers and start must be less than stop. It generates numbers starting from start and ending at stop-1. The number 'stop' is not included in the sequence generated.
3. **range(start, stop, step)** - Here the third parameter, step can be-
    a. **positive step value-** In this case start must be less than stop. The sequence generated is start, start+step, start+ 2*step and so on. The ending number generated must be less than stop.
    b. **negative step value-** In this case start must be greater than stop. The sequence generated is start, start-step, start - 2*step, start-3*step and so on. The ending number generated is greater than the stop value.

**while statement-** while statement repeats the block of code many times until some condition evaluates to False.

Usually while using the while statement we use a test/counter variable before the while loop starts and initialize it to some value. Then in the condition of the while statement we test that variable. Inside the body of the while loop there must be some statement which changes the value of the test/counter variable and leads to the test condition being turned False at some point of time. Otherwise the loop will continue forever and this condition is known as infinite loop.

| | |
|---|---|
| i = 1<br>while i <= 5:<br>  print(i ** 2)<br>  i += 1<br>print('Loop body ends')<br>print("value of i outside the loop is :", i) | o/p<br>1<br>4<br>9<br>16<br>25<br>Loop body ends<br>value of i outside the loop is : 6 |

**break statement-** The break statement is used to break out of the innermost loop in which it is situated. Usually break statement is used along with some conditional statement inside the body of for/while loop. When the condition is True the break is encountered and the loop terminates.

| | |
|---|---|
| for i in range(1, 10 ):<br>  if i==5:<br>  break<br>  print(i)<br>print('end of loop') | o/p:<br>1<br>2<br>3<br>4<br>end of loop |

**continue statement-** The continue statement is used to skip the rest of the statements of the current loop block and to move to next iteration, of the loop. The continue statement also is usually enclosed in a conditional statement and situated inside a for/while loop.

| | |
|---|---|
| i=0<br>while i<10:<br>  i=i+1<br>  if i%2==0:<br>  continue<br>  print(i)<br>print('end of loop') | o/p:<br>1<br>3<br>5<br>7<br>9<br>end of loop |

**else block in the for and while statement**

The else block is an optional block in the loop statements (for & while loop statements). It executes when the condition of the loop turns False(in the while loop) or when the items in the range of the for list have been exhausted and the loop is exited normally. The else block is not executed when the loop is exited due to a break statement inside the for/while loop.

| | |
|---|---|
| for i in range(2,5):<br>  print(i)<br>else:<br>  print('Loop ended normally')<br>print('Outside loop') | for i in range(2,10):<br>  print(i)<br>  if i==5:<br>  break<br>else:<br>  print('Loop ended normally')<br>print('Outside loop') |
| o/p:<br>2<br>3<br>4<br>Loop ended normally<br>Outside loop | o/p:<br>2<br>3<br>4<br>5<br>Outside loop |

**Bugs**- Errors in a program are known as bugs.

**Debugging**- The process of identifying and removing errors in programming is called debugging.

Types of Errors- Errors in programming can be classified into-
i) Syntax errors
ii) Logical errors
iii) Runtime errors

**i) Syntax errors**- If the syntax or rule of any statement in programming language is violated, then it is known as syntax error. E.g., missing parentheses, wrong indentation, incorrect keywords etc.

**ii) Logical/Semantic errors**- In Logical errors the program runs without abrupt termination of program but it does not give the desired output or the program does not behave as expected. We have to trace backward from the incorrect output and find out the source of the error. For example, to compute the average of two variable the code may have been written as-
x= a+ b/2
Although the program may run without giving any errors but still for finding average the above code is incorrect as there must be parentheses around a+b.
Logical errors are also called semantic errors as they occur when the meaning of the program (its semantics) is not correct.

**iii) Runtime errors-** A runtime error causes abnormal termination of program while it is executing. Runtime error is when the statement is correct syntactically, but the interpreter cannot execute it. Runtime errors do not appear until after the program starts running or executing.
For example, we have a statement having division operation in the program. By mistake, if the denominator entered is zero then it will give a runtime error like “division by zero”.

# 2. Python Revision Tour 2

**Sequence Data Types:** A sequence data type is an ordered data type where each element can be accessed by their position or index using the index [] operator. Examples of sequence data type are strings, lists and tuples.

**Strings -** Strings are characters enclosed in either single quotes or double quotes or triple single quotes or triple double quotes. E.g. 'hello', "world", '''have a nice day ''', """Good Morning""".

## String Functions/Operations

| Sr. No. | Function Name / Syntax | Description | Example |
|---|---|---|---|
| 1 | Displaying a String | | S='hello world'<br>print(S)<br><br>o/p:<br>*hello world* |
| 2 | Accessing a single character of a String | We can use forward indexing or backward indexing | S = 'hello'<br>print(S[4] )<br>print (S[-5])<br><br>o/p:<br>o<br>h |
| 3 | len() | Finding number of characters in a String | S='hello world'<br>x=len(S)<br>print(x)<br><br>o/p:<br>11 |
| 4 | + | Joining/Concatenating two Strings | S1 = 'hello'<br>S2= 'world'<br>S3=S2+S1<br>print(S3)<br><br>o/p:<br>worldhello |
| 5 | * | Replicating Strings | S1 = 'hello'<br>S2=S1*3<br>print(S2)<br><br>o/p:<br>hellohellohello |
| 6 | isalpha() | Checks if all characters in string are alphabetic | s1='abcd123'<br>print(s1.isalpha())<br><br>o/p:<br>False |
| 7 | isalnum() | Checks if all characters in string are alphanumeric | s1='abcd123'<br>print(s1.isalnum())<br><br>o/p:<br>True |
| 8 | isdigit() | Checks if all characters in string are digits | s2='12345'<br>print(s2.isdigit())<br><br>o/p:<br>True |

| | | | |
|---|---|---|---|
| 9 | islower() | Checks if all alphabets in string are in lowercase | s3='abc123#$%'<br>print(s3.islower())<br><br>o/p:<br>True |
| 10 | isupper() | Checks if all alphabets in string are in uppercase. | s4='ABCD'<br>print(s4.isupper())<br><br>o/p:<br>True |
| 11 | isspace() | Checks if all characters in string are whitespace characters. | s5='  '<br>print(s5.isspace())<br><br>o/p:<br>True |
| 12 | upper() | return a new string with all characters turned to uppercase | s1 = 'abcDEF123  XYZ#'<br>s3=s1.upper()<br>print(s3)<br><br>o/p:<br>ABCDEF123  XYZ# |
| 13 | lower() | return a new string with all characters turned to lowercase | s1 = 'abcDEF123  XYZ#'<br>s3=s1.lower()<br>print(s3)<br><br>o/p:<br>abcdef123  xyz# |
| 14 | capitalize () | returns a new string with the first character of the string turned to uppercase and all other characters turned to lowercase. | s1 = 'HELLO WORLD 123'<br>s3=s1.capitalize ()<br>print(s3)<br><br>o/p:<br>Hello world 123 |
| 15 | title () | returns a new string with the first alphabet of each word in uppercase and all the other alphabets of the word in lowercase | s1 = 'HELLO WORLD 123'<br>s3=s1.title()<br>print(s3)<br><br>o/p:<br>Hello World 123 |
| 16 | in /  not in | String membership testing | s = 'hello world'<br>x= 'llo'<br>if x in s:<br>  print(x, 'is present in', s)<br>if 'WOR' not in s:<br>  print('WOR', 'is not present in', s)<br><br>o/p:<br>llo is present in hello world<br>WOR is not present in hello world |
| 17 | *string.find( substring [, start_index, end_index] )* | returns the index of the first occurrence of substring within the given string. If the substring is not present then find() returns -1. | s = 'hello world'<br>print(s.find('llo'))<br><br>o/p:<br>2 |
| 18 | *string.index( substring [, start_index, end_index] )* | returns the index of the first occurrence of substring within the given string. If the substring is not present then the index() method raises an error. | s = 'hello world'<br>print(s.index('llo'))<br><br>o/p:<br>2 |
| 19 | *string.count( substring [, start_index, end_index] )* | returns the number of times a substring occurs in the given string. | s = 'hello world good morning have a nice day'<br>y=s.count('or')<br>print('or is present', y, 'times') |

| | | | |
|---|---|---|---|
| | | | o/p:<br>or is present 2 times |
| 20 | Traversing a String using the index | | S = 'hello'<br>x=len(S)<br>for i in range(x):<br>print(S[i])<br><br>o/p:<br>h<br>e<br>l<br>l<br>o |
| 21 | Traversing a String using the iterator | | S='hello'<br>for i in S:<br>print(i)<br>o/p:<br>h<br>e<br>l<br>l<br>o |
| 22 | lstrip and rstrip | used to remove characters from the left or right of the string. If no parameters is passed to these functions then only whitespaces are removed. If a string is passed as a parameter to these functions then all the combinations of the characters forming the parameter are removed. | s1 = ' abcdpqrstuv'<br>s2=s1.lstrip()<br>print('s2=', s2)<br>s3=s1.rstrip('putv')<br>print('s3=', s3)<br><br>o/p:<br>s2= abcdpqrstuv<br>s3= abcdpqrs |
| 23 | strip | used to remove characters from the both sides of the string. If no parameters are passed to the strip function then only whitespaces are removed from both sides. If a string is passed as a parameter to the strip function then all the combinations of the characters forming the parameter are removed from both sides. | s1 = ' abcdpqrstuv '<br>s2=s1.lstrip()<br>print('s2=', s2)<br>s3=',,,,,rrttgg.....banana....rrr'<br>s4=s3.strip(",.grt")<br>print('s4=', s4)<br><br>o/p:<br>s2= abcdpqrstuv<br>s4= banana |
| 24 | split() | used to split a line into words based on the default separator space. The individual words become members of the list. If the default separator is not a space then the separator must be passed as a parameter to the split. | s1 = 'hello world good morning'<br>L1=s1.split()<br>print('L1=', L1)<br>s2='have,a,nice,day'<br>L2=s2.split(',')<br>print('L2=', L2)<br><br>o/p:<br>L1= ['hello', 'world', 'good', 'morning']<br>L2= ['have', 'a', 'nice', 'day'] |
| 25 | partition() | It partitions the given string at the first occurrence of the substring (separator) and returns the string partitioned into three parts.<br>1. Substring before the separator<br>2. The Separator itself<br>3. Substring after the separator | str1 = 'India is a Great Country'<br>s1,s2,s3=str1.partition('is')<br>print('s1=', s1)<br>print('s2=', s2)<br>print('s3=', s3)<br><br>o/p:<br>s1= India<br>s2= is<br>s3= a Great Country |

<table>
<tr><td>26</td><td>replace()</td><td>replaces all the occurrence of a specified phrase with another specified phrase and returns this new string created.</td><td>txt = "I like bananas. Monkeys like bananas."<br>x = txt.replace("bananas", "apples")<br>print(x)<br><br>o/p:<br>I like apples. Monkeys like apples.</td></tr>
<tr><td>27</td><td>ord('char')</td><td>used to return the ordinal int value associated with a 'char'.<br><br>common ordinal values are:<table><tr><th>Characters</th><th>Ordinal Integer values</th></tr><tr><td>'0' to '9'</td><td>48 to 57</td></tr><tr><td>'A' to 'Z'</td><td>65 to 90</td></tr><tr><td>'a' to 'z'</td><td>97 to 122</td></tr></table></td><td>print(ord('0'), ord('9'))<br><br>o/p:<br>48 57</td></tr>
<tr><td>28</td><td>chr(int_val)</td><td>used to find the character represented by the integer 'int_val'.</td><td>print(chr(212))<br><br>o/p:<br>Ô</td></tr>
<tr><td>29</td><td>s1.join(s2)</td><td>For using the join function, s1 object must be a string and the argument s2 can be any sequence object / iterable. The join function returns back a string in which in between each element of s2 the string s1 is inserted. All the elements returned back on iterating the s2 object must be strings otherwise an error occurs.</td><td>s1='abc'<br>s2='def'<br>s3=s1.join(s2)<br>print(s3)<br><br>s4=s1.join(['mn','op','qr'])<br>print(s4)<br><br>s5=s1.join({'tu':'vw', 'xy':10})<br>print(s5)<br><br>o/p:<br>dabceabcf<br>mnabcopabcqr<br>tuabcxy</td></tr>
<tr><td>30</td><td>str.startswith(prefix[, start[, end]])</td><td>returns True if a given string str starts with the given prefix and otherwise returns False. The prefix can be a string or it can be a tuple of strings.</td><td>s1='hello world good morning'<br>print(s1.startswith('world'))<br>print(s1.startswith('world', 6))<br>print(s1.startswith('good', -12,-3))<br>t=('a','e','i','o','u')<br>print(s1.startswith(t, 6))<br><br>o/p:<br>False<br>True<br>True<br>False</td></tr>
<tr><td>31</td><td>str.endswith(suffix[, start[, end]])</td><td>returns True if a given string str ends with the given suffix and it returns False if the given string does not end with the given suffix. The suffix can be a string or it can be a tuple of strings.</td><td>s1='hello world good morning'<br>print(s1.endswith('morn'))<br>print(s1.endswith('morning', 6))<br>print(s1.endswith('good', 6,-8))<br>t=('a','e','i','o','u')<br>print(s1.endswith(t, 6, -2))<br><br>o/p:<br>False<br>True<br>True<br>True</td></tr>
</table>

**String Comparisons**

The relational operators >, >=, <, <=, ==, != can be applied to two strings to arrive at a True or False value. The following rules are observed:

a. **For equality (==) operator** - If equality evaluate to True only when all the characters of one string match all the characters of the second string and both the strings have the same number of characters.
b. **For all other operators** - The characters are compared **with respect to their ordinal integer values** sequentially using the operator under consideration. If a clear answer (either True or False) is arrived then that becomes the result of the comparison between the two strings. A clear answer is not arrived when characters of both the strings happen to have same value. Then in that case the next set of characters is checked and the process is repeated.
c. If two strings are of different lengths and if one of the string has a character at one index position and the other string does not have a character at that position then the evaluation proceeds as follows:
    - something/some_char > empty/nothing evaluates to True
    - something/some_char < empty/nothing evaluates to False

| **Comparison** | **Result** |
|---|---|
| 'abc' == 'abc' | True |
| 'abc' == 'ABC' | False |
| 'abc' != 'ABC' | True |
| 'abc'  > 'abd' | False |
| 'x' > 'abcd' | True |
| 'B' > 'xyz' | False |
| 'Ca' < 'cZ' | True |
| 'abc' < 'ab' | False |
| 'ab2' < 'A9' | False |
| '92' > 'AB' | False |

**String Slicing**

String slice is creating a new string using elements of an existing string. It is created as:
*StringName[start : stop : step]* where start, stop , step are integers

**The basic rules of slice are same for strings, lists and other objects in python:**

i. The slice generates index/integers from - start, start + step, start + step + step, and so on. All the numbers generated must be less than the stop value when step is positive.
ii. If step value is missing then by default is taken to be 1
iii. If start value is missing and step is positive then start value is by default taken as 0.
iv. If stop value is missing and step is positive then start value is by default taken to mean till you reach the ending index(including the ending index)
v. A negative step value means the numbers are generated in backwards order i.e. from - start, then start - step, then start -step -step and so on. All the numbers generated in negative step must be greater than the stop value.
vi. If start value is missing and step is negative then start value takes default value -1
vii. If stop value is missing and step is negative then stop value is by default taken to be till you reach the first element(including the 0 index element)

Consider the String S:

| forward indexing -----> | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| S= | H | E | L | L | O | W | O | R | L | D | |
| | -10 | -9 | -8 | -7 | -6 | -5 | -4 | -3 | -2 | -1 | <----- Backward Indexing |

| **Slice** | **Generates** |
|---|---|
| S[4: ] | 'OWORLD' |

| | |
|---|---|
| S[ : 3] | 'HEL' |
| S[2: 5] | 'LLO' |
| S[4: : 2] | 'OOL' |
| S[2 : 7 : 2] | 'LOO' |
| S[ : : -1] | 'DLROWOLLEH' |
| S[ : : -2] | 'DRWLE' |
| S[6 : 1: -2] | 'OOL' |
| S[-8 : 8 ] | 'LLOWOR' |
| S[3 : -2 : 2] | 'LWR' |
| S[6 : -9 : -2] | 'OOL' |

**List -** A List is a standard mutable data type of python than can store a sequence of values belonging to any data type. Lists are enclosed in square brackets and the elements of the list are separated by commas.

**List Functions/Operations:**

| Sr. No. | Function Name / Syntax | Description | Example |
|---|---|---|---|
| 1 | Creating a List | | L1=[]<br>L2=eval(input('Enter a list:'))<br>print(L1)<br>print(L2)<br><br>o/p:<br>Enter a list:[1,2,3,4,5]<br>[]<br>[1, 2, 3, 4, 5] |
| 2 | Displaying a List | | L=[1,2,3,4,5]<br>print(L)<br><br>o/p:<br>[1,2,3,4,5] |
| 3 | Accessing / Changing list element | We can use forward indexing or backward indexing | L=['abc', 'def', 'ghi', 'jkl']<br>print(L[2] )<br>L[1]='xyz'<br>print(L)<br><br>o/p:<br>ghi<br>['abc', 'xyz', 'ghi', 'jkl'] |
| 4 | Creating True copy of list | assignment statement does not create a new list, both old and new variable points to same list in memory. To create new copy use list() method. | L1=[1,2,3,4]<br>L2 = L1<br>L2[1]=99<br>print(L1, L2)<br><br>L3=[11,12,13,14]<br>L4=list(L3)<br>L3[1]=99<br>print(L3, L4)<br><br>o/p:<br>[1, 99, 3, 4] [1, 99, 3, 4]<br>[11, 99, 13, 14] [11, 12, 13, 14] |
| 5 | len() | Finding number of elements in a List | L=['abc', 'def', 'ghi', 'jkl']<br>x=len(L)<br>print(x)<br><br>o/p:<br>4 |

| | | | |
|---|---|---|---|
| 6 | + | Joining/Concatenating two lists | L1 = [1,2,3]<br>L2=[7,8,9]<br>L3=L2+L1<br>print(L3 )<br><br>o/p:<br>[7, 8, 9, 1, 2, 3] |
| 7 | * | Replicating lists | L1 = [1,2,3]<br>L2=L1*3<br>print(L2)<br><br>o/p:<br>[1, 2, 3, 1, 2, 3, 1, 2, 3] |
| 8 | append() | used to add a single element at the end of a List. | L1 = [1,2,3]<br>L1.append(9)<br>print(L1)<br><br>o/p:<br>[1, 2, 3, 9] |
| 9 | extend() | used to add multiple elements that are present in another list to an existing list at the end | L1 = [1,2,3]<br>L2=[9,10]<br>L1.extend(L2)<br>print(L1)<br><br>o/p:<br>[1, 2, 3, 9,10] |
| 10 | listname.insert(pos,item) | used to add 'item' element at index pos in the list - listname. | L1 = [4,9,12,17,19]<br>L1.insert(2, 99) #insert 99 at index 2<br>print(L1)<br><br>o/p:<br>[4, 9, 99, 12, 17, 19] |
| 11 | del listname[index] | used to delete an element at a particular index location in the list. | L1 = [2,4,6,8,10]<br>del L1[3]<br>print(L1)<br><br>o/p:<br>[2, 4, 6, 10] |
| 12 | pop() | used to delete from either the end of the list(when no parameter is passed) or from a particular index(when a single index value is passed). Returns back the deleted element. | L1 = [2,4,6,8,10]<br>print('Now popping ', L1.pop())<br>print('List is', L1)<br>x=L1.pop(2)<br>print('Then popped', x)<br>print('List is now', L1)<br><br>o/p:<br>Now popping 10<br>List is [2, 4, 6, 8]<br>Then popped 6<br>List is now [2, 4, 8] |
| 13 | listname.remove(value) | used to remove an element containing a particular 'value'. if there are multiple elements in a list having the same 'value' then only the first occurrence of 'value' is removed. | L1 = [1,2,3,4,1,2,3,4,1,2,3,4]<br>L1.remove(2)<br>print(L1)<br><br>o/p:<br>[1, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4] |
| 14 | clear() | clears all the elements of the list and makes it an empty list | L1 = [1,2,3,4,1,2,3,4,1,2,3,4]<br>L1.clear()<br>print(L1)<br><br>o/p:<br>[] |

| | | | |
|---|---|---|---|
| 15 | count() | used to count the number of times an item appears in the list. | L1 = [1,2,3,4,1,2,3,4,1,2,3,4]<br>print(L1.count(2))<br>print(L1.count(75))<br><br>o/p:<br>3<br>0 |
| 16 | reverse() | reverses a list in place | L1 = [1,2,3,4]<br>L1.reverse()<br>print(L1)<br>o/p:<br>[4, 3, 2, 1] |
| 17 | sort() | sorts a list in ascending order. To sort in descending order pass the parameter - (reverse = True) | L1 = [9,3,2,4]<br>L1.sort()<br>print(L1)<br><br>L2=[2, 7, 21, 4]<br>L2.sort(reverse=True)<br>print(L2)<br><br>o/p:<br>[2, 3, 4, 9]<br>[21, 7, 4, 2] |
| 18 | *sorted(iterable, *, key=None, reverse=False)* | sorted() method works on any iterable sequence(such as a list, tuple, dictionary) and returns back a **list** of elements in sorted order. The original iterable object remains unchanged. The parameter reverse=True, if specified sorts in descending order. The 'key=str.lower' parameter when passed when sorting an iterable consisting of strings then the sort is performed in a case-insensitive manner. | L1 = [9,3,2,4]<br>**L2 = sorted(L1)**<br>print('L1=', L1)<br>print('L2=', L2)<br><br>L3=[2, 7, 21, 4]<br>**L4=sorted(L3,reverse=True)**<br>print('L3=',L3)<br>print('L4=',L4)<br><br>L5=['Pqr', 'abc', '2np']<br>**L6=sorted(L5)**<br>**L7=sorted(L5,key=str.lower)**<br>print('L5=',L5)<br>print('L6=',L6)<br>print('L7=',L7)<br><br>o/p:<br>*L1= [9, 3, 2, 4]*<br>*L2= [2, 3, 4, 9]*<br>*L3= [2, 7, 21, 4]*<br>*L4= [21, 7, 4, 2]*<br>*L5= ['Pqr', 'abc', '2np']*<br>*L6= ['2np', 'Pqr', 'abc']*<br>*L7= ['2np', 'abc', 'Pqr']* |
| 18 | min(), max(),sum() | Finds minimum, maximum , sum of elements of a list.<br>For min() and max() function all elements must be same data type and must be comparable to each other.<br><br>For sum() function all elements must be numbers | L1 = [16,55,3,4]<br>print(min(L1))<br>print(max(L1))<br>print(sum(L1))<br><br>o/p:<br>3<br>55<br>78 |
| 19 | index() | returns the index of first occurrence of element in list. If the list does not contain the element then it gives error. | L1 = [9,3,1,2,4, 7, 9, 2]<br>print(L1.index(2))<br><br>o/p:<br>3 |

| | | | |
|---|---|---|---|
| 20 | Traversing a list using the index | Note: The index method is used when we want to do some processing based on the index of the List or when we want to update the List *in-place.* | L1 = [9,3,2,4]<br>x=len(L1)<br>for i in range(x):<br>print(L1[i])<br>o/p:<br>9<br>3<br>2<br>4 |
| 21 | Traversing a list using the iterator | Note: The iterator method is used to quickly go through all the elements of the list and we DO NOT want to update the list *in-place*. | L1 = [9,3,2,4]<br>for i in L1:<br>print(i)<br>o/p:<br>9<br>3<br>2<br>4 |
| 22 | List Comparisons | Comparison done in similar manner to strings in element by element manner. | print( [1,2,3] == [1,2,3])<br>print( [1,2,3] != [1,2,3])<br>print( [1,2,8,9] < [9,1])<br>print( [1,2,8,9] >= [1,9])<br>print( [1,2] < [1,2,3])<br>print( [2] > [1,2,3])<br><br>o/p:<br>True<br>False<br>True<br>False<br>True<br>True |

<table>
<tr><td>23</td><td>List Slicing</td><td>Slicing concept similar to that of strings</td><td>L=[2, 7, 9, 10, 13, 15, 17, 19, 23, 25]<table><tr><th>Slice</th><th>Generates</th></tr><tr><td>L[4: ]</td><td>[13, 15, 17, 19, 23, 25]</td></tr><tr><td>L[ : 3]</td><td>[2,7,9]</td></tr><tr><td>L[2: 5]</td><td>[9,10, 13]</td></tr><tr><td>L[4:  : 2]</td><td>[13, 17, 23]</td></tr><tr><td>L[2 : 7 : 2]</td><td>[9, 13, 17]</td></tr><tr><td>L[ :  :  -1]</td><td>[25, 23, 19, 17, 15, 13, 10, 9, 7, 2]</td></tr><tr><td>L[ :  : -2]</td><td>[25, 19, 15, 10, 7]</td></tr><tr><td>L[6 : 1: -2]</td><td>[17, 13, 9]</td></tr><tr><td>L[-8 : 8 ]</td><td>[9, 10, 13, 15, 17, 19]</td></tr><tr><td>L[3 : -2 : 2]</td><td>[10, 15, 19]</td></tr><tr><td>L[6 : -9 : -2]</td><td>[17, 13, 9]</td></tr></table></td></tr>
<tr><td>24</td><td>Updating / Deleting multiple elements using List Slice</td><td>List slice can be used to update/delete multiple elements from list</td><td>L=[2,7,9,10,13,15,17,19,23,25]<br>L[1:5:2]=[77,99]<br>print(L)<br><br>del L[5: :2]<br>print(L)<br><br>o/p:<br>[2, 77, 9, 99, 13, 15, 17, 19, 23, 25]<br>[2, 77, 9, 99, 13, 17, 23]</td></tr>
<tr><td>25</td><td>in , not in</td><td>used to test whether a value is present / is not present in a list</td><td>L=[1,2,3,4,5]<br>x=int(input('Enter an element to search:'))<br>if x in L:<br>  print(x, 'is present in', L)<br>if x not in L:<br>  print(x, 'is not present in', L)<br><br>o/p1:<br>Enter an element to search:2<br>2 is present in [1, 2, 3, 4, 5]<br>o/p2:<br>Enter an element to search:20<br>20 is not present in [1, 2, 3, 4, 5]</td></tr>
<tr><td>26</td><td>Accessing Nested Lists</td><td>For accessing nested list member we can use two square brackets- the first showing the index in the outer list and the next showing the index for the inner list.<br><br>There can be any level of nesting within the list elements. Use as many [ ] as there are levels of nesting within the list.</td><td>L=[1,2,[3,4,5], [6,7,8] ]<br>print('L[0]=', L[0])<br>print('L[2]=', L[2])<br>print('L[2][1]=', L[2][1])<br>print('L[3][2]=', L[3][2])<br><br>o/p:<br>L[0]= 1<br>L[2]= [3, 4, 5]<br>L[2][1]= 4<br>L[3][2]= 8</td></tr>
</table>

**Tuple -** A Tuple is a standard immutable data type of python than can store a sequence of values belonging to any data type. Tuples are enclosed in round brackets i.e. parentheses and the elements of the tuple are separated by commas. A tuple with only a single element must have a comma after the single element otherwise the element is considered as a single element of that particular data type and not as a tuple of one element.

Eg. ( ) - empty tuple

(7,) - tuple containing only one element must have a comma after the element

(1,2,3) - tuple of integers

('abc', 'def', 'ghi' ) - tuple of strings

**Tuple Functions/Operations:**

| Sr. No. | Function Name / Syntax | Description | Example |
|---|---|---|---|
| 1 | Creating/Displaying a Tuple | Parentheses () is used to create a tuple.<br><br>If tuple is containing only one element, then that element must have a comma after it.<br><br>Simply placing multiple elements separated by commas after the = operator creates a tuple object<br><br>For accepting/creating a tuple directly from the user input, the eval() function is used. | # 1 Creating a Tuple<br>T1 = ()<br>T2 = (3,)<br>T3 = (1, 5, 'abc', 56.7)<br>T4= 'mn', 'pq', 'rs'<br>T5= 29,<br>T6 = eval(input('Enter a tuple:'))<br>print('T1=', T1)<br>print('T2=', T2)<br>print('T3=', T3)<br>print('T4=', T4)<br>print('T5=', T5)<br>print('T6=', T6)<br><br>*o/p:*<br>*Enter a tuple:(3,7,5,9)*<br>*T1= ()*<br>*T2= (3,)*<br>*T3= (1, 5, 'abc', 56.7)*<br>*T4= ('mn', 'pq', 'rs')*<br>*T5= (29,)*<br>*T6= (3, 7, 5, 9)* |
| 2 | Accessing tuple elements | We can use forward indexing or backward indexing to access tuple elements.<br><br>As tuple is immutable, we cannot change an individual element of a tuple. | # 2 Accessing Tuple elements<br>T = (2,7,9,13,19)<br>print(T[2] )<br>print (T[-4])<br><br>*o/p:*<br>*9*<br>*7* |
| 3 | len() method | The len() functions returns the number/count of elements present in a tuple. | # 3 len method<br>T = (2,7,9,13,19)<br>x=len(T)<br>print(x)<br><br>*o/p:*<br>*5* |
| 4 | Joining/Concatenating two Tuples | The + operator can be used to join/concatenate two tuples and create a new tuple. | # 4 Joining / Concatenating two Tuples<br>T1 = (1,2,3)<br>T2=(7,8,9)<br>T3=T2+T1<br>print(T3 )<br><br>*o/p:*<br>*(7, 8, 9, 1, 2, 3)* |

| | | | |
|---|---|---|---|
| 5 | Replicating Tuples | The * operator can be used to replicate tuples. One of the operands must be an int value. | # 5 Replicating Tuples<br>T1 = (1,2,3)<br>T2=T1*3<br>print(T2)<br><br>*o/p:*<br>*(1, 2, 3, 1, 2, 3, 1, 2, 3)* |
| 6 | Deleting tuple object | The del command can be used to delete an entire tuple object. | # 6 Deleting tuple object<br>T1 = (2,4,6,8,10)<br>del T1<br>#print(T1) |
| 7 | count() method | The count() method can be used to count the number of times an item appears in the tuple | # 7 count method<br>T1 = (1,2,3,4,1,2,3,4,1,2,3,4)<br>print(T1.count(2))<br>print(T1.count(75))<br><br>*o/p:*<br>*3*<br>*0* |
| 8 | min() max() sum() methods | The min() method returns the minimum element in a tuple and the max() method returns the maximum element in a tuple.<br><br>For min() and max() functions all elements must be same data type and must be comparable to each other.<br><br>For sum() function all elements must be numbers | # 8 min max methods<br>T1 = (1,2,3,4)<br>print(min(T1))<br>print(max(T1))<br>print(sum(T1))<br><br>*o/p:*<br>*1*<br>*4*<br>*10* |
| 9 | index() method | returns the index of the first occurrence of existing element of a tuple. If element does not exist then error is given. | # 9 index method<br>T1 = (9,3,1,2,4, 7, 9, 2)<br>print(T1.index(2))<br>#print(T1.index(99))  # will give error<br><br>*o/p:*<br>*3* |
| 10 | Packing and Unpacking tuples | Creating tuples by placing elements/variables in parentheses is called packing.<br><br>Reverse method of assigning tuple elements to individual variables is called unpacking. The number of variables on left side must match the number of elements of the tuple. | # 10 Packing and Unpacking tuples<br>a=5<br>b='hello'<br>c=9.5<br>t=(a,b,c)  #packing a tuple variable<br>print(t, type(t))<br><br>t1=(7, 'world', 2.7)<br>x,y,z=t1  #unpacking tuple object<br><br>print('x=', x)<br>print('y=', y)<br>print('z=', z)<br><br>*o/p:*<br>*(5, 'hello', 9.5) <class 'tuple'>*<br>*x= 7*<br>*y= world*<br>*z= 2.7* |

| | | | |
|---|---|---|---|
| 11 | Traversing a tuple using the index | The for loop can be used to go through all the valid indexes of the tuple and process the tuple. | # 11 Traversing a tuple using the index<br>T1 = (9,3,2,4)<br>x=len(T1)<br>for i in range(x):<br>print(T1[i])<br><br>*o/p:*<br>*9*<br>*3*<br>*2*<br>*4* |
| 12 | Traversing a tuple using the iterator | The for loop can be used to iterate over the elements of the tuple and process the tuple. | # 12 Traversing a tuple using the iterator<br>T1 = (9,3,2,4)<br>for i in T1:<br>print(i)<br><br>*o/p:*<br>*9*<br>*3*<br>*2*<br>*4* |
| 13 | Tuple Comparisons | Similar to Lists, for tuples also element by element comparison is done to arrive at a True or False value when relational operators >, >=, <, <=, ==, != is applied to two tuple objects | # 13 Tuple Comparisons<br>print( (1,2,3) == (1,2,3))<br>print( (1,2,3) != (1,2,3))<br>print( (1,2,8,9) < (9,1))<br>print( (1,2,8,9) >= (1,9))<br>print( (1,2) < (1,2,3))<br>print( (2,) > (1,2,3))<br><br>*o/p:*<br>*True*<br>*False*<br>*True*<br>*False*<br>*True*<br>*True* |
| 14 | Tuple Slicing | Tuple slicing follows rules similar to list slicing and creates a new tuple object. | # 14 Tuple Slicing<br>T = (2,7, 9,10,13,15,17,19,23, 25)<br>print(T[4: ])<br>print(T[ : 3])<br>print(T[2: 5])<br>print(T[4: : 2])<br>print(T[2 : 7 : 2])<br>print(T[ : : -1])<br>print(T[ : : -2])<br>print(T[6 : 1: -2])<br>print(T[-8 : 8 ])<br>print(T[3 : -2 : 2])<br>print(T[6 : -9 : -2])<br><br>*o/p:*<br>*(13, 15, 17, 19, 23, 25)*<br>*(2, 7, 9)*<br>*(9, 10, 13)*<br>*(13, 17, 23)*<br>*(9, 13, 17)*<br>*(25, 23, 19, 17, 15, 13, 10, 9, 7, 2)*<br>*(25, 19, 15, 10, 7)*<br>*(17, 13, 9)*<br>*(9, 10, 13, 15, 17, 19)*<br>*(10, 15, 19)*<br>*(17, 13, 9)* |

| | | | |
|---|---|---|---|
| 15 | in not in operators | The in and not in operators can be used to test whether a value is present / is not present in a tuple. | # 15 in not in operators<br>T=(1,2,3,4,5)<br>x=int(input('Enter an element to search:'))<br>if x in T:<br>  print(x, 'is present in', T)<br>if x not in T:<br>  print(x, 'is not present in', T)<br><br>*o/p:*<br>*Enter an element to search:7*<br>*7 is not present in (1, 2, 3, 4, 5)* |
| 16 | Accessing Nested Tuples | For accessing nested tuple member we can use two square brackets- the first showing the index in the outer tuple and the next showing the index for the inner tuple. | # 16 Accessing Nested Tuples<br>T=(1,2,(3,4,5), (6,7,8) )<br>print('T[0]=', T[0])<br>print('T[2]=', T[2])<br>print('T[2][1]=', T[2][1])<br>print('T[3][2]=', T[3][2])<br><br>*o/p:*<br>*T[0]= 1*<br>*T[2]= (3, 4, 5)*<br>*T[2][1]= 4*<br>*T[3][2]= 8* |
| 17 | *sorted(iterable, *, key=None, reverse=False)* | sorted() method works on any iterable sequence(such as a list, tuple, dictionary) and returns back a **list** of elements in sorted order. The original iterable object remains unchanged. The parameter reverse=True, if specified sorts in descending order. The 'key=str.lower' parameter when passed when sorting an iterable consisting of strings then the sort is performed in a case-insensitive manner. | T1 = (9,3,2,4)<br>L2 = sorted(T1)<br>print('T1=', T1)<br>print('L2=', L2)<br><br>T3=(2, 7, 21, 4)<br>L4=sorted(T3,reverse=True)<br>print('T3=',T3)<br>print('L4=',L4)<br><br>T5=('Pqr', 'abc', '2np')<br>L6=sorted(T5)<br>L7=sorted(T5,key=str.lower)<br>print('T5=',T5)<br>print('L6=',L6)<br>print('L7=',L7)<br><br>o/p:<br>*T1= (9, 3, 2, 4)*<br>*L2= [2, 3, 4, 9]*<br>*T3= (2, 7, 21, 4)*<br>*L4= [21, 7, 4, 2]*<br>*T5= ('Pqr', 'abc', '2np')*<br>*L6= ['2np', 'Pqr', 'abc']*<br>*L7= ['2np', 'abc', 'Pqr']* |

**Dictionary -** A Dictionary is a mutable collection with elements or items in the form of key:value pairs. Dictionary is enclosed in curly/brace brackets and the elements/items in the dictionary are separated by commas and each element of a dictionary consist of a key and value pair separated by a colon(:).
Eg. d={1 : 'abc', 5: 'xyz', 3 : 'pqr'}
The keys of a dictionary must be unique and of immutable data type (int,float, str,tuple)

**NOTE:** If a dictionary definition contains multiple elements with the same key then it does not give an error. Instead the last key(that is repeated) and value pair will overwrite the earlier key's value.
E.g. d={10:'a', 20:'b', 10:'c'} will not give error
print(d) will give output as {10:'c', 20:'b'}

**Dictionary Functions/Operations:**

| Sr. No. | Function Name / Syntax | Description | Example |
|---|---|---|---|
| 1 | Creating/Displaying a Dictionary | 1. Using dict() or {} to create empty dictionary.<br><br>2. using key:value pair inside brace<br><br>3. using keyword argument to the dict() method. The keywords are written without quotes<br><br>4. using zip function<br><br>5. using nested list/tuple<br><br>6. using eval function<br><br>7. using fromkeys() method- if second parameter is omitted then None is the value for all elements otherwise the value is the second parameter | # 1 Creating a dictionary<br><br>#a Creating empty dictionary<br>d1 = {}<br>d2= dict()<br>print('d1=', d1)<br>print('d2=', d2)<br><br>#b using key:value pair<br>d3 = {1 : 'abc', 5: 'xyz', 3 : 'pqr'}<br>print('d3=', d3)<br><br>#c using keyword argument<br>d4 = dict(name='abc', age=16, address='xyz')<br>print('d4=', d4)<br><br>#d using zip function<br>d5=dict (zip(('name','age','address') , ('abc',16,'xyz')))<br>print('d5=', d5)<br><br>#e using nested list/tuple<br>d6=dict([['name','abc'], ['age', 16], ['address','xyz']])<br>print('d6=', d6)<br><br>#f using eval function<br>d7=eval(input('Enter a dictionary:'))<br>print('d7=', d7)<br><br>#g using fromkeys() method<br>L=['abc', 'def', 'xyz']<br>d8=dict.fromkeys(L,'hello world')<br>print('d8=', d8)<br><br>*o/p:*<br>*d1= {}*<br>*d2= {}*<br>*d3= {1: 'abc', 5: 'xyz', 3: 'pqr'}*<br>*d4= {'name': 'abc', 'age': 16, 'address': 'xyz'}*<br>*d5= {'name': 'abc', 'age': 16, 'address': 'xyz'}*<br>*d6= {'name': 'abc', 'age': 16, 'address': 'xyz'}*<br><br>*Enter a dictionary:{'a':'hello', 'b':'world'}*<br>*d7= {'a': 'hello', 'b': 'world'}*<br>*d8= {'abc': 'hello world', 'def': 'hello world', 'xyz': 'hello world'}* |

| | | | |
|---|---|---|---|
| 2 | Accessing dictionary elements | Using the key, we can access the value using following methods:<br>a) using square brackets d[key], if key is not present then error is given<br><br>b) using get(key[,default]) - if key is not present then the second parameter is returned back. If second parameter is omitted then None is returned back. | # 2 Accessing dictionary elements<br>d={3 : 'abc', 'xyz': 5 , (2,3) : [55,88]}<br>print(d[3])<br>print(d[(2,3)])<br>m= d.get('xyz')<br>print(m)<br>n=d.get('pqr', 'hello world')<br>print(n)<br><br>*o/p:*<br>*abc*<br>*[55, 88]*<br>*5*<br>*hello world* |
| 3 | Adding/Modifying a Dictionary element | d[key]=value<br>statement can be used to add the key:value pair to the dictionary d. If key is already present then the value is modified. | #3 Adding/Modifying a Dictionary element<br>d={'abc' : 55, 'def' : 66, 'ghi': 77 }<br>d['xyz']=88<br>d['def']=101<br>print(d)<br><br>*o/p:*<br>*{'abc': 55, 'def': 101, 'ghi': 77, 'xyz': 88}* |
| 4 | Traversing a Dictionary | The for loop can be used to iterate over the keys of the dictionary. The values associated with a key can be accessed using the notation d[key] | # 4 Traversing a Dictionary<br>d = { 'abc' : 56, 'def' : 21 , 'xyz' : 54 }<br>for k in d:<br>print(k, d[k])<br><br>*o/p:*<br>*abc 56*<br>*def 21*<br>*xyz 54* |
| 5 | len() method | used the find the number of elements/items of a dictionary. | # 5 len() method<br>d = { 'abc' : 56, 'def' : 21 , 'xyz' : 54 }<br>print(len(d))<br><br>*o/p:*<br>*3* |

| | | | |
|---|---|---|---|
| 6 | Various methods of deleting from a dictionary | a) using del d[key] to delete an element of dictionary<br><br>b) using pop() method to get back the value that is being deleted.<br><br>c) using del dictobj<br>to delete the dictionary object<br><br>d) using clear() method to remove all elements of the dictionary.<br><br>e) using popitem() to delete the last element of dictionary and get it back as a tuple of (key,value) | # 6 Various methods of deleting from dictionary<br>d = { 'abc' : 56, 'def' : 21 , 'xyz' : 54 }<br>del d['def']<br>print('d=',d)<br><br>x= d.pop('xyz')<br>print(x, 'value is deleted')<br>print('d=',d)<br><br>del d<br>#print(d) #gives error as d object is not present<br><br>d2 = { 'abc' : 56, 'def' : 21 , 'xyz' : 54 }<br>m=d2.popitem() #last element in d is removed<br>print('datatype of m is',type(m))<br>print('m=', m)<br>print('d2=', d2)<br><br>d2.clear()<br>print('d2=', d2)<br><br>*o/p:*<br>*d= {'abc': 56, 'xyz': 54}*<br>*54 value is deleted*<br>*d= {'abc': 56}*<br>*datatype of m is <class 'tuple'>*<br>*m= ('xyz', 54)*<br>*d2= {'abc': 56, 'def': 21}*<br>*d2= {}* |
| 7 | in not in operators | in and not in operators can check for presence or absence of a key in the dictionary. | # 7 in not in operators<br>d = { 'abc' : 56, 'def' : 21 , 'xyz' : 54 }<br>if 'def' in d:<br>print('def present in dictionary')<br>else:<br>print('def present in dictionary')<br><br>if 'pqr' not in d:<br>print('pqr not present in dictionary')<br>else:<br>print('pqr present in dictionary')<br><br>*o/p:*<br>*def present in dictionary*<br>*pqr not present in dictionary* |
| 8 | keys() method | used to access all the keys of the dictionary object together as a sequence object which is usually converted into a list object for processing. | # 8 keys method<br>d = { 'abc' : 56, 'def' : 21 , 'xyz' : 54 }<br>kys=d.keys()<br>print(type(kys))<br>keys=list(d.keys())<br>print(type(keys))<br>print(keys)<br><br>*o/p:*<br>*<class 'dict_keys'>*<br>*<class 'list'>*<br>*['abc', 'def', 'xyz']* |

| | | | |
|---|---|---|---|
| 9 | copy() method | Assigning one variable to another dictionary variable refers to the same dictionary object in memory. copy() method is used to create a new copy of an existing dictionary object. | # 9 copy method<br>d1 = { 'abc' : 56, 'def' : 21 , 'xyz' : 54 }<br>d2 = d1 #both d1 and d2 refer to same object in memory<br>d2['def']=999<br>print('d1=', d1)<br>print('d2=', d2)<br><br>d3=d1.copy() #d3 and d1 refer to different objects in memory<br>d3['def']=222<br>print('d1=', d1) #remains unchanged<br>print('d3=', d3) #only d3 has 222 value for 'def' key<br><br>*o/p:*<br>*d1= {'abc': 56, 'def': 999, 'xyz': 54}*<br>*d2= {'abc': 56, 'def': 999, 'xyz': 54}*<br>*d1= {'abc': 56, 'def': 999, 'xyz': 54}*<br>*d3= {'abc': 56, 'def': 222, 'xyz': 54}* |
| 10 | values() method | used to access all the values of the dictionary object together as a sequence object which is usually converted to a list object for processing. | # 10 values() method<br>d = { 'abc' : 56, 'def' : 21 , 'xyz' : 54 }<br>vls=d.values()<br>print(type(vls))<br>vals=list(d.values())<br>print(type(vals))<br>print(vals)<br><br>*o/p:*<br>*<class 'dict_values'>*<br>*<class 'list'>*<br>*[56, 21, 54]* |
| 11 | items() method | used to access all the (key,value) pairs as a sequence of tuples, which can be iterated using for loop using either one control variable (tuple object) or using two control variables. | # 11 items method<br>d = { 'abc' : 56, 'def' : 21 , 'xyz' : 54 }<br>itms=d.items()<br>print(type(itms))<br><br>for t in itms: # using single control variable<br>print(t)<br><br>for k,v in itms: # using two control variables<br>print(k, v)<br><br>*o/p:*<br>*<class 'dict_items'>*<br>*('abc', 56)*<br>*('def', 21)*<br>*('xyz', 54)*<br>*abc 56*<br>*def 21*<br>*xyz 54* |
| 12 | update() method | used to add new elements to a dictionary or to update the values present in an existing dictionary from another dictionary object. | # 12 update method<br>d1= { 'abc' : 56, 'def' : 21 , 'xyz' : 54 }<br>d2={'def': 501, 'pqr': 777}<br>d1.update(d2) #d2 must be another dictionary<br>print(d1)<br><br>*o/p:*<br>*{'abc': 56, 'def': 501, 'xyz': 54, 'pqr': 777}* |

| | | | |
|---|---|---|---|
| 13 | setdefault(key[, default]) method | Used to return back the value associated with the key argument passed. In addition, if key is not present then it adds the key:default value as an element of the dictionary. If default(second parameter) is not specified then it takes a value of None. | # 13 setdefault method<br>d= { 'and' : 3, 'an' : 2 , 'the' : 5 }<br>x=d.setdefault('an')<br>print('x=',x)<br>y=d.setdefault('my')<br>print('y=',y)<br>z=d.setdefault('our', '22')<br>print('z=',z)<br>print('d=',d)<br><br>*o/p:*<br>*x= 2*<br>*y= None*<br>*z= 22*<br>*d= {'and': 3, 'an': 2, 'the': 5, 'my': None, 'our': '22'}* |
| 14 | sorted(iterable, *, key=None, reverse=False) | Sorted method can be used on any iterable object. When it is used with a dictionary, it sorts the keys of the dictionary in ascending order and returns back a List object. If sorting is to be done in case insensitive manner then key=str.lower parameter is passed. If descending order is needed then reverse=True is passed. | # 14 sorted method<br>d= { 'Xyz' : 'mmm', 'abc': 'nnn', 'Pqr': 'ppp'}<br>L1=sorted(d)<br>print('L1=', L1)<br>L2=sorted(d, key=str.lower)<br>print('L2=', L2)<br>L3=sorted(d, key=str.lower, reverse=True)<br>print('L3=', L3)<br><br>*o/p:*<br>*L1= ['Pqr', 'Xyz', 'abc']*<br>*L2= ['abc', 'Pqr', 'Xyz']*<br>*L3= ['Xyz', 'Pqr', 'abc']* |
| 15 | min() max() methods | min() returns the minimum key and max() returns the maximum key. | # 15 min max methods<br>d= { 'Xyz' : 'mmm', 'abc': 'nnn', 'Pqr': 'ppp'}<br>mx=max(d)<br>mn=min(d)<br>print('mx=', mx)<br>print('mn=', mn)<br>mx_case_insensitive = max(d, key=str.lower)<br>print('max value case insensitive=', mx_case_insensitive)<br><br>*o/p:*<br>*mx= abc*<br>*mn= Pqr*<br>*max value case insensitive= Xyz* |

# 3. Functions

**Functions -** Functions are a named unit of a group of program statements which can be called from other parts of the program.

**Why are Functions needed?**

- To group similar/logically related statements performing a single/similar task together
- To provide structure to a big program by dividing it into smaller modules. Programs are better organized and more readable
- To group statements that are repeated many times in a program under a single logical unit and then call this logical unit/function everytime. It reduces length of code
- To promote code reusability by reusing the function
- It makes debugging easier

**Types of Functions**

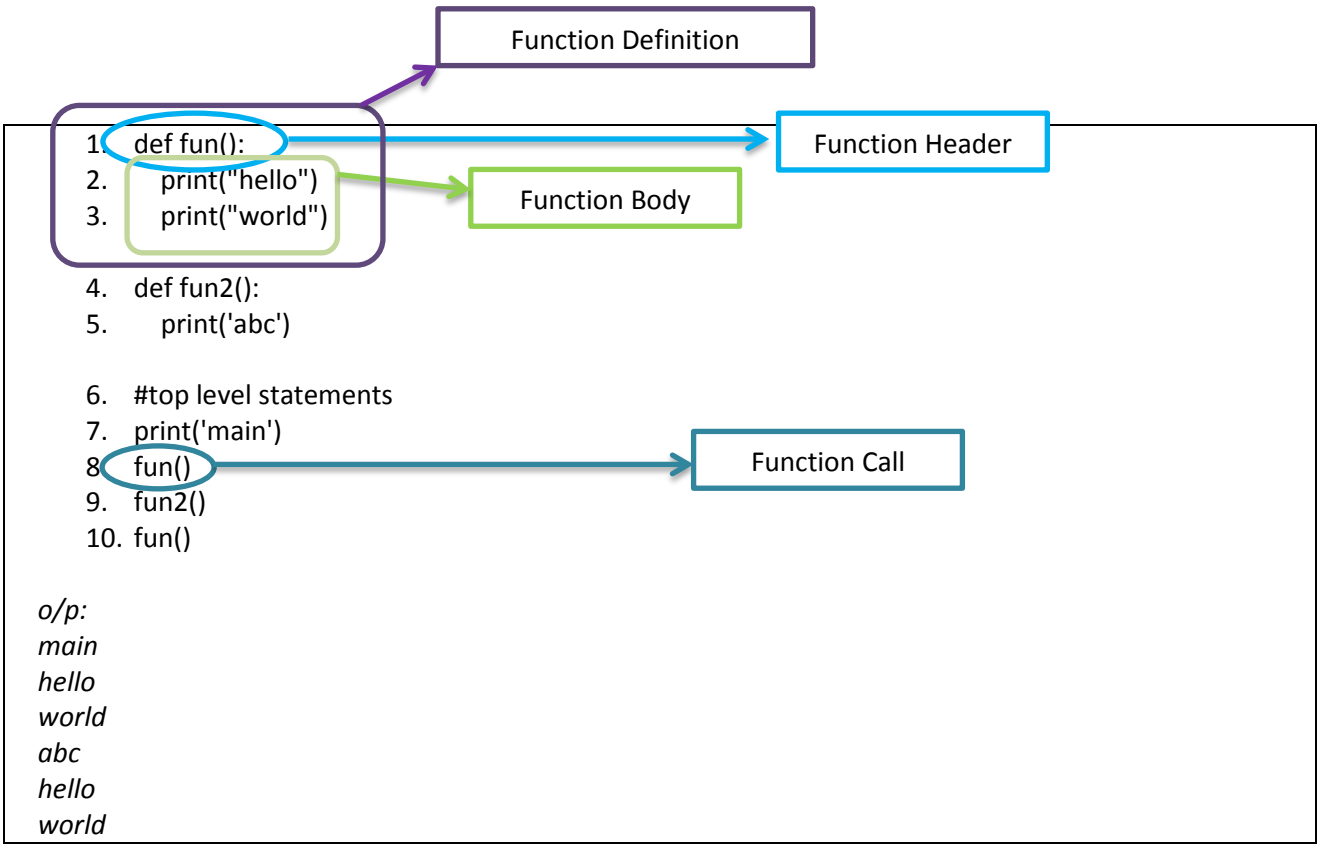
- **Built-In Functions-** These are an integral part of Python programming language and can be used anywhere in the program. e.g. input(), int(), print(), range() etc.
- **Library/Module Functions-** Python contains many libraries/modules for specific tasks. In order to use it we need to use the import libraryname command. After this the functions defined within the libraries/modules can be used
- **User defined Functions-** These are defined by the programmer to carry out some specific task. They are created using the def keyword.

**Creating User Defined Functions**

The def (short for define) keyword is used for creating user defined functions.
Syntax:

*def function_name([parameter1, parameter2,….]) :*
    *statement1*
    *statement2*
    *…*
    *…*
    *[return <value1, value2, …> ]*

**Program 1: Defining a Function with no parameters**




The ***def*** keyword is used to define a function. After the ***def*** the function name is specified. The function name is also an identifier and the rules that are applicable for identifier names are also applicable to function names. After the function name, the parentheses () must come and then a colon(:) appears. The first line of the function definition is also known as **function header**.

After the function header, all the statements indented at the first level under the function header and the nested statements within it become the **function body**. All the statements in the function body are executed whenever a function is called.

The statements that are at level zero or the first/top level of indentation form the main program. The execution of a python program starts at the first statement of the top level indentation. A function can be defined anywhere in the python script but the function call should always follow the function definition in the order of execution, as the python interpreter must correctly recognize the function name.

A function call in its most basic format is simply the function name followed by empty parentheses(). Whenever a function call is encountered the program control passes to the first statement in the function body and goes on execution till the function has finished execution/returned. Then control passes back to the statement that made the function call and the program execution continues.

**Order of Execution** of the program: In the above program statements are executed in the order 7-8-1-2-3-8-9-4-5-9-10-1-2-3-10

**Program 2: Defining a Function with parameters**

```
def findsum(x,y):                    #x,y are parameter
   z=x+y
   print(z)

a=int(input("enter number, a :"))
b=int(input("enter number, b :"))

findsum(a,b)                         #a,b are arguments
findsum('5','4')
```

*o/p:*
*enter number, a :2*

*enter number, b :3*
*5*
*54*

A function can be defined to receive values when it is called. In the function header within the parentheses () we define variable names, called **parameter**, which receive the values at the time of the function call. In the above program the variables x,y are the parameters of the function findsum.

In a function call the **argument** is the value that is passed to a function during a particular function call. In the above program the variables a,b as well as the string literals '5' and '4' form the arguments for the function findsum.

While defining a function as well as while making a function call care must be taken to see that the operations occurring within the function are done on compatible data types. This responsibility lies with the programmer and if an incompatible operation is found python will give an error.

**Functions containing Default parameters**

Usually when a function is called, the number of arguments must match exactly the number of parameters, otherwise python gives an error. But there is an exception, if a function is defined with a default parameter, then the function call can have less number of arguments than the number of parameters in the function header.

**Program 3: Function with default parameters**

```
def fun1(x,y,z=2):    # z is default parameter
   m= x/y-z
   print(m)

#def fun2(m=2,n,p):  # error default arguments must be rightmost
#    return m+n+p

a,b,c= 1,2,3
fun1(a,b,c)
fun1(a,b)
#fun1(b)   #error as at least two parameters must be passed
```

*o/p:*
*-2.5*
*-1.5*

In the above code the function header line - def fun1(x,y,z=2): the parameter z=2 has been specified with a default parameter. It means that this function, fun1 can be called with three arguments (e.g. fun1(a,b,c) ) or with two arguments [ e.g. fun1(a,b) ]. In the second case since the third argument has not been passed the third parameter z takes a default value of 2 and the function works with this default value of z.

If the function fun1 is called with one argument [e.g. fun1(b) ] then python gives an error as the function cannot be called with one argument.

*When a function definition has default parameters specified, then they must be the rightmost parameter.*

For e.g. if there is one default parameter, then the function header - def fun1(x, y, z=2): is correct whereas the function header - def fun2(m=2, n,p): is incorrect.

If there are two default parameters, then the function header - def fun1(x,y=1,z=2): is correct whereas the function headers - def fun1(x=1, y=2, z): , def fun1(x=1, y, z=2): are incorrect.

**Ways of Calling Functions**

There are two different ways of calling a function with parameters. They are using:

a) Positional Arguments
b) Keyword Arguments

**Positional Arguments**- When function calls are made, there is a one to one correspondence between the argument and the parameter based on the order/position in which the argument appears in the function call.

**Program 4: positional arguments**

```
# positional arguments
def fun1(x,y,z):
    m= x/y-z
    print(m)

a,b,c = 5,2,3
fun1(a,b,c)
fun1(4,b,b)
#fun1(a,b)   #Error as the third argument is missing

t=(7,2,5)
fun1(*t)     #using an iterable as positional argument

o/p:
-0.5
0.0
-1.5
```

In the above program function fun1 is defined with three parameter x, y, z and in that order.

When the function call fun1(a,b,c), is made,

i) the value of the first argument a is passed to the first parameter x
ii) the value of the second argument b is passed to the second parameter y
iii) the value of the third argument c is passed to the third parameter z

The function calls can be made with any supported argument/variable/literal and the argument can also be repeated as is shown in the function call, fun1(4,b,b).

There is one more way of specifying the positional arguments and that is by using an iterable object such as a tuple or a list. The tuple/list must have the same number of parameters as that of that of the function being called. The tuple t=(7,2,5) has been defined with three elements and we place a single * in front of the argument to let python know that it is an iterable positional argument.

**Keyword arguments-** In keyword arguments, when function calls are made the parameter name and the equal to symbol from the function header is used to specify the value/argument being passed.

**Program 5: keyword arguments**

```
#4 keyword arguments
def fun1(x,y,z):
    m= x/y-z
    print(m)

a,b,c= 1,2,3
fun1(y=a,x=b,z=a)       #using keyword arguments

fun1(7,z=a,y=b)         #using combination of positional & keyword arguments
#fun1(x=a,y=b)          #error missing one argument

d={'x': 10, 'y':5, 'z': 3}  #keys must match parameter names
fun1(**d)               #using dictionary for keyword argument
o/p:
1.0
2.5
-1.0
```

In the above program, the function fun1() has been specified with three parameters x, y, z. In keyword arguments, when making a function call the arguments are specified using the parameter names followed by the = symbol to pass values as shown in the function call - fun1(y=a,x=b,z=a). The advantage of using this method is that we can change the order of the parameters while calling a function.

A combination of positional and keyword arguments can also be used as shown in the function call - fun1(7,z=a,y=b).

*The rule is that all the positional arguments must be specified first and then the keyword arguments can follow the positional arguments.*

So function calls such as fun1(z=b, 4,x=a), will give an error as the positional argument 4, follows the keyword argument.

Another way of using keyword argument is by using a dictionary containing keys as the names of the parameters and the values of the dictionary are the arguments/values that need to be passed to the function. When using this method ** must precede the dictionary argument. All the parameters of the function must be present as keys of the dictionary object otherwise python will give an error.

**return keyword**

The return keyword is used within a function definition to signify that a function has ended. As soon as a return statement is encountered the function stops then and there and control is passed back to the line which made the function call.

**Program 6: return statement / Functions not returning any value**

```
#7 Return statement /Functions not returning any value
def fun1():
   print('hello')          #function with no return statement

def fun2():
   print('world')
   return
   print('Hi!!!') #unreachable code statement not executed

print('In main/top level')
fun1()
fun2()
x=fun1()
print('x=',x)

o/p:
In main/top level
hello
world
hello
x= None
```

In the above program the function fun1() is not defined with any return statement. In such situations python automatically considers the last statement that is executed during function run time as the return statement.

There can be many return statements within a function definition, returns embedded within loops, conditionals etc. During program execution when the first return statement is encountered then the function execution stops and control is passed back to the statement that made the function call. In the above program, for function fun2() the return is encountered and the statement print('Hi!!!') is never executed.

Whenever a function call is made a value is always returned back to the calling statement or line which made the function call. In the case of a function with an empty return or no return statement a value of None is returned back to the calling statement which is shown in the line x=fun1(). The displayed value of x is None.

**Functions returning a single value**

Usually functions are used to compute some values and give back a single answer. The return statement can be used to return back a single value or an expression that evaluates to a single value by placing that variable name/expression after the return statement.

**Program 7: Functions returning a single value**

```
#5 Functions returning single value
def fact(x):
    mult=1
    for i in range(1,x+1):
        mult=mult*i
    return mult             #Function returning a single value

def sum(x,y):
    return x+y              #Function returning a single value in form of expression

a=int(input('Enter an integer:'))
b=fact(a)          #assigning the returned value to a variable
print('Factorial of ', a , 'is',b )
print('2+3=', sum(2,3))  #using returned value in expression

o/p:
Enter an integer:5
Factorial of  5 is 120
2+3= 5
```

In the above program the function fun1() is calculating the factorial of the number that is passed and returning back a single value contained in the variable mult.

The returned value can be assigned to a variable [ e.g. b=fact(a) ], it can be used in an expression[e.g. b= 3+ fact(a) ], it can be directly used in the print statement [ e.g. print(fact(5)) ] etc. In short wherever we can use a literal value we can use a function call to make use of the value that was returned.

**Function returning multiple values**

A function can return multiple values which is written after the return statement and the values are separated by commas. The multiple return values are returned as a tuple object to the statement that made the function call.

**Program 8: Function returning a multiple values**

```
# 8 Functions returning multiple values
def fun1(x,y,z):
   return x+y,y+z,z+x

a,b,c= 3,5,9
#multiple returned values accepted as a tuple
m = fun1(a,b,c)
print(m)

#multiple returned values accepted in multiple variables
p,q,r=fun1(10,11,12)
print(p,q,r)

o/p:
(8, 14, 12)
21 23 22
```

The returned tuple object can either be accepted as a single tuple [ e.g. m = fun1(a,b,c) ] or the tuple object can be unpacked into different variables [ e.g. p,q,r=fun1(10,11,12) ]. When using the unpacking method the number of variables must match the number of values returned by the function.

**Scope of a variable**

Scope refers to the parts of the program in which a particular variable can be accessed. Scope can be of two types - local and global.

**Local variables-** A variable defined within a function is said to have local scope. A local variable can only be accessed within that function. Local variables are created at the time of function call and destroyed when the function returns.

The parameters of a function can also be considered to have local scope and can only be accessed within a function.

**Program 9: Local variables**

```
#local variables
def fun1():
   a=2     # a is local to fun1
   print(a)

def fun2(c):#parameter c is local to fun2() not accessible outside
   print(c)

fun1()
fun2(4)
#print('a=',a)  #error a is not accessible outside fun1()
#print('c=',c)  #error c is not accessible outside fun2()

o/p:
2
4
```

The variable a is local to fun1() and variable c is local to fun2() and if variables a or c are tried to be accessed in the main program then it gives an error.

**Global variable-** A variable defined outside of any function at the topmost level is said to have global scope. A global variable can be accessed from the statement following the global variable declaration till the end of the program.

**Program 10: Global variables**

```
#global variables
a=5                             #a is a global variable and can be accessed from here onwards

def fun1():
   print('In fun1 a :', a)      #ok
   #print(b)                    #error as b is tried to be accessed before being created

def fun2():
   print('In fun2 b :', b)

#main program
fun1()
b=11    #b is also global but it can be accessed from here onwards only
fun2()

o/p:
In fun1 a : 5
In fun2 b : 11
```

In the above program the variables a and b are defined at the topmost indentation level and so are global variables. The global variables can be accessed from the next statement onwards only. So if global variable b is tried to be accessed before being declared, inside function fun1- [ print(b) statement ] it will give an error.

**What happens when same variable name is used globally as well as locally?**

If a variable name is defined using an assignment statement inside a function as well as declared globally also using an assignment statement i.e. both local and global variables with same name are present then both variables are entirely different having different memory addresses (id's). Within a function the local variable has precedence and outside a function the global variable is used.

**Program 11: same variable name in global as well as local scope**

```
# same variable name in global as well as local
x=5  #this x is global x

def fun1():
   x=10     #this x is local
   print('In fun1 x:', x)

print('In main before fun1 called x:',x)
fun1()
print('In main after fun1 called x:',x)
x=x+11
print('In main after x updated, x:',x)
```

*o/p:*

*In main before fun1 called x: 5*
*In fun1 x: 10*
*In main after fun1 called x: 5*
*In main after x updated, x: 16*

**Problem in assigning/updating a global variable inside a function**

A peculiar problem arises in python due to the behaviour of the assignment operator. The assignment operator has dual use. It is used to assign a value to a variable as well as it checks if a variable is not present in memory then it tries to first create the variable and then assign it a value.

**Program 12: problem in assigning/updating global variable in function**

```
# problem in assigning/updating global variable in function
x=5  #this x is global x
def fun1():
   #error left side of = tries to create a value
   #while right side expects a local variable  x
   x=x+2

print(x)
fun1()
```

*o/p:*

```
  File "I:/ function.py", line 6, in fun1
    x=x+2

UnboundLocalError: local variable 'x' referenced before assignment
```

The line x=5 makes x a global variable. When function fun1() is called, due to the behaviour of the assignment operator, python tries to create a local variable x, but on evaluating the rhs side of the = operator, it doesn't find the value for the expression 'x+2' in which it tries to find also the value of the local variable x which has till now not been created.

So while using global variable (eg. x) inside function definitions, statements involving global variable on both lhs side as well as rhs side should not be written [ e.g. x = 7 * x,  x += y etc.]

In addition if a global variable is used with an assignment operator inside a function then a local variable is automatically created (*we cannot use an assignment operator to change the value of a global variable inside a function, doing so creates a local variable which has no effect on the global variable*)

```
x=5

def f1(a,b):
   x=a+b     #x is a local variable, it doesn't effect global x

f1(10,12)
print(x)
```

*o/p:*

```
5
```

**global keyword**

We can prevent a local variable from being created and instead use the global variable itself within a function by using the global keyword as -
global varname

**Program 13: using a global variable in local scope**

```
# using a global variable in local scope
1.  x=5
2.  def fun1():
3.          global x
4.          x=x+2
5.          print('In fun1 x:', x)

6.  print('In main before fun1 called x:',x)
7.  fun1()
8.  print('In main after fun1 called x:',x)
9.  x=x+11
10. print('In main after x updated, x:',x)
```

*o/p:*
*In main before fun1 called x: 5*
*In fun1 x: 7*
*In main after fun1 called x: 7*
*In main after x updated, x: 18*

The order of execution is 1-6-7-2-3-4-5-7-8-9-10

We are using the same program as the previous example with a slight modification at line number 3 where we have added the statement global x. This statement when it is executed says that the global variable of x must take precedence in the local scope and if any reference to x is made from that point onwards within the function, then it refers to the global variable x.

**Program 14: global keyword example 2**

```
1.  def fun1():
2.          global x
3.          x=51
4.          print('x=',x)
5.
6.  x=23
7.  fun1()
8.  print('x=',x)
```

*o/p:*
*x= 51*
*x= 51*

The order of execution is 6-7-1-2-3-4-7-8

Initially at line 6 the global variable x is created with value 23. Then at line 2 within the function, the statement - global x, says that within that function fun1() if x variable is reference from that point onwards that x will refer to the global x and if any value is updated for the global x then it will be reflected everywhere in the program including the main program. So outside the function also at line 8 the updated value of the global variable x is displayed in the output.

**Passing immutable arguments to a function**

There is a difference in behaviour of a function depending on whether the argument passed to a function is immutable or mutable. When immutable datatypes (int,float,string,tuple) are passed as arguments to a function, then any changes made to the corresponding immutable parameters are not reflected back in the main/calling program.

**Program 15: Passing immutable arguments to a function**

```
# passing immutable arguments to a function
def fun1(x):
    x=x+2
    print("Inside fun1, x:", x)

a=100
print("In main, a:", a)
fun1(a)
print("In main after function call, a:", a)

o/p:
In main, a: 100
Inside fun1, x: 102
In main after function call, a: 100
```

In the above program the variable a is int, i.e. an immutable data type. It is passed as an argument to the function fun1() and inside fun1() the value is changed. This changed value is not reflected back in the calling/main program and the value of the variable a, still remains unchanged at 100.

**Passing mutable datatype arguments to a function**

When mutable datatypes (lists, dictionary) are passed as arguments to a function, then -

(a) Changes made inside the function to a mutable object are reflected back in the main/calling program if inside the function changes/additions/deletions are made to the same object in memory

(b) Changes are NOT reflected back in the calling/main program if inside the function, the object is assigned another value of same datatype using the assignment operator or if the variable assigned a different datatype value using the assignment operator.

**Program 16: Passing mutable arguments to a function**

```
# 16 passing mutable arguments to a function
def fun1(x):
   m=len(x)
   for i in range(m):
      #update list (index method)
      x[i] += 10
   print("list inside fun1 ", x)

def fun2(y):
   y=[101,102]
   print("Inside fun2 y=",y)

a=[1,2,3,4]
print("list a before fun1() call", a)
fun1(a)
print("list a after fun1() call", a)

b=[31,32, 33, 34]
print("list b before fun2() call", b)
fun2(b)
print("list b after fun2() call", b)
```

*o/p:*

*list a before fun1() call [1, 2, 3, 4]*
*list inside fun1  [11, 12, 13, 14]*
*list a after fun1() call [11, 12, 13, 14]*

*list b before fun2() call [31, 32, 33, 34]*
*Inside fun2 y= [101, 102]*
*list b after fun2() call [31, 32, 33, 34]*

In the above program the list a is *[1, 2, 3, 4]* before the function call. Inside fun1(), the index method of iteration is used to update the values of the list object. Since the same list object is modified, the updated values are reflected back in the calling program and it shows the updated value *[11, 12, 13, 14]*.

List b in the main program is *[31, 32, 33, 34]*. Inside function fun2(), the parameter y which receives the list object b as the argument is assigned a different list object *[101, 102].* Hence as the parameter is assigned a different object, the changes are not reflected back in the calling program and in the main program the list object still has the original value *[31, 32, 33, 34]*.

*[Note: If we use the iterator method in the main program or within any function it does not change the value of the list elements*

*Eg.*

| | |
|---|---|
| *L=[1,2,3,4]*<br>*print('L=', L)*<br>*for i in L:*<br>*i += 3*<br>*print('L=', L)*<br><br>*o/p:*<br>*L= [1, 2, 3, 4]*<br>*L= [1, 2, 3, 4]* | *def fun3(z):*<br>*for i in z:*<br>*#updating the iterator variable*<br>*i += 7*<br><br>*c=[51,52,53,54]*<br>*print("list c before fun3() call", c)*<br>*fun3(c)*<br>*print("list c after fun3() call", c)*<br><br>*o/p:*<br>*list c before fun3() call [51, 52, 53, 54]*<br>*list c after fun3() call [51, 52, 53, 54]* |

*]*

**LEGB rule**

LEGB stands for Local - Enclosing - Global - Bulit-In rule. Whenever a variable is accessed anywhere in the program, python searches/uses the variable in the following order-

1. Local Scope - Local scope means the variables defined within a function or the parameters of a function.
2. Enclosing scope - If we have a function within another function, and a variable name is used in the inner function, then the block of code of the outer function forms the enclosing scope.
3. Global scope - It means variables defined at topmost/zero indentation level.
4. Built-In - If variable names are imported into our current program by using the import statement, then it the last place where python searches for variable names.

**Programs demonstrating LEGB rule**

```
1. def f1():
2.         def f2():
3.                 pi=21
4.                 print('f2 pi:',pi)   #pi=21 is local
5.         f2()
6. f1()
```

Order of execution is - 6-1-5-2-3-4-5-6

*o/p:*
*f2 pi: 21*

```
def f1():
   pi=10
   def f2():
      print('f2 pi:',pi)  #pi=10 is in enclosing scope
   f2()
f1()
```

o/p:
f2 pi: 10

```
pi=5
def f1():
   def f2():
      print('f2 pi:',pi) #pi=5 is in global scope
   f2()
f1()
```

*o/p:*
*f2 pi: 5*

```
from math import pi
def f1():
   def f2():
      print('f2 pi:',pi) # pi=3.14159 is in Built-in scope
   f2()
f1()
```

*o/p:*
*f2 pi: 3.141592653589793*

```
from math import pi

pi=5
def f1():
   pi=10
   def f2():
      pi=21
      print('f2 pi:',pi)

   print('f1 pi:',pi)
   f2()
print('main pi:',pi)
f1()
```

*o/p:*
*main pi: 5*
*f1 pi: 10*
*f2 pi: 21*

```
from math import pi

def f1():
   pi=10
   def f2():
      print('f2 pi:',pi)

   print('f1 pi:',pi)
   f2()
print('main pi:',pi)
f1()
```

*o/p:*
*main pi: 3.141592653589793*
*f1 pi: 10*
*f2 pi: 10*

# 4. Using Built-In Functions and Standard Python Libraries

**Using built-in python functions-**

Built-in functions are built into the core python programming language and do not need any import statement in order to execute the function. We have already seen/used many built in functions already, such as-

1. print() - used to display output
2. input() - used to accept user input
3. min() - used for finding minimum of a sequence
4. max() - used for finding maximum of a sequence
5. len() - used to find the number of elements within a sequence/iterable object.
6. abs() - used to find the absolute(positive) value of any number.
7. pow(x,y) - used to find $x^y$
8. sum() - used to find the sum of a numeric sequence object.

Some of the other built in python functions are illustrated in the following program:

**Program 6: using pythons built in functions**

```
# 7 using pythons built in functions
x=20
#oct method
y=oct(x)
print('y=', y, 'dtype of y', type(y))

#hex method
z=hex(x)
print('z=', z, 'dtype of z', type(z))

#int method
m=int(12.5)
print('m=', m)

#abs method
r=abs(-4.5)
print('r=',r)

#round method
n=round(12.53456, 2)
print('n=', n)
m=round(12345.456,-2)
print('m=', m)

#join method
s1='hello'
L1=['have', 'a', 'nice', 'day']
s2='$$$'.join(s1)
print('s2=', s2)
s3='###'.join(L1)
print('s3=', s3)

#split method
s4="have a              nice day"
L2=s4.split()
```

```
print('L2=', L2)
L3=s4.split('e')
print('L3=', L3)

#replace method
s5=s4.replace('day', 'night')
print('s5=', s5)
```

*o/p:*

```
y= 0o24 dtype of y <class 'str'>
z= 0x14 dtype of z <class 'str'>
m= 12
n= 12.53
m= 12300.0
r=4.5
s2= h$$$e$$$l$$$l$$$o
s3= have###a###nice###day
L2= ['have', 'a', 'nice', 'day']
L3= ['hav', ' a                nic', ' day']
s5= have a nice night
```

The use of the built in function are:

1. **oct()** - it gives a string representation of a number in octal format
2. **hex()** - it gives a string representation of a number in hexadecimal format
3. **int()** - it converts to an integer
4. **abs()**- converts negative number to positive number.
5. **round(x, n)** - it performs rounding operation on a floating number x with an optional second parameter n.
   if n is omitted then it performs rounding to the nearest integer value
   if n is positive then it round to n decimal places
   if n is negative then it rounds to the nearest tens, hunderds, thousands … depending on n=-1,-2,-3,…
   *[Note: Sometimes due to the inaccuracies of floating point number Python rounds fraction having 5 as the next decimal digit down to 0*
   *For eg. round(7.55,1) should give answer as 7.6 but python gives answer as 7.5 only*
   *in other cases it works fine - round(8.75,1) gives answer correctly as 8.8 ]*
6. **join()** method- The join method accepts an iterable as a parameter and returns back a new string in which in between each element of s2 the string s1 is inserted.
   (a) **s1.join(s2)** - when **string s2** is passed as a parameter, then in between each character of the string s2, the string s1 is inserted.
   (b) **s1.join(L)** - When **L** is a **list of strings**, then in between each element of the list L, the string s1 is inserted.
7. **split()** method- The split() method accepts a string as a parameter and returns back a **list** of substrings. The string parameter is split based on the separator parameter passed. If no parameter is passed then the separator parameter is a space ' ' and **all the continuous spaces** in the string are removed and returning back only a list of words without spaces.
8. **replace()** method- It works on a string and is used to replace one substring with another substring and return back a new string.

**Using Python Standard Libraries**

The standard distribution of python contains several modules which are useful for many kinds of applications. Some of which are the math module, random module and the statistics module.

We can use standard modules in the following ways-

**a) import statement-**

We can use the import statement as – ***import modulename***

The import statement can be written in any part of the program prior to the function/attribute inside the module being used is called. The standard way is to write the import statement as the first line of code in the script file. If multiple modules/libraries are being imported, then they can be written in the same line and the module name being separated by commas –

***import module1, module2***

Whenever we are using the import statement, we must use the complete name for the attribute/function contained within that module. For example – if we want to access the ceil function present within the math library, then the import statement will be-

***import math***

And the function call for the ceil() function within the math library will be-

***x=math.ceil(9.2)***

Another flexibility that python provides is then we can use an alias for the module/library name while using the import statement as follows –

***import math as mt***

So that the call to the ceil() function will now need to be written as-

***x=mt.ceil(9.2)***

This facility of using an alias for library/module name is especially useful if the library name is quite long and needs to be shortened. When we use an alias for a module/library then we must use the alias name in our current program. If we use the original module name then python will raise an error.

**b) from statement-**

We can load only the needed attributes/functions present within a module/library by means of the ***from*** statement. The advantage of the *from* statement is that if we have a large library then we can add only the function names that we need into our current program, which makes our program more efficient. Another advantage is that we can directly use the attribute/function name from the library instead of using the *libraryname.functionname* notation for calling.

For example- On using the following statement in our program:

***from math import ceil***

we can directly make a call to ceil in our program as:

***x=ceil(9.2)***

**Using math module**

The python math module has several useful constants and functions that can be useful in scientific and other applications.

**Program 7: using math module**

```
#7a using math library
import math

print('ceil(-9.7)=',math.ceil(-9.7))
print('ceil(9.7)=',math.ceil(9.7))

print('floor(-4.5)=',math.floor(-4.5))
print('floor(4.5)=',math.floor(4.5))

print('fabs(-6.7)=',math.fabs(-6.7))
print('fabs(6.7)=',math.fabs(6.7))

print('factorial(5)=',math.factorial(5))

print('fmod(4,4.9)=',math.fmod(4,4.9))
print('fmod(-4.9,2.5)=',math.fmod(-4.9,2.5))

print('gcd(12,8)=',math.gcd(12,8))
#no lcm function available

print('pow(2,3.2)=',math.pow(2,3.2))
print('sqrt(5.4)=',math.sqrt(5.4))
print('sin(1.5)=',math.sin(1.5))
```

*o/p:*

```
ceil(-9.7)= -9
ceil(9.7)= 10
floor(-4.5)= -5
floor(4.5)= 4
fabs(-6.7)= 6.7
fabs(6.7)= 6.7
factorial(5)= 120
fmod(4,4.9)= 4.0
fmod(-4.9,2.5)= -2.4000000000000004
gcd(12,8)= 4
pow(2,3.2)= 9.18958683997628
sqrt(5.4)= 2.32379000772445
sin(1.5)= 0.9974949866040544
```

Some of the constants defined in math module are:

| Sr. No. | Constant | Description | Example |
|---|---|---|---|
| 1 | math.pi | The mathematical constant π = 3.141592.... subject to the available precision of the system | *print(10* math.pi)*<br>*Output:*<br>*31.41592653589793* |
| 2 | math.e | The mathematical constant e = 2.718281.... subject to the available precision of the system | *print(10* math.e)*<br>*Output:*<br>*27.18281828459045* |

The uses of the mathematical function are:

| Sr. No. | Function | Description | Example |
|---|---|---|---|
| 1 | math.ceil(x) | It returns the smallest integer number that is larger than x | *math.ceil(-9.7)= -9*<br>*math.ceil(9.7)= 10* |
| 2 | math.floor(x) | It returns the highest integer number smaller than x | *math.floor(-4.5)= -5*<br>*math.floor(4.5)= 4* |
| 3 | math.fabs(x) | It returns the absolute/positive value of x | *math.fabs(-6.7)= 6.7*<br>*math.fabs(6.7)= 6.7* |
| 4 | math.factorial(x) | It returns the factorial of the positive integer x | *math.factorial(5)= 120* |
| 5 | math.fmod(x,y) | It returns the modulus of x%y. Both x and y can be positive or negative floating numbers. The sign of x is preserved in the answer. | *math.fmod(4,4.9)= 4.0*<br>*math.fmod(-4.9,2.5)= -2.4* |
| 6 | math.gcd(x,y) | It returns the gcd of the positive numbers x and y. | *math.gcd(12,8)= 4* |
| 7 | math.pow(x,y) | It returns the $x^y$ for any float x and y. | *math.pow(2,3)= 8* |
| 8 | math.sqrt(x) | It returns the square root of the positive number x | *math.sqrt(5.4)= 2.32379000772445* |
| 9 | math.sin(x) | It returns the sine of the angle x expressed in radians | *math.sin(1.5)= 0.9974949866040544* |

**Generating Random Numbers in Python using the Random Library**

Pseudo-random numbers can be generated in Python using the *random* library. This module uses the mersenne-twister for generating random numbers from another number known as the seed. By default, the current system time in milliseconds is used as a seed.

If the seed happens to be the same then the same set of numbers are generated. If the seed is available to anybody else, then the same set of random numbers can be generated by anybody.

Hence the random module should not be used for security or cryptographic purposes, instead the *secrets* module should be used which uses **true** random number generators.

**Functions in the random library**

| Sr. No. | Function | Description | Example |
|---|---|---|---|
| 1 | random.seed(x) | sets the seed of the random number. By default python uses current system time in milliseconds. Under normal circumstances we don't need to use/set it. | random.seed(35) |
| 2 | random.random() | Return the next random **floating point** number in the range [0.0, 1.0) i.e. in the interval which includes 0 but excludes 1.0. | random.random()<br>*o/p:*<br>*0.5207009136762631* |
| 3 | random.uniform(*a*, *b*) | Return a random **floating point** number *N* such that a <= N <= b | random.uniform(2.5,3.7)<br>*o/p:*<br>*3.0635845279514045* |
| 4 | random.randrange (stop) | Returns a random **int** in the range [0,stop) | random.randrange(7)<br>*o/p:*<br>*4* |
| 5 | random.randrange (start,stop[,step]) | Returns a random **int** from the **slice/range** of int (start,stop,step) | random.randrange(2,12,3)<br>*o/p:*<br>*5* |
| 6 | random.randint(a,b) | Returns a **random int** N such that a <= N <= b | random.randint(3,9)<br>*o/p:*<br>*7* |
| 7 | random.choice(L) | Returns **one** random element from the non-empty sequence/list L | L=['apple','mango',<br>'banana','orange','grape']<br>print(random.choice(L))<br>*o/p:*<br>*grape* |
| 8 | random.choices(L,k=1) | Returns a **list of k elements** from a sequence/list L. The k must be integer. The returned elements ***can be repeated***. | L=['apple','mango',<br>'banana','orange','grape']<br>print(random.choices(L,k=3))<br>*o/p:*<br>*['mango','mango','grape']* |
| 9 | random.shuffle(L) | Shuffles the elements of the mutable sequence (lists/dictionary) **inplace**. | L=['apple','mango',<br>'banana','orange','grape']<br>random.shuffle(L)<br>print(L)<br>*o/p:*<br>*['banana', 'grape', 'orange',<br>'apple', 'mango']* |
| 10 | random.sample(L,k) | Returns **a list of randomly selected k elements** from any sequence object (list/tuple/ dictionary/string). Here the selected element will **never appear twice** in the returned list. | t=('shirt','pant','tshirt','frock','<br>socks')<br>print(random.sample(t,3))<br>*o/p:*<br>*['frock', 'tshirt', 'shirt']* |

**statistics module/library**

statistics module provides functions for calculating mathematical statistics of numeric data.

**Functions in the statistics library**

| Sr. No. | Function | Description | Example |
|---|---|---|---|
| 1 | statistics.mean(L) | It returns the arithmetic mean of the list/iterable object L. | import statistics<br>L=[4,7,8]<br>print(statistics.mean(L))<br>***o/p:***<br>6.333333333333333 |
| 2 | statistics.median() | It returns the median (middle value) of the list/iterable object L when sorted. If the number of elements in L is odd then the center value in the sorted list L is returned. If the number of elements in L is even then the average of the two center values is returned. | import statistics<br>print(statistics.median([3,2,5,1,4]))<br>print(statistics.median([2,4,1,3]))<br>***o/p:***<br>*3*<br>*2.5* |
| 3 | statistics.mode() | It returns the mode, i.e. the data value that is repeated most number of times.<br><br>In python version 3.8 and later if the data contains more than one mode then the first mode that is encountered in the data is returned. In earlier versions of python if multiple modes are encountered then python returns an error. | import statistics<br>print(statistics.mode([2,2,2,1,1,3,3,3,3]))<br>*o/p:*<br>*3* |

# 5. Exception Handling

**Exceptions**

Errors detected during execution are called exceptions in Python. Python uses them to indicate that something exceptional has occurred, and that the program cannot continue unless it is handled.

Most exceptions are not handled by programs, and result in program termination with error messages such as shown below:

1.
**10*(1/0)**
Traceback (most recent call last):

File "<ipython-input-3-fbc7e86ae71a>", line 1, in <module>
10*(1/0)

**ZeroDivisionError**: division by zero

2.
**4 + age/3**
Traceback (most recent call last):

File "<ipython-input-4-5de9b54978d0>", line 1, in <module>
4 + age/3

**NameError**: name 'age' is not defined

3.
**2 + '2'**
Traceback (most recent call last):

File "<ipython-input-5-82150feed70c>", line 1, in <module>
2 + '2'

**TypeError**: unsupported operand type(s) for +: 'int' and 'str'

The last line of the message indicates what happened. Exceptions are of different types and the type of exception is also printed. Some of the Exception types are:

| S.No. | Exception Name | Description |
|---|---|---|
| 1. | SyntaxError | Raised when python syntax rules are violated by a statement |
| 2. | AssertionError | Raised when an *assert* statement fails. |
| 3. | EOFError | Raised when the input() function hits and end-of-file condition (EOF) without reading any data |
| 4. | ImportError | Raised when the import statement cannot load a module |
| 5. | IndexError | Raised when a sequence subscript is out of range. |
| 6. | KeyError | Raised when a dictionary key is not found in the set of existing keys. |
| 7. | KeyboardInterrupt | Raised when the user hits the interrupt key (normally the *Control+C* or *Delete*) during program execution. |
| 8. | MemoryError | Raised when an operation runs out of memory |
| 9. | NameError | Raised when a local or global name is not found |
| 10. | IndentationError | Raised when there is incorrect use of indentation |

| | | |
|---|---|---|
| 11. | TypeError | Raised when an operator or function is applied to an object of inappropriate type. |
| 12. | ValueError | Raised when an operator or function receives a parameter that has the right type but inappropriate value. |
| 13. | ZeroDivisionError | Raised when the second argument of a division or modulo operation is zero. |
| 14. | IOError | Raised when a file specified cannot be opened. |
| 15. | OverFlowError | Raised when a result of numeric computation cannot be accommodated in a numeric object. |

**try – except statements**

To handle possible exceptions, the try-except block of code is used.

**Program 1: try and except statements**

```
try:
   mrk = int(input("Enter your marks: "))
   print("I see that you have got" , mrk, 'marks' )
except ValueError:
   print("You have not entered a number")
print('Goodbye')
```

*o/p1:*
*Enter your marks: 20*
*I see that you have got 20 marks*
*Goodbye*

*o/p1:*
*Enter your marks: abc*
*You have not entered a number*
*Goodbye*

Python processes all the statements inside the try block and if it encounters a ValueError, then the control is immediately passed to the except block, skipping any remaining statements in the try block. After the exception is handled by the except block, control is passed to the next statement following the try-except block.

**Handling multiple exceptions**

A try-except block can have multiple except clauses to handle multiple exceptions. If an exception occurs then the except block is checked in top to bottom manner to find the matching exception. If there is no matching exception in the except clause, then the program crashes.

**Program 2: Multiple except statements**

```
try:
   a=int(input('Enter the dividend:'))
   b=int(input('Enter the divisor:'))
   c=a/b
   print("Result is" , c )
except ValueError:
   print("Please enter numbers")
except ZeroDivisionError:
   print('The divisor cannot be zero')
```

*o/p1:*
*Enter the dividend:abc*
*Please enter numbers*

*o/p2:*
*Enter the dividend:25*
*Enter the divisor:0*
*The divisor cannot be zero*

**Multiple exceptions in except clause**

An except clause can handle multiple exceptions by passing a tuple of exceptions in the except header line.

**Program 3: Handling multiple exceptions with a single except**

```
try:
    a=int(input('Enter the dividend:'))
    b=int(input('Enter the divisor:'))
    c=a/b
    print("Result is" , c )
except (ValueError, ZeroDivisionError):
    print("Please enter correct values for dividend and divisor")
```

*o/p1:*
*Enter the dividend:25*
*Enter the divisor:0*
*Please enter correct values for dividend and divisor*

**Handling all exceptions**

An except clause written without any exceptions in the header line will catch all the exceptions in the corresponding try block. Using this method has the advantage that all exceptions can be handled, but pinpointing the exact cause of the exception may be difficult in such a scenario.

**Program 4: Handling all exceptions**

```
try:
    a=int(input('Enter the dividend:'))
    b=int(input('Enter the divisor:'))
    c=a/b
    print("Result is" , c )
except :
    print("Please correct your mistakes")
```

*o/p1:*
*Enter the dividend:abc*
*Please correct your mistakes*

**else clause in try-except statement**

The else block is used to execute statements that should be run if there are no errors in the program/block of code enclosed within the try block.

**Program 5: else block**

```
try:
   age = int(input("Please enter your age: "))
except ValueError:
   print("Hey, that wasn't a number!")
else:
   print("I see that you are",age, "years old")
```

*o/p1:*
*Please enter your age: 25*
***I see that you are 25 years old***

*o/p2:*
*Please enter your age: ab*
*Hey, that wasn't a number!*

**finally clause in the try-except statement**

The finally block of code is executed regardless of whether an error is encountered or not within the statements enclosed in the try block.

**Program 6: finally block**

```
try:
   age = int(input("Please enter your age: "))
except ValueError:
   print("Hey, that wasn't a number!")
else:
   print("I see that you are",age, "years old")
finally:
   print('Thank you for using our software')
```

*o/p1:*
*Please enter your age: 25*
*I see that you are 25 years old*
***Thank you for using our software***

*o/p2:*
*Please enter your age: ab*
*Hey, that wasn't a number!*
***Thank you for using our software***

**raise statement**

The programmer can generate an exception programmatically by using the raise statement.

The syntax of the raise statement is :

*raise exception-name[ (optional-argument) ]*

**Program 7: raise statement**

```
age=int(input('Enter your age:'))
if age<18:
   raise ValueError('Age should be greater than 18')
print('Good Bye')
```

*o/p1:*
*Enter your age:21*
*Good Bye*

*o/p2:*

```
 File "E:/ raising errors.py", line 3, in <module>
   raise ValueError('Age should be greater than 18')
```

***ValueError: Age should be greater than 18***

**Handling exception generated by the raise statement**

The raise statement is typically used to flag an error to one level above the current scope of program execution, to tell that the program has encountered an invalid condition or value and that the program cannot move further until the error is corrected. A try except block typically encloses a raise statement which then handles the exception generated by the raise statement.

**Program 8: Handling exception generated by raise statement**

```
try:
   age = int(input("Please enter your age: "))
   if age< 18:
      raise ValueError
except:
   print("Please check the inputs and try again")
print('Good Bye')
```

*o/p1:*
*Please enter your age: 21*
*Good Bye*

*o/p2:*
*Please enter your age: 15*
***Please check the inputs and try again***
*Good Bye*

## 6. FILE HANDLING

**Why are Files needed?**

A microprocessor/CPU needs **program** to give instructions on the work that needs to be performed. Additionally it needs **input data** to work on and usually it computes and gives some **output data**. In short it needs *program, input data and output data* in order to perform the desired task.

But the CPU only works with programs and data from the RAM which is volatile. So all the programs as well as the input/output data must be stored on secondary storage devices in the form of files. These files are loaded into the primary memory/RAM in order to be run on the microprocessor. And in order to give a logical structure to the file management, a folder or directory structure is maintained by the Operating System.

**Data Files** are used by the program/application to either accept input or to store the output or sometimes both.

**Types of Files**

All files are stored as sequences of 0s and 1s in a computer. But depending on the way the bits are stored there are two broad categories of files -

1. **Text files**
   In Text files all the data are stored as characters (usually ASCII characters) and all the characters usually have the same byte size. For example if the following content/data is to be stored in a text file -
   7552 230.25 ABC

   Then it will be stored as-

| Byte1 | Byte2 | Byte3 | Byte4 | Byte5 | Byte6 | Byte7 | Byte8 | Byte9 | Byte10 | Byte11 | Byte12 | Byte13 | Byte14 | Byte15 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 7 | 5 | 5 | 2 | | 2 | 3 | 0 | . | 2 | 5 | | A | B | C |

   where each of the boxes will be 1 byte in length and stores the ASCII code for each of the characters shown. In the above example the content/data occupies **15 bytes.**

   **Advantages of Text files**

   - Simplicity
   - Text files can be read easily as all the characters are identifiable and usually have the same length (for ASCII text files 1 byte)
   - Even if there is an error of a few bits in data, the rest of the data can be safely recovered

   **Disadvantages of Text files**

   - It occupies more storage than is strictly necessary
   - Anybody can read the data. It does not offer any scope for adding security features
   - If data is to be transferred as text files then care must be taken to see that there is some separator between the different data, especially between two numeric data
   - When numeric data is to be read from a text file, then a translation step to read the number in (ascii) text format to binary format which is understood by a computer/program must be done. This step can reduce the speed/performance of the application.

**2. Binary Files**

Any file that is not a text file is a binary file. Usually in binary files the data is stored in the same format in which the data is stored in the computer's memory/RAM.

For example, consider that the three data -
7552 230.25 ABC
i.e. the int value 7552, the float value 230.25, the string 'ABC', are to be stored in binary format files. Then, the binary representation of the numbers and strings are stored as -

| Byte1 | Byte2 | Byte3 | Byte4 | Byte5 | Byte6 | Byte7 | Byte8 | Byte9 |
|---|---|---|---|---|---|---|---|---|
| 7552 | | 230.25 | | | | A | B | C |
| 000011101 | 10000000 | 01000011 | 01100110 | 01000000 | 00000000 | 01000001 | 01000010 | 01000011 |

The above data occupies 9 bytes. So the binary data is more space efficient compared to text data, especially when numbers are involved.

**Advantages of Binary Files**

- They occupy less space compared to text files
- Processing of binary files is faster as minimum processing needs to be done to load it into memory
- Security measures can be applied to binary data

**Disadvantages of Binary Files**

- The program/programmer creating the binary data as well as the program/programmer reading the binary data must be very careful with regards to the order of the data as well as datatype of the data as there is no scope of missing even a single bit.
- For reading the binary data we must know the exact order and the datatype of the data to read it correctly.
- Even a few bits of corruption of the data may render the binary data file useless.

**Difference between Text and Binary File**

| **Text File** | **Binary File** |
|---|---|
| Its Bits represent character. | Its Bits represent a custom data. |
| Less prone to get corrupt as change reflects as soon as made and can be undone. | Can easily get corrupted, corrupt on even a single bit change |
| Store only plain text in a file. | Can store different types of data (audio, text , image) in a single file. |
| Widely used file format and can be opened in any text editor. | Developed for an application and can be opened in that application only. |
| Mostly .txt and .rtf are used as extensions to text files. | Can have any application defined extension. |

**Basic steps in working with Files**

1. Open a File
2. Read the file
3. Process the file
4. Close the file

**Opening a File**
The open() method is used to open a file.
**Syntax:**
*<file object> = open( filepath, mode='r', encoding=None)*
where
*file object* - It is returned by the open method, which is used to work with the file on the external storage media.
*filepath* - It is a string that can represent either the absolute path or the relative path.
*mode* - represents the mode in which the file is being opened.
*encoding*- This parameter is used only with text files and it shows the encoding/decoding scheme that is to be used with text files. If it is not passed for text files then a default value of *None* is used and the system defined default encoding scheme is used for text files. Some of the encoding schemes that can be used are 'ascii', 'utf8', 'utf16' etc. The default encoding scheme for text files in windows is **'ascii'.**

**Absolute path-** The absolute path starts with the root directory (drive letter C: , D: etc. in windows) and moves up to the directory and subdirectory till it reaches the subdirectory in which the file is actually situated. Eg. 'C:\directory1\subdirectory1\subdirectory2\filename.txt'

**Relative path-** The relative path starts with the **current working directory (.)** or the **parent directory(..)** relative to the current working directory and then moves on to the directory and the subdirectory till the filename is reached.
Eg. '.\directory1\filename1.txt' , '..\directory1\subdirectory1\subdirectory2\filename1.txt'

**Paths and the escape character backslash **
While writing paths in windows the problem comes that the backslash, \ represents the escape character and if we need to show it inside the string then we need to type two back slashes \\.
Eg. 'C:\\dir1\\subdir1\\file1.txt' , '.\\dir1\\subdir1\\file1.txt'

**Raw strings**
If we want that the backslash appearing inside the strings should not be pre-processed by python then we can use raw strings. Raw strings have the letter r or R, in front of the string literal.
Eg. R'c:\dir1\subdir1\file1.txt', r'.\dir1\subdir1\file1.txt'

**Examples of valid paths in the open method**
```
f1=open('c:\\dir1\\file1.txt')
f1=open(R'c:\dir1\file1.txt')
f1=open(r'.\dir1\file1.txt')
```

**Examples of invalid paths in the open method**
```
f1=open("c:\dir1\file1.txt")
f1=open("c:\\\dir1\file1.txt")
f1=open(r'c:\\dir1\\file1.txt")
```

**File opening modes-**

| S. No. | Mode | Description |
|---|---|---|
| **1** | **r / rt** | Reading text files only. Sets file pointer at beginning of the file. This is the default mode. Gives error if file does not exist. |
| **2** | **rb** | Reading with binary file. Sets file pointer at beginning of the file. Gives error if file does not exist. |
| **3** | **r+ / rt+** | Both reading and writing text file. The file pointer placed at the beginning of the file. Gives error if file does not exist. |
| **4** | **rb+** | Both reading and writing with binary file. The file pointer placed at the beginning of the file. Gives error if file does not exist. |
| **5** | **w / wt** | Writing only text file. Truncates file, if file exists. If not, creates a new file for writing. |
| **6** | **wb** | Writing with binary file. Truncates file, if file exists. If not, creates a new file for writing. |
| **7** | **w+ / wt+** | Both writing and reading text file. Truncates, if file exists, creates a new file if file does not exist. |
| **8** | **wb+** | Writing and reading with binary file. Truncates, if file exists, otherwise creates a new file. |
| **9** | **a / at** | For appending text file. Move file pointer at end of the file. Creates new file for writing, if it does not exist. |
| **10** | **ab** | Appending with binary file. Move file pointer at end of the file. Creates new file for writing, if it does not exist. |
| **11** | **a+ / at+** | For both appending and reading text file. If the file does not exist, it creates a new file for reading and writing. For read operations file pointer is at position 0 but as soon as the first write occurs the file pointer is changed to EOF. |
| **12** | **ab+** | Appending with binary file. If the file does not exist, it creates a new file for reading and writing. For read operations file pointer is at position 0 but as soon as the first write occurs the file pointer is changed to EOF. |
| **13** | **x / xt** | Exclusive text files creation for writing only. If file already exists then it gives error. |
| **14** | **xb** | Exclusive binary file creation for writing only. If file already exists then it gives error. |

**Examples of opening files-**

```
f1=open("c:\\dir1\\subdir1\\file1.txt", mode="r+")
f1=open(r'.\dir1\file1.txt', mode='w+')
f1=open(R'..\dir2\file2.txt', mode='wb+')
```

**Closing a file**

The command *<file object>.close()* is used to close the file associated with the file object. If any open file is not closed by the programmer explicitly then after the program run is over, then the python environment automatically closes any open files.

**Basic commands for reading a File**

1. read()
2. readline()
3. readlines()

**1. <file object>.read( [*size*] )**

This command reads *size* characters in text mode or *size* bytes in binary mode from the file associated with the <file object>. If the size parameter is omitted or is negative, then it reads all the bytes, till the end of file (EOF) is reached. The read() method returns an empty string '' when the end of file has been reached.

*[NOTE:*

1. *While working with text file be aware of the encoding scheme/language used in the text file, the number of bytes used per character and the particular method you are using for reading/writing/displaying a character.*
2. *Assume that for the discussion here onwards we are working with ASCII encoded text files only in which each character is stored in one byte only.*

*]*

**File Pointer**

Whenever a file is opened, an internal integer python variable, the file pointer, is associated with each file object. The file pointer defines the byte location within the file, where the next read or write will take place.

When a file is opened for reading, the file pointer is at byte position 0 (in python programming numbering usually starts from 0)

For example consider the ASCII text file, 'sample.txt', containing the text -

```
hello world
good day
```

**Program 1: Reading a text file using read command**

```
#1 Reading a text file using read command
myfile=open('sample.txt')
s1=myfile.read(8)
print(s1)
s2=myfile.read(5)    #the current read starts reading from where the previous read stopped
print(s2)

s3=myfile.read()    #if no parameter specified it reads from current position till end of file
print(s3)
myfile.close()

o/p:
hello wo
rld
g
ood day
```

1. When the file is opened by the command - *myfile=open('sample.txt')* - the file pointer is at byte 0.

| Byte 0 | B-1 | B-2 | B-3 | B-4 | B-5 | B-6 | B-7 | B-8 | B-9 | B-10 | B-11 | B-12 | B-13 | B-14 | B-15 | B-16 | B-17 | B-18 | B-19 | B-20 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| h | e | l | l | o | | w | o | r | l | d | \n | g | o | o | d | | d | a | y | EOF |



File pointer

2. When the command - s1=myfile.read(8) - is executed, python reads 8 bytes from the current file pointer position and ***after*** the read command the file pointer is at Byte 8.

| Byte 0 | B-1 | B-2 | B-3 | B-4 | B-5 | B-6 | B-7 | B-8 | B-9 | B-10 | B-11 | B-12 | B-13 | B-14 | B-15 | B-16 | B-17 | B-18 | B-19 | B-20 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| h | e | l | l | o | | w | o | r | l | d | \n | g | o | o | d | | d | a | y | EOF |



File pointer

3. When the command - s2=myfile.read(5) - is executed, python reads 5 bytes from the current file pointer position and ***after*** the read command the file pointer is at Byte 13. Note that the newline/enter key is considered as one character.

| Byte 0 | B-1 | B-2 | B-3 | B-4 | B-5 | B-6 | B-7 | B-8 | B-9 | B-10 | B-11 | B-12 | B-13 | B-14 | B-15 | B-16 | B-17 | B-18 | B-19 | B-20 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| h | e | l | l | o | | w | o | r | l | d | \n | g | o | o | d | | d | a | y | EOF |

File pointer

4. When the command - s3=myfile.read() - is executed, python reads all the bytes from the current file pointer position till the end of File (EOF) character.

| Byte 0 | B-1 | B-2 | B-3 | B-4 | B-5 | B-6 | B-7 | B-8 | B-9 | B-10 | B-11 | B-12 | B-13 | B-14 | B-15 | B-16 | B-17 | B-18 | B-19 | B-20 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| h | e | l | l | o | | w | o | r | l | d | \n | g | o | o | d | | d | a | y | EOF |

File Pointer

2. **<file object>.readline ( [*size*] )**

The readline() method reads a single line and returns it as a string, from the file associated with the <file object>. The newline character '\n' is left at the end of the string and is only omitted if the last line in the text file does not end in a newline.

A blank line is represented by '\n'. When the end of file is reached, the readline() method returns an empty string ''.

If the *size* is specified, then readline method reads at most *size* bytes (in which case the \n is not inserted at the end of the string) or till the '\n' character is reached whichever comes earlier.

Consider the following text file, 'test.txt'-

```
hello
good morning
Do have a nice day
Good Bye!!!
```

**Program 2: Reading lines from a text file using readline command**

```
myfile=open('test.txt')
s1=myfile.readline()  # \n from input is left in string s1
# s1=s1.strip()              # use this to remove the \n at the end
print(s1,len(s1))          #print statement adds its own newline at the end
s2=myfile.readline()
print(s2)
s3=myfile.readline(5)  #reads only 5 bytes \n not inserted at end
print(s3)
s4=myfile.readline(50)  #from the current position till end of line
print(s4)
s5=myfile.readline()
print(s5)
myfile.close()

o/p:
hello
 6
```

```
good morning

Do ha
ve a nice day

Good Bye!!!
```

1. The statement - *s1=myfile.readline()* - reads into the variable s1, the string

| | | | | | |
|---|---|---|---|---|---|
| h | e | l | l | o | \n |

including the newline character '\n' representing the enter key. This string is of length 6. When we say - *print(s1, len(s1))* - then apart from the \n at the end of the string s1, the print() method by default adds its own end character \n once again. So while using the output of the readline() statement if the additional blank line is not needed then either

1. the newline character can be removed from the string s1 as -
   s1=s1.strip() OR
2. while using the print statement the end character can be made an empty string as -
   print(s1, end='')

The first method is more useful if further processing of string s1 needs to be done.

2. The statement - s3=myfile.readline(5) - reads only 5 characters 'Do ha' and does not add the \n character at the end.

3. The statement - s4=myfile.readline(50) - will read from the current position of the file pointer till either the 50 character limit is reached or the \n is encountered in the file. Since the \n is encountered first, it reads the string 've a nice day\n' , which also includes the \n character.

3. **<file object>.readlines ( ) OR list(<file object>)**

The readlines() method can read an entire text file and split it up into different lines. The lines of the text file including the '\n' character is returned back as a list of strings. The same effect can be observed by using the list() method directly on the <file object> also.

**Program 3 : Reading all lines of text file together**

```
myfile=open('sample.txt')
line1=myfile.readlines()
print(line1)
myfile.close()

f=open('test.txt')
L=list(f)
f.close()
print(L)

o/p:
['hello world\n', 'good day']
['hello\n', 'good morning\n', 'Do have a nice day\n', 'Good Bye!!!']
```

Since the last line of the text file did not have the Enter key pressed, the last element in both the lists also do not have the \n character at the end.

The list() method also works on the <file object> f, as the **file object is an iterable**. If we use the for loop iterator on a valid file object then it gives one line at a time. The returned line will have \n as the last character.

**Program 4: Iterating over a file object**

```
myfile=open('test.txt')
for ln in myfile:
   print(ln,end='')

o/p:
hello
good morning
Do have a nice day
Good Bye!!!
```

Note the use of the end='' parameter to prevent an additional blank line between two lines of text.

The while loop can also be used in combination with the readline() to show the same behaviour.

**Program 5: Using while loop to read one line at a time from a text file**

```
myfile=open('sample.txt','r')
s1 = 'abc'                  # initialize s1 to some non-empty value
while s1:
   s1=myfile.readline()
   print(s1.strip())

o/p:
hello world
good day
```

The s1 string returns an empty string '', when it encounters the end of file character. This property is used in the condition of the while loop to terminate the loop when the file is read completely. Also note the use of the s1.strip() statement to remove the \n character from the end of the string s1.

**Operations on a Text File**

File operations are the most intensive and time consuming operations in any application. For performing an operation on a Text file, there can be different ways. Some ways may be more efficient in some situations.

Usually if we are performing only a read operation on a small or medium sized text file, then the read() method without any parameters can read the entire text file in memory and the file can be closed immediately after executing the read() method, leaving the file free for use by other applications. But if the file is large size and cannot be accommodated in memory then it is preferable to use the read(x) or the readline() method so that only a small portion of the file is in memory at any one point of time.

Also while doing any file operations, care must be taken so that there is less number of operations in between the file open and the file close statements. So that the file is used by our program or application for the least amount of time and make our applications efficient.

**Basic operations on a Text file**

The basic operations on a text file can be categorized into-

1. Character-by-Character processing
2. Word-by-Word processing
3. Line-by-Line processing.

**1. Character by Character processing**

**Program 6: Count how many a's are present in a file**

| **#Method 1** | **#Method 2** |
|---|---|
| f=open('test.txt')<br>s=f.read()<br>f.close()<br>count=0<br>for ch in s:<br>  if ch .lower()=='a':<br>  count=count+1<br>print('a occurs',count,'times')<br><br>*o/p:*<br>*a occurs 3 times* | f=open('test.txt')<br>count,s1 = 0, 'abc'<br>while s1:<br>  s1=f.read(1)<br>  if s1 .lower()=='a':<br>  count=count+1<br>print('a occurs',count,'times')<br>f.close()<br><br>*o/p:*<br>*a occurs 3 times* |

In **Method 1** we have read the entire text file into a single string s and then processed that string using the iterator to count how many a's are there in the string *[We could also have omitted the loop and used count=s.count('a') ]*. Here the file is in use/locked by our program for only a short duration. But the disadvantage is that if the 'test.txt' file is large, then it will not be accommodated in the computer's memory.

In **Method 2** we have used the command - s1=f.read(1) - to read only one character at a time from the file object and till the time our processing of file is over, the file is locked for use by our program. But the advantage is that this program can work with any large file size.

**2. Word-By-Word processing**

**Program 7: Count how many times the word 'me' appears in a text file**

**#Method 1**

```
wc=0
f=open("story.txt", 'rt')
s=f.read()
f.close()

L=s.split()
x=len(L)
for i in range(x):
   L[i]=L[i].lower().strip(' .,\'\"?!')
wc=L.count('me')
print('me occurs',wc,'times')
```

*o/p:*
*me occurs 3 times*

**#Method 2**

```
wc=0
f=open("story.txt", 'rt')
L=f.readlines()
f.close()

for i in L:
   L1=i.split()
   for j in L1:
      j=j.strip(' .,\'\"?!').lower()
      if j=='me':
         wc=wc+1
print('me occurs',wc,'times')
```

*o/p:*
*me occurs 3 times*

**#Method 3**

```
wc=0
f=open("story.txt", 'rt')
wc = 0

for ln in f:
   L=ln.split()
   for i in L:
      i=i.lower().strip(' .,\'\"?!')
      if i=='me':
         wc=wc+1
f.close()
print('me occurs',wc,'times')
```

*o/p:*
*me occurs 3 times*

In **Method 1**, we read the entire text file into the string s. After that using - *L=s.split()* - the string s is split into list of words L. Then using the for iterator we go through all the words in list L and using the strip() function, we strip off all the punctuation symbols around that word. Using the count() method on the list L, gives us the desired answer.

In **Method 2**, all the lines of the text file are read into the list L using - L=f.readlines(). Then we iterate over all the lines and split them into word. Then using the inner nested for loop we go through the list of words and strip off the punctuation symbols and match it with the string 'me' to increment our counter wc.

In **Method 3**, we use the iterator over the file object f. The counter variable ln is a line in the text file. Inside the iterator, we split the line into words. The words are stripped off the punctuation marks and then checked if it matches the string 'me'. If yes then the counter variable, wc is incremented by 1.

**3. Line-By-Line processing**

**Program 8: Count the lines starting with 'g'**

| **#Method 1** | **#Method 2** | **#Method 3** |
|---|---|---|
| f=open('test.txt')<br>lc=0<br>L=f.readlines()<br>f.close()<br>for i in L:<br>  if i[0].lower() == 'g':<br>  lc = lc+1<br>print('line starts with g:', lc)<br><br>*o/p:*<br>*line starts with g: 2* | f=open('test.txt')<br>lc=0<br>for i in f:<br>  if i[0].lower() == 'g':<br>  lc = lc+1<br>print('line starts with g:', lc)<br>f.close()<br><br>*o/p:*<br>*line starts with g: 2* | f=open('test.txt')<br>lc=0<br>s=f.read()<br>f.close()<br>L=s.split('\n')<br>for i in L:<br>  if i[0].lower() == 'g':<br>  lc = lc+1<br>print('line starts with g:', lc)<br><br>*o/p:*<br>*line starts with g: 2* |

In **Method 1**, we use the readlines() to read all the lines into a List L. Then using the for iterator we go through all the lines and check if the starting letter is 'g'. If yes then a counter variable, lc is incremented.

In **Method 2,** we iterate directly over the file object f to get one line at a time in each iteration. Then inside the loop we check if the starting letter is 'g'. If yes then the counter variable is incremented.

In **Method 3**, we use the read() method to get the entire contents of the text file into a single string, s. Then we use the statement - L=s.split('\n') - to split the string s into a list of lines. The advantage is that each of the lines does not have the '\n' character at the end of the line. Then we go through the list using the for iterator and check if the starting character is 'g'. If yes then the counter variable is incremented.

**Writing Files**

For writing a file, the file must be opened in one of the modes that support writing to files.

(a) When opening the file in the **read and write mode** (r+, rb+), the file must exist prior to the open statement and the file pointer is placed at Byte 0. If you directly perform a write, any pre-existing e

(b) When opening the file in any combination involving write 'w' mode**(w,w+,wb,wb+),** if the file already exists, then it will be truncated and any data written will be written to a new blank file each time you run the program.

(c) When the file is opened in **'a','ab'** mode, if the file already exists, then the file pointer is placed at EOF position, and any new data is appended at the end.

(d) When the file is opened in **'a+', 'ab+'** mode, the file pointer is placed initially at EOF.

(e) When the file is opened in **'x', 'xb'** mode, then if the file already exists then an error is given. If the file does not exist then a new file is created and file pointer is at byte 0 position and only write operations are allowed in 'x' mode.

**Commands used for writing**

1. <file object>.write(stringobject)
2. <file object>.writelines(L)

**<file object>.write(stringobject)**
The write() method takes a *stringobject* and writes it to the file that is associated with the <file object>

**Program 9 : Using the write method**

```
# 8 Writing a text file
# 'w' mode
a,b,c='10','20','30'
f1=open('sample1.txt', 'w')
f1.write(a)    #parameters to write must always be strings
f1.write(b)
f1.write(c)
f1.close()

# 'a' mode
f2=open('sample2.txt', 'a')    #file opened in append mode
f2.write(a)
f2.write(b)
f2.write(c)
f2.close()
```

In the above program we have taken the variables a,b,c with the string values '10', '20' and '30'. Even if our data is in other data types, it should be converted to a string in order for it to be passed to the write() method.

The file objects f1 and f2 are similar only difference being the file opening modes. Since f1 is opened in 'w' mode, even if we run the program multiple times, each time the file *'sample1.txt'* is first truncated and then the data '102030' is written into it.

But since f2 is opened in append 'a' mode, if we run the program multiple times, the file *'sample2.txt'* is not truncated each time. Instead the data '102030' is appended to the end of the file each time and if the program is run 2 times then the file will contain the output 2 times, if the program is run 3 times then the file will contain the output 3 times and so on.

**<file object>.writelines(L)**

The writelines() method accepts a list of strings L as a parameter and writes it to the file associated with the <file object>.

**Program 10 : Using the writelines method**

```
L1 = ['hello world',
     'good morning',
     'have a nice day']
f1=open('sample1.txt', 'w')
f1.writelines(L1)
f1.close()

L2 = ['hello world\n',
     'good morning\n',
     'have a nice day\n']
f2=open('sample2.txt', 'w')
f2.writelines(L2)
f2.close()

print('file written...')
```

In the above program we have two list of strings L1 and L2. In L1, all the list elements do not have \n character at their ends, whereas in L2, all the list elements have the \n character at their ends. Since the writelines() method writes the lines as is one by one, the first file *'sample1.txt'* will have all the sentences following one after the other.

*sample1.txt*

```
hello worldgood morninghave a nice day
```

Whereas in the second file *'sample2.txt'* the lines will be displayed one on each line.

*sample2.txt*

```
hello world
good morning
have a nice day
```

**CSV Files**

The CSV (Comma Separated Values) is a text file format in which rows of data are present and the individual data elements are separated by commas. The CSV format is the most common import and export format for spreadsheets and databases. There is a lack of a well-defined standard for the CSV file developed in different applications/platforms. The subtle differences among CSV formats make it annoying to process CSV files from multiple sources.

Examples of some valid formats of CSV files are:

| |
|---|
| 1. Each record is located on a separate line, delimited by a line break (CRLF). For example:<br><br>*aaa,bbb,ccc CRLF*<br>*zzz,yyy,xxx CRLF* |
| 2. The last record in the file may or may not have an ending line break. For example:<br><br>*aaa,bbb,ccc CRLF*<br>*zzz,yyy,xxx* |
| 3. There maybe an optional header line appearing as the first line of the file with the same format as normal record lines. For example:<br><br>*field_name,field_name,field_name CRLF*<br>*aaa,bbb,ccc CRLF*<br>*zzz,yyy,xxx CRLF* |
| 5. Each field may or may not be enclosed in double quotes. If fields are not enclosed with double quotes, then double quotes may not appear inside the fields. For example:<br><br>*"aaa","bbb","ccc" CRLF*<br>*zzz,yyy,xxx* |
| 6. Fields containing line breaks (CRLF), double quotes, and commas should be enclosed in double-quotes. For example:<br><br>*"aaa","b CRLF*<br>*bb","ccc" CRLF*<br>*zzz,yyy,xxx* |
| 7. If double-quotes are used to enclose fields, then a double-quote appearing inside a field must be escaped by preceding it with another double quote. For example:<br><br>*"aaa","b""bb","ccc"* |

*[Note:*

1. *CR stands for the character 'Carriage Return' with Integer ASCII code - 13 and C++/Python notation \r or '\r'.*
2. *LF stands for the character 'Line Feed' with Integer ASCII code - 10 and C++/Python notation \n or '\n'.*
3. *In Windows platform the Enter Key / End of Line(EOL) / newline character is represented by the character combination CRLF i.e. '\r\n' in python.*
4. *In Unix/Linux platforms the Enter Key /End of Line (EOL) / newline character is represented by the character LF only i.e. '\n' in python.*
5. *In Macintosh platforms the Enter Key / End of Line (EOL) / newline character is represented by the character CR only i.e. '\r' in python.*
6. *When opening a file for reading CSV file add the parameter, newline='EOL_character' to process the End of Line correctly.*
7. *While opening a CSV file for reading on Windows platform, add the parameter, newline='\r\n' in the open() method to avoid adding an extra blank line while processing/displaying output on the screen ]*

**csv module**

The csv module implements classes to read and write tabular data in CSV format. The csv module handles the different types of delimiters and quoting characters in CSV files and efficiently manipulate such data, hiding the details of reading and writing the data from the programmer.

**Reading/writing csv files**

The csv file must first be opened in the appropriate text file opening mode for reading or writing. When opening the file for reading/writing on windows platform add the parameter *newline='\r\n' or newline = ''* to the open method in order to interpret the newline character correctly. Then get a reader/writer object by calling the csv module's reader() and writer() method. The reader and the writer object encapsulate the task of reading / writing from/to a csv file.

**1. csv.reader( <file object> )**

The reader() method returns back a reader object that is associated with the <file object>. The returned reader object is an iterable and iterating over it gives a list of strings representing each row of the csv file.

**2. csv.writer( <file object> )**

The writer() method returns back a writer object that is associated with the <file object>. The writer object can be used to write rows to the csv file using the writerow() or writerows() method.

- The writerow() method accepts a list of strings as a parameter representing the row that needs to be written to the csv file.
- The writerows() method accepts a nested list in which the inner list is a list of strings. This method can be used to write multiple rows of data to the csv file.

Consider the csv file 'employee.csv' with the following data-

```
empno,empname,dept
101,abc,planning
102,def,marketing
103,ghi,sales
```

**Program 24 : Using csv module**

```
# 24 Using csv module
import csv

#reading a csv file
f=open('employee.csv', newline='\r\n')
cr=csv.reader(f)
for r in cr:
   print(r)
f.close()

#writing to a csv file
f=open('employee.csv', 'a', newline='')
cw=csv.writer(f)
cw.writerow(['104', 'jkl', 'production'])

L=[ ['105','nnn','sales'],
    ['106','ppp','marketing'],
    ['107','qqq','sales']]
cw.writerows(L)
f.close()
print('file written..')
```

After executing the above program the 'employee.csv' file will contain additional rows added by the writerow() and the writerows() functions :

```
empno,empname,dept
101,abc,planning
102,def,marketing
```

103,ghi,sales
*104,jkl,production*
***105,nnn,sales***
***106,ppp,marketing***
***107,qqq,sales***

**Working with Binary Files**

Python has the built-in data types - *bytes* and *bytearray* for working with binary data. When a binary file is opened for reading/writing, the same commands - read*()* and *write()* are used. . The only difference is that for-

- *<bytes object> = <file object>.read( [x] )*
  Here x, is the number of bytes that is to be read and it returns back a *bytes* object
- <file object>.write( x )
  Here x, is either a *bytes / bytearray* object

Whenever a basic data type such as – *int/float/str* has to be written to a binary file, then it must first be converted/encoded into a *bytes* object. Also whenever a binary file is read then the returned object also is a *bytes* object which must be decoded back to *int/float/str.*

**Converting objects to binary form**

If we are having simple data types such as str, int or float then the process of converting it to *bytes* form is simple. But if we are having complex object such as - lists, tuples, dictionary, nested lists, dictionary containing lists etc. then the programmer would manually have to go through each of the element find its datatype and then use the correct form of encoding it. This becomes very cumbersome. The solution to this is Serialization.

**Serialization/Deserialization**

In Serialization, we use a common standard or a protocol, in which python object can be given as input and a standard representation of a sequence of bytes is obtained. This sequence of bytes can be written on to a binary file or sent over a network.

While reading the file, the reverse process using the same standard/protocol allows us to copy the binary data directly to objects in python. This process is called deserialization.

**Packages used for Serialization**

**pickle module**

The pickle module implements protocols for serialization and de-serialization of python objects. A python object can be converted to a bytes stream object and the process is called ***pickling***. The reverse process in which a bytes stream is converted back to a python object is called ***unpickling***.

**Limitations**

- The pickle module uses a compact binary representation and the bytes stream object i.e. the pickled data file that is generated is not human-readable.
- Different versions of python use different protocols for pickling/unpickling. So generating a pickled data in one version of python may not be readable in another version of python.
- The data formats used by pickle are python-specific. So the data files generated in a python application can only be read by another application developed in that version of python. Software developed in other languages will not be able to read the pickled data files generated by our python application.
- It is possible to construct malicious pickle data which will **execute arbitrary code during unpickling**. Never unpickle data that could have come from an untrusted source, or that could have been tampered with.

**Serializing/Deserializing an object using pickle**

The dump() method of the pickle module is used for serialization and the load() method is used for deserialization of a python object.

**Program 16 : Pickle an object**

```
# 16 Pickle an object
import pickle
L = [101, 'abc', 'mumbai']
f = open('employee.dat','wb')
pickle.dump(L,f)

d={444:'mmm',555:'ppp'}
pickle.dump(d,f)
f.close()
print('File written...')
```

In order to use the pickle library, first we need to use the statement - *import pickle*. Then we create any pickable object (here the list L). After that we open the file 'employee.dat' in binary write mode (wb). If we need the existing data to be preserved then open the file in 'ab' mode.

The statement - *pickle.dump(L,f)* - will serialize the list L, and write the bytes into the file f. Any kind of python object tuple, dictionary etc can be pickled and stored in a binary file.

**Program 17 : Unpickle an object**

```
# 17 Unpickle an object
import pickle
f = open('employee.dat','rb')
L = pickle.load(f)
d=pickle.load(f)

f.close()
print(L)
print(d)

o/p:
[101, 'abc', 'mumbai']
{444: 'mmm', 555: 'ppp'}
```

To unpickle an object stored in a binary file, first open the file in read binary mode 'rb'. Then using the statement - L = pickle.load(f) - the entire data is loaded correctly into the variable L. The load() method correctly recognises *one python object at a time* from the binary file associated with f and returns back a python object ( in this case a list object is returned back to variable L).

If you execute the pickle.load(f) object the next time, then the next dictionary object that was pickled when the binary file was created is read correctly and returned back to the variable, d.

**To pickle a set of records**

In order to pickle a set of similar records such as employee details - (employee number, employee name, city), we can

(a) create a list in memory and then pickle the list repeatedly and append it to the same binary file.
(b) Create a dictionary in memory and pickle the dictionary repeatedly and append it to the binary file.

**Program 18a: pickle a list of records**

```
#18 pickle a list of records
import pickle
f = open("employee.dat","ab")
while True:
    eno = int(input("Enter Employee number:"))
    ename  = input("Enter Employee Name :")
    city=input('Enter city:')
    L = [eno,ename,city]
    pickle.dump(L,f)
    ch= input("Add more records(y/n) :")
    if(ch=='n'):
        break

f.flush()
f.close()

o/p:
Enter Employee number:101
Enter Employee Name :abc
Enter city:mumbai
Add more records(y/n) :y

Enter Employee number:102
Enter Employee Name :def
Enter city:delhi
Add more records(y/n) :y

Enter Employee number:103
Enter Employee Name :ghi
Enter city:mumbai
Add more records(y/n) :n

File written...
```

1. We open the binary file 'employee.dat' in append mode 'ab'.
2. The code highlighted in yellow is used to accept user input and create a list whose elements consist of the employee details - employee number(eno) and employee name(ename) and city.
3. The command - pickle.dump(L,f) - is used to populate/pickle the file with the data contained in list L. The newly added list object is appended to the existing file.

On using the dump() command, the python environment usually writes it to a buffer and later when the buffer becomes full then it performs the actual write to the file on the hard disk. The flush() command is used to immediately clear the pythons buffer and transfer all the write buffer records from pythons buffer to the OS buffer and it ***requests*** the OS to write the file. If you need to ***immediately*** write it to the hard disk, then the command - *os.fsync(f2.fileno())* from the ***os*** library- will need to be given after the f2.flush() statement, to force the OS to clear its buffer and write the file back to hard disk.

**Program 18b: pickle a dictionary of records**

```
#18b pickle a dictionary of records
import pickle
f = open("items.dat","wb")
while True:
   it_id  = input("Enter Item ID :")
   it_name  = input("Enter Item Name :")
   amt=float(input('Enter amount:'))
   d = {it_id:[it_name,amt]}
   pickle.dump(d,f)
   ch= input("Add more records(y/n) :")
   if(ch=='n'):
      break

f.flush()
f.close()
print(' File written...')
```

*o/p:*

```
Enter Item ID :101
Enter Item Name :nut
Enter amount:70
Add more records(y/n) :y

Enter Item ID :103
Enter Item Name :spanner
Enter amount:500
Add more records(y/n) :y

Enter Item ID :102
Enter Item Name :screw driver
Enter amount:175
Add more records(y/n) :n

File written...
```

1. The file f is used to open the file 'items.dat' in binary mode.
2. The same method as we have used to pickle a list of records is used when we have to pickle a dictionary of records to a binary file.
3. Inside a loop we accept the individual elements of the dictionary :
   (a) The key which is the item id is accepted from the user
   (b) The value which is a list containing the elements of item name and the amount is individually accepted from the user
4. After accepting the individual elements, we construct the dictionary d and then pickle that dictionary object onto the file f.

**Viewing/Processing list of records from a pickled binary file**

**(a) Viewing/processing a list of records stored in a binary file**

The binary file containing lists of records can be read in a loop by reading one list / record at a time using the command- L = pickle.load(f). When all the inputs from the file are over, an error (EOFError) is raised by the load() method, which is caught by the try-except block. The file is then closed in the *except* block.

**Program 19: unpickling/processing a list of records**

**Program to display details of employees from 'Mumbai' stored in binary file as list [eno,ename,city]**

```
# 19a unpickling a list of records
import pickle

f = open("employee.dat", "rb")
try:
   while True:
      L = pickle.load(f)
      if L[2] == 'mumbai':                #L is [eno, ename, city] , L[2] is checking the city name
         print(L)
except :    # EOFError catched
   f.close()
```

*o/p:*

*[101, 'abc', 'mumbai']*

*[103, 'ghi', 'mumbai']*

**(a) Viewing/processing a dictionary of records stored in a binary file**

When a dictionary of records is stored in a binary file, then the command pickle.load(f) will read one dictionary, d at a time. If we need to access / process either the key or the value of the dictionary, d then we have to iterate over the d.items() method. The d.items() returns back a tuple object on each iteration in which the first element is the key and the second element is the value of the dictionary.

**Program 19: unpickling/processing a dictionary of records**

**Program to display details of Items priced more than 100 stored in binary file as dictionary {item_id: [item_name,amount]}**

```
# 19a unpickling a dictionary of records
import pickle
f = open("items.dat","rb")
try:
   while True:
      d=pickle.load(f)       # dictionary object d returned on each iteration
      for k,v in d.items():   #d.items() returns back a tuple containing (key,value) at each iteration
         if v[1]>100:          #the value v, is itself a list [item_name,amount], v[1] is amount
            print(k,v)
except:
   f.close()


o/p:
103 ['spanner', 500.0]
102 ['screw driver', 175.0]
```

**Searching for a particular record from a binary file**

The same method used for reading multiple records from a binary file can be used to go through the binary file having multiple records of the same form.

We first ask the user for search criteria and then iterate over the individual records from the binary file. Each object in the binary file is a list and we check if the search criterion matches the particular element of the list. If yes then the record is displayed and the *found* check variable is set to 1.

If the record is not found then the *found* variable has value 0 and an appropriate message is displayed.

**Program 20 : Searching for a record in binary file**

```
# 20 Searching for a record in binary file
import pickle

num = int(input('Enter employee number to search :'))
found = 0

f = open("employee.dat", "rb")
try:
   while True:
      L = pickle.load(f)
      if L[0]==num:
         found=1
         print('Employee found. Details are:')
         print(L)
         break
except :
   f.close()

if found ==0:
   print('Employee not found')
```

*o/p:*

*Enter employee number to search :102*

*Employee found. Details are:*

*[102, 'def', 'delhi']*

**Update one record from a binary file**

To perform an update, we make use of a temporary binary file temp.dat. For all the non-matching records are copied from the original file 'employee.dat' to the 'temp.dat'.

For the matching record, the list object is updated in memory first and then this updated list object is copied/dumped into the 'temp.dat' file.

After processing of the file is over, we make use of the *remove()* and *rename()* methods from the **os** library to remove the original file 'employee.dat' and then rename our temporary file 'temp.dat' to the original file 'employee.dat'.

**Program 21 : update one record from a binary file**

```
# 21 update one record from a binary file
import pickle
import os

num = int(input('Enter employee number to update the record :'))

f = open("employee.dat", "rb")
f2 = open("temp.dat", "wb")
found=0

try:
   while True:
      L = pickle.load(f)
      if L[0]==num:
         found=1
         L[0]=int(input('Enter new employee number:'))
         L[1]=input('Enter new employee name:')
         L[2]=input('Enter new city:')
      pickle.dump(L,f2)
except:
   f.close()
   f2.flush()
   f2.close()

if found ==0:
   print('Employee not found')
   os.remove('temp.dat')
else:
   os.remove('employee.dat')
   os.rename('temp.dat', 'employee.dat')
print('File updated...')
```

*o/p:*

```
Enter employee number to update the record :102
Enter new employee number:999
Enter new employee name:zzz
Enter new city:chennai
File updated...
```

**Delete a record from a binary file**

Method similar to that used for performing an update to a binary file, can be used to perform a delete on a record from a binary file.

We use a temporary file 'temp.dat' and only the non-matching record are copied to the 'temp.dat' file and the record matching the delete criteria is not copied to the 'temp.dat' file.

**Program 22 : delete a record from a binary file**

```
#22 delete a record from a binary file
import pickle, os

num = int(input('Enter employee number to delete :'))

f = open("employee.dat", "rb")
f2 = open("temp.dat", "wb")

try:
   while True:
      L = pickle.load(f)
      if L[0]==num:
         found=1
      else:
         pickle.dump(L,f2)
except:
   f.close()
   f2.close()

if found ==0:
   print('Employee not found')
   os.remove('temp.dat')
else:
   os.remove('employee.dat')
   os.rename('temp.dat', 'employee.dat')
print('File updated...')

o/p:
Enter employee number to delete :999
File updated...
```

**Working with File Pointers**

We have the following methods for working with file pointers-

1. **<file object>.tell()**
   This method works well with **binary files** and returns an integer value denoting the byte position from the beginning of the file.

   For **text file** the tell() function returns an opaque number *(Note from python 3 documentation regarding **opaque number for text file**- this is not the same as byte position)* which can only be used in combination with the seek for a text file.

2. **<file object>.seek(offset, whence)**
   The offset is an integer value specifying the number of bytes from a particular position identified by the second parameter whence.

   The second parameter whence can have the possible value of either 0, 1 or 2. A *whence* value of 0 measures from the beginning of the file, 1 uses the current file position, and 2 uses the end of the file as the reference point. *whence* can be omitted and defaults to 0, using the beginning of the file as the reference point.
   For a **binary file** the seek() method works fine as stated above.

   But for a **text file,** the seek() can only be used with the whence value as 0, i.e. only offsets from the beginning of the file are valid. Also for a text file going to the end of the file i.e. f.seek(0,2) only is permitted and no other offsets can be used with whence=2 or whence=1 for text files.

   Example-
   Consider the following ASCII text file 'novel.txt' with the following data-

   ```
   ABCDEFGHIJK
   ```

   When the following program is executed

**Program 23 : File Pointers**

```
# 23 File pointers
f=open('novel.txt','r+')
f.write('XYZ')
f.seek(6)
f.write('PQR')
f.seek(0,2)
f.write('UVW')
f.close()
```

The contents of the file 'novel.txt' is changed to -

```
XYZDEFPQRJKUVW
```

**How it works?**

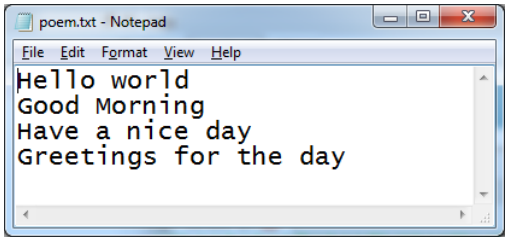
1. When the file is opened, the file pointer is at byte position 0. So when the 'XYZ' is written, it overwrites 'ABC'.
2. The command - f.seek(6) - works by placing the file pointer at byte number 6 from the start of the file. The starting number is 0, so byte offset 6 starts at character 'G' in the original file and when write 'PQR' is done, it overwrites 'GHI'.
3. The command - f.seek(0,2) - moves the file pointer to the end of the file character and when the characters 'UVW' are written, they appear at the end.

*[Note- In ASCII text files all characters are 1 byte width]*

**with statement**

The with statement can be used in a similar manner to the open() command to open a file and associate it with a file object. The file operations are coded within the *with* block. When control exits the *with* block all open files are closed automatically. In addition if any errors happen within the with block then all open files are closed automatically and only then the program terminates.

For example consider the file 'poem.txt' as shown below:




**Program 24 : with statement**

```
with open('poem.txt') as f:
   s1=f.readline()
   print(s1)
print(f.closed)

o/p-
Hello world

True
```

In the above program the file object f is associated with the text file 'poem.txt'. As soon as the with block is exited, then the file is automatically closed and on printing the attribute *f.closed* we can see that the status returned is True indicating that the file object f is currently in a closed state and no read/write operations can now be done on the 'poem.txt' file.

## 7. DATA STRUCTURE

**Data Structure**

Data structure is the arrangement of data in such a way that permits efficient access and modification.

**Stack**

Stack is a linear data structure in which the operations are performed in the Last In First Out (LIFO) order.

Stack has two principle operations:

1. **Push** - It adds an element to the top / end of the stack
2. **Pop** - It deletes/removes elements from the top / end of the stack

**Applications of Stack:**

1. It is used to evaluate arithmetic expressions
2. It is used in backtracking algorithms to store the previous states onto a stack.
3. It is used to check if an expression has the same number of opening brackets and same number of closing brackets in a programming environment.
4. It is used in function calls to store the previous states on a stack.
5. It is used to keep track of undo and redo operations.

**Program 1: Implementing Stack using List**

```
# 1 Implementing Stacks using List
s=[]
while True:
    print('===========MENU==========')
    print('1. Push')
    print('2. Pop')
    print('3. Display')
    print('4. Exit')
    print('=========================')
    ch=int(input('Enter a choice:'))
    if ch==1:
        x=int(input('Enter an element to add to Stack:'))
        s.append(x)
        print('Added element to stack')
    elif ch==2:
        if s==[]:
            print('Stack is empty')
        else:
            print('Deleted element is:', s.pop())
    elif ch==3:
        if s==[]:
            print('Stack is empty')
        else:
            print('Stack is:', s)
    elif ch==4:
        break
    else:
        print('Wrong Input')
```

*o/p:*

*===========MENU==========*

*1. Push*

```
2. Pop
3. Display
4. Exit
=========================
Enter a choice:1
Enter an element to add to Stack:10
Added element to stack
==========MENU==========
1. Push
2. Pop
3. Display
4. Exit
=========================
Enter a choice:1
Enter an element to add to Stack:20
Added element to stack
==========MENU==========
1. Push
2. Pop
3. Display
4. Exit
=========================
Enter a choice:3
Stack is: [10, 20]
==========MENU==========
1. Push
2. Pop
3. Display
4. Exit
=========================
Enter a choice:2
Deleted element is: 20
==========MENU==========
1. Push
2. Pop
3. Display
4. Exit
=========================
Enter a choice:4
```

Additionally, the same program can be written using functions as shown below:

**Program 2 : Implementing Stacks using List and Functions**

```
# 2 Implementing Stacks using List and Functions
def push(L):
    x=int(input('Enter an element to add to Stack:'))
    L.append(x)
    print('Added element to stack')

def pop(L):
    if L==[]:
        print('Stack is empty')
    else:
        print('Deleted element is:', L.pop())

def display(L):
```

```
    if L==[]:
        print('Stack is empty')
    else:
        print('Stack is:', L)

s=[]
while True:
    print('===========MENU==========')
    print('1. Push')
    print('2. Pop')
    print('3. Display')
    print('4. Exit')
    print('=========================')
    ch=int(input('Enter a choice:'))
    if ch==1:
        push(s)
    elif ch==2:
        pop(s)
    elif ch==3:
        display(s)
    elif ch==4:
        break
    else:
        print('Wrong Input')
```

**Stacks with tuples as elements**

A stack can have tuples/list/dictionary or any data type as its element. Consider the case where we want to add a tuple having elements as (book number , book name ) .

```
# 3 Implementing Stacks of Book Details
def push(s):
    bno=int(input('Enter book number:'))
    bname=input('Enter book name:')
    b=(bno,bname)
    s.append(b)
    print('Element added to stack')

def pop(s):
    if s==[]:
        print('Stack is empty')
        return
    else:
        print('Element deleted is:', s.pop())

def display(s):
    if s==[]:
        print('Stack is empty')
    else:
        print('Stack is:', s)

stk=[]
while True:
    print('===========MENU==========')
    print('1. Push')
    print('2. Pop')
    print('3. Display')
    print('4. Exit')
```

```
    print('==========================')
    ch=int(input('Enter a choice:'))
    if ch==1:
        push(stk)
    elif ch==2:
        pop(stk)
    elif ch==3:
        display(stk)
    elif ch==4:
        print('bye')
        break
    else:
        print('Wrong Input')
```

*o/p:*

```
==========MENU==========
1. Push
2. Pop
3. Display
4. Exit
==========================

Enter a choice:1
Enter book number:101
Enter book name:abc
Element added to stack
==========MENU==========
1. Push
2. Pop
3. Display
4. Exit
==========================
Enter a choice:3
Stack is: [(101, 'abc')]
==========MENU==========
1. Push
2. Pop
3. Display
4. Exit
==========================
Enter a choice:2
Element deleted is: (101, 'abc')
==========MENU==========
1. Push
2. Pop
3. Display
4. Exit
==========================
Enter a choice:4
bye
```

# 8. COMPUTER NETWORKS

**Communication:** Communication is the act of sending messages from a one source to another destination through the use of mutually understood signs, symbols and rules.

**Data Communication:** Data communication refers to the exchange of digital data between a sender and a receiver over a transmission media such as a wire or even wirelessly.

## Components of Data Communication

1. **Sender:** A sender is a computer or any such device which is capable of sending data over a network. It can be a computer, mobile phone, smartwatch, walkie-talkie, video recording device, etc.
2. **Receiver:** A receiver is a computer or any such device which is capable of receiving data from the network. It can be any computer, printer, laptop, mobile phone, television, etc.
3. **Message:** It is the data or information that needs to be exchanged between the sender and the receiver. Messages can be in the form of text, number, image, audio, video, multimedia, etc.
4. **Communication/Transmission media/channel:** It is the path through which the message travels between source and destination. It is also called medium or link which is either wired or wireless. For example, a television cable, telephone cable, Ethernet cable, satellite link, microwaves, etc.
5. **Protocols:** It is a set of rules that need to be followed by the communicating parties in order to have successful and reliable data communication. E.g. Ethernet, HTTP etc.

### Measuring Capacity of Communication Media

The capacity of a channel is the maximum number of signals or traffic that a channel can carry. It is measured in terms of bandwidth and data transfer rate

1. **Bandwidth:** Bandwidth of a channel is the range of frequencies available for transmission of data through that channel. Higher the bandwidth, higher the data transfer rate. Normally, bandwidth is the difference of maximum and minimum frequency contained in the composite signals.

   Bandwidth is measured in Hertz (Hz).
   1 Kilo Hertz = 1 KHz = $10^3$ Hz = 1000 Hz
   1 Mega Hertz = 1 MHz = $10^6$ Hz = 1000 KHz = 1000000 Hz

2. **Data Transfer Rate/Bit rate:** Data transfer rate is the number of bits transmitted between source and destination in one second. It is measured in terms of bits per second (bps). Other units for data rates are:
   1 Kilo bit per second = 1 Kbps = $2^{10}$ bps = 1024 bps
   1 Mega bit per second = 1 Mbps = $2^{20}$ bps = 1024 Kbps
   1 Giga bit per second = 1 Gbps = $2^{30}$ bps = 1024 Mbps
   1 Tera bit per second = 1 Tbps = $2^{40}$ bps = 1024 Gbps

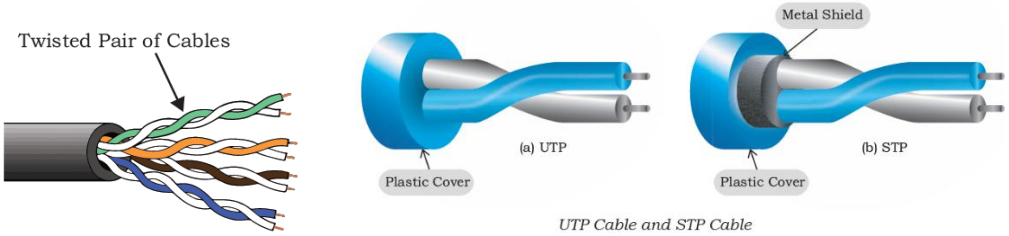
   **Example:** A user wants to upload a text document at the rate of 10 pages per 20 second. What will be the required data rate of the channel? (Assume that 1 page contains 1600 characters and each character is of 8 bits).
   **Solution:**
   Required data rate = (10x1600x8) / 20 = 6400 bps

*[NOTE- In computing terminology both the terms bit as well as bytes are used and the short forms of both bits and bytes are same alphabet, b. In order to differentiate between the two whenever small 'b' is mentioned then it refers to bit and whenever a capital 'B' is mentioned then it refers to byte. For example, Kbps - means Kilobits per second and KBps - means KiloBytes per second. ]*

**Types of Transmission Media**

There are two broad categories of Transmission Media- wired and wireless.

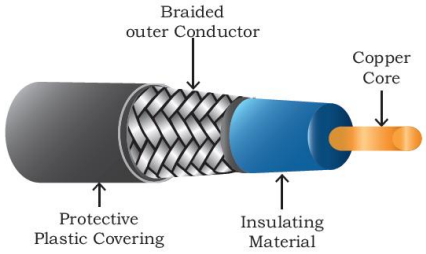
1. **Wired/Guided Transmission Media-** In this type of transmission media, there is a physical link made of wire/cable through which data in terms of signals are propagated between the nodes. These are usually metallic cable, Fiber-optic cable, etc. They can be further classified into:
    a. **Twisted Pair Cable:** A twisted-pair consists of two copper wires twisted like a DNA helical structure. Both the copper wires are insulated with plastic covers. Usually, a number of such pairs are combined together and covered with a protective outer wrapping.




*UTP Cable and STP Cable*

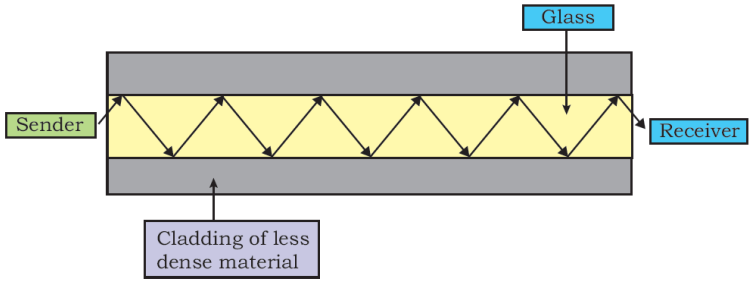
Twisted pairs are less expensive and most commonly used in telephone lines and LANs. These cables are of two types: **Unshielded twisted-pair (UTP)** and **Shielded twisted-pair (STP)** depending on whether they are enclosed in a metal shield. A metal shield in STP cable reduces noise and interference from the outside but is more expensive than UTP cable.

    b. **Coaxial cable:** Coaxial cable has a copper wire at the core/centre of the cable which is surrounded with insulating material. The insulator is further surrounded with an outer conductor (usually a copper mesh). This outer conductor is wrapped in a plastic cover. Due to its structure, coaxial cables are better shielded from noise outside and have more bandwidth than twisted pair cables. These types of cables are used to carry signals of higher frequencies to a longer distance and are used in cable TV, cables for security cameras/videos etc.



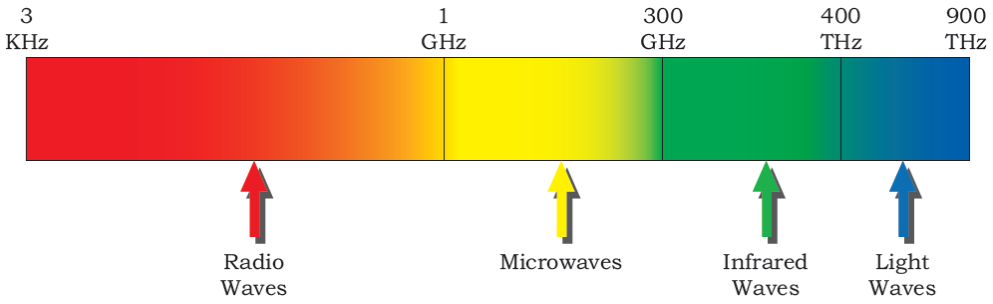

*A coaxial cable*

    c. **Optical Fibre:** Optical fibre has the highest data transfer rates usually in Gbps. They use light signals which are transmitted inside glass tubes to send the data. The glass tube at the centre of fibre optic cable is covered with a layer of less dense glass called cladding which is further covered with an outer jacket made of PVC or Teflon. Signals can travel in fibre optic cables over thousands of kilometres without degrading and electromagnetic noise cannot affect the signals travelling inside the cable. However optic fibres are expensive and are used in undersea cables and in backbone internet network of a country.




*Fiber optic cable*

2. **Wireless Transmission Media:** In wireless communication technology, information travels in the form of electromagnetic signals through air/space. The electromagnetic spectrum range (3KHz to 900THz) can be divided into 4 categories - Radio waves, Microwaves, Infrared waves and Visible or Light waves, according to their frequency ranges.




*Electromagnetic waves spectrum*

**a. Radio Waves**

i. Waves of frequency range 3 kHz - 1 GHz
ii. Omni-directional, these waves can move in all directions
iii. Radio waves of frequency 300KHz-30MHz can travel long distance
iv. Susceptible to interference
v. Radio waves of frequency 3-300 kHz can penetrate walls
vi. These waves are used in AM and FM radio, television, cordless phones.

**b. Microwaves**

i. Electromagnetic waves of frequency range 1GHz - 300GHz.
ii. Unidirectional, can move in only one direction.
iii. Cannot penetrate solid objects such as walls, hills or mountains.
iv. Needs line-of-sight propagation i.e. both communicating antenna must be in the direction of each other.
v. Used in point-to-point communication or unicast communication such as radar and satellite.
vi. Provide very large information-carrying capacity.

**c. Infrared waves**

i. Electromagnetic waves of frequency range 300GHz - 400THz.
ii. Very high frequency waves.
iii. Cannot penetrate solid objects such as walls.
iv. Used for short-distance point-to-point communication such as mobile-to-mobile, mobile-to-printer, remote-control-to-TV, and Bluetooth-enabled devices to other devices like mouse, keyboards etc.

**Network**: A group of two or more similar things or people interconnected with each other is called network

**Computer Network**: A computer network is an interconnection among two or more computers or computing devices.

**Node:** In a communication network, each device that is a part of a network and that can receive, create, store or send data is called a node. E.g. a node can be a device such as a modem, hub, bridge, switch, router, digital telephone handset, a printer, a computer or a server.

**Advantages of Computer Networks:**

1. Allows sharing of resources such as files, printers etc.
2. Faster communication
3. It increases efficiency by providing faster exchange of information
4. Much more data is available to a large number of people over the network which can be shared/collaborated to increase knowledge/capabilities of everybody

**Disadvantages of Computer Networks:**

1. Since everything is interconnected in a network, virus/spam/unwanted activities are also easier to perform over a network
2. Maintaining security of device connected to a network becomes challenging
3. The initial cost of setting up a network can be high
4. Maintaining a network that provides consistent performance under different conditions is also challenging

**Types of Networks**

Based on geographical area covered and data transfer rate, computer networks are broadly categorized as:

1. **PAN (Personal Area Network)**
   PAN is a network formed by connecting a few personal devices like computers, laptops, mobile phones, smart phones, printers etc., within an approximate range of 10 metres.
   E.g. Mobile phone connected with USB to a computer, two mobile phones connected via Bluetooth etc.

2. **LAN (Local Area Network)**
   LAN is a network that connects computers, mobile phones, tablet, mouse, printer, etc., placed at a limited distance usually within one kilometre range. The connectivity in a LAN is done by means of Ethernet cables, fibre optics, or Wi-Fi and the data transfer rates varies from 10Mbps to 1000Mbps.
   E.g. Network inside a school, office or a university campus.

3. **MAN (Metropolitan Area Network)**
   A MAN covers a larger geographical area like a city or a town usually up to a range of 30-40 KM.
   E.g. Cable TV network, Broadband Internet services etc.

4. **WAN (Wide Area Network)**
   A WAN spans a large geographical area and typically connects cities, countries, continents. It connects different LAN, MAN together into a single network.
   E.g. Internet is the largest WAN.

**Network Devices**

The Network Devices are used to connect the different devices and form a computer network. They are:

1. **Port**

   A port is a physical socket through which devices/network are connected to a computer. In a wireless communication such as Bluetooth/Wi-Fi the chip/port making the connection may not be physically visible from the outside.

2. **MODEM (MODulator DEModulator)**

   A computer works on binary signals 0 and 1. These signals need to be carried over a network through physical medium such as wire or wireless media. The signals in the wired/wireless media are analog in nature. The MODEM performs the interconversion between the digital signals and the analog signals.

3. **Ethernet Card/Network Interface Card (NIC)**

   The Ethernet card is a circuit board connected to the motherboard of the computer which permits a wired connection between the computer and the network. Typical Data transfer rates are from 10Mbps to 1000 Mbps.

   Each network card has a **MAC (Media Access Control)** address which is a number given by the network card manufacturer and is used to uniquely identify the network card/computer on the network. Each MAC address is a 12-digit hexadecimal numbers (48 bits in length), of which the first six digits (24 bits) contain the manufacturer's ID called Organisational Unique Identifier (OUI) and the later six digits (24 bits) represents the serial number assigned to the card by the manufacturer. A sample MAC address looks like:

4. **RJ45 (Registered Jack 45)**

   RJ45 is an 8 pin connector used to connect Ethernet cables to an Ethernet card/port on a computer/networking device.

5. **Repeater**

   In a wired network, signals loose strength with increasing distance, usually beyond 100 metres. So the weakened signals need to be regenerated. A repeater regenerates weak signals on a wired connection.

6. **Hub/Ethernet Hub**

   A Hub is a networking device used to connect together different devices over a wired Ethernet cable. In a Hub at any point of time only two devices can communicate with each other. Also, when a sender sends some communication/message to a hub, the hub broadcasts it to all the other ports on the hub. Only the intended recipient will accept the message and all the others will discard the message. Hence a hub is less secure compared to a switch.

7. **Switch**

   A switch is also a networking device used to make a wired network connection using Ethernet cables. The difference between a switch and a Hub is that when a sender sends a communication/message to the switch, the switch will forward it to only the intended recipient. It can do so as it maintains a table which lists all the networking devices connected to it. Another difference is that in a switch simultaneous communication between two different pairs of devices is possible. Hence a switch is more efficient than a Hub and so it is also known as an **'intelligent hub'**.

8. **Router**
   A router connects many networks together. The different networks connected to a router may follow same or different protocols. The router maintains an internal table about the protocols as well as the devices on the different networks. When a message comes to a router, it can take an intelligent decision based on the prevailing network conditions as to which network is best suitable to forward the message.
   E.g. routers perform traffic directing functions on the Internet, Wi-Fi routers connect wired and wireless network together etc.

9. **Gateway**
   A gateway is a networking device through which an organization's internal network is connected to the outside network, typically the internet. A gateway can be implemented in hardware, software or both. A gateway can perform conversions between the protocols followed within the organization and the protocols followed by the network outside. Also security features such as firewalls, authentication mechanisms are also applied at the gateway. Nowadays gateway features are also integrated within routers and switches.
   E.g. On home networks the broadband router serves as the gateway, For small business a computer can also configured to work as a gateway etc.

10. **Bluetooth:** Bluetooth is a short-range wireless technology that can be used to connect mobile-phones, mouse, headphones, keyboards, computers, etc. wirelessly over a short distance.

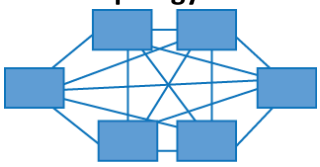
11. **Wireless LAN/ Wi-Fi (Wi**reless **Fi**delity**)**
    It is a wireless network that can connect devices such as computers, smart phones etc wirelessly using the wireless LAN protocol IEEE 802.11. A device called **Access point (AP**) is used to provide Wireless LAN facility. Devices with a Wi-Fi network card can connect to the Wireless LAN using authorized credentials.

12. **WIMAX**
    WIMAX is also a Wireless broadband communication protocol based on the IEEE 802.16 protocol. It is similar to Wi-Fi but provides greater speeds and for longer distances. It has low setup costs and is used to provide last-mile broadband connectivity wirelessly.

**Network Topology**

The arrangement of computers and other peripherals in a network is called its topology. Common network topologies are Mesh, Ring, Bus, Star and Tree.

1. **Mesh Topology**



In mesh topology each device is connected with every other device in the network.

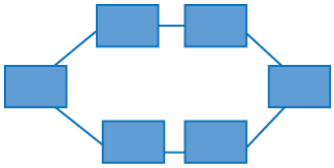
**Advantages**

a) Can handle large amounts of data
b) It is more reliable. If one link is down another link can be used.
c) More secure as each cable carries different data
d) More efficient as simultaneous communication between different nodes is possible

**Disadvantages**

a) Wiring is complex
b) High cost

2. **Ring Topology**



In ring topology each node is connected to its two neighbours to form a ring. Data can be sent over a ring network in one direction only.

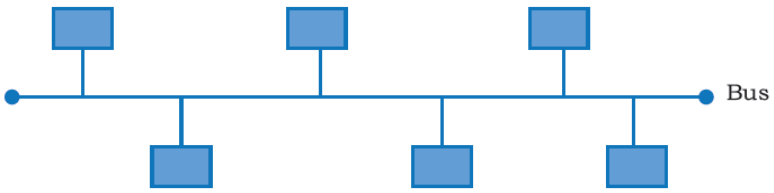
**Advantages**

a) Easy to install and configure
b) Less cost

**Disadvantages**

a) Failure at single node causes entire network to fail.
b) More time taken for communication
c) Less secure as data is passed through multiple nodes before reaching the destination.

3. **Bus Topology**




In Bus topology all devices are connected to a common backbone wire known as Bus. All data is transmitted through the Bus.

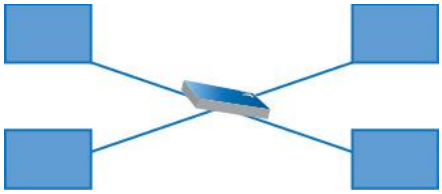
**Advantages**

a) Easy to install and configure
b) Less cost

**Disadvantages**

a) Failure of bus brings the network down
b) Less efficient as only two devices can communicate at a time over the common bus
c) Less easy to find faults in the network
d) Less secure as any node can read the message not intended for it.

4. **Star Topology**



In star topology each device is connected to a central node, which is a networking device like a hub or a switch.

**Advantages**

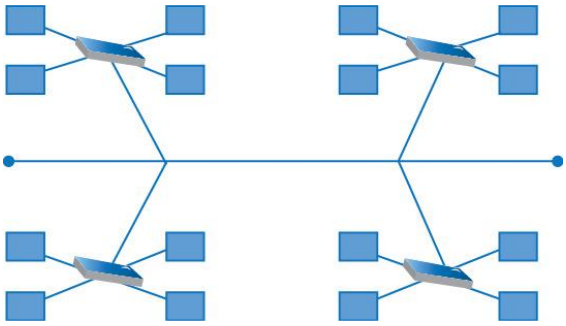
a) Very efficient
b) Failure of one line will not bring the entire network down.
c) Easy to find and rectify fault

**Disadvantages**

a) High initial cost of wires
b) If central networking device fails then entire network is down

**5. Tree/Hybrid Topology**

In tree/hybrid topology a hierarchy is there with multiple branches, with each branch being a different kind of network such as a ring, star, bus. This kind of topology is commonly found in WANs which connect together different kinds of neworks.



E.g.

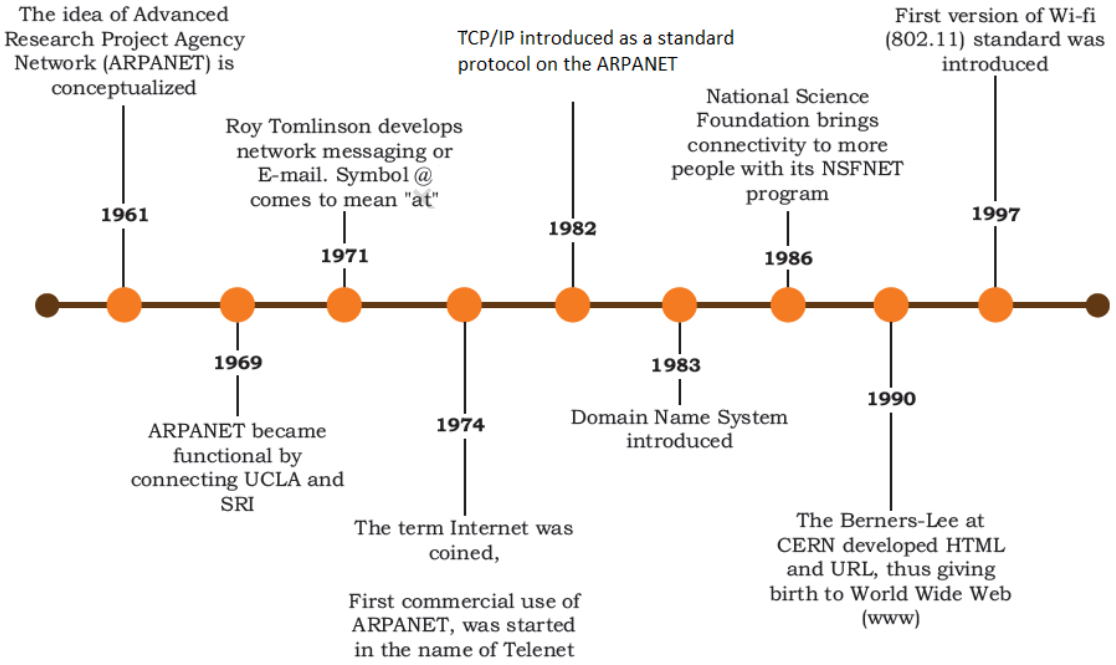
**Internet**

The Internet is the largest global WAN network of computing devices. Access to the internet is provided by Internet Service Providers (ISP) which usually is the telecom operator or your local Cable TV network provider.

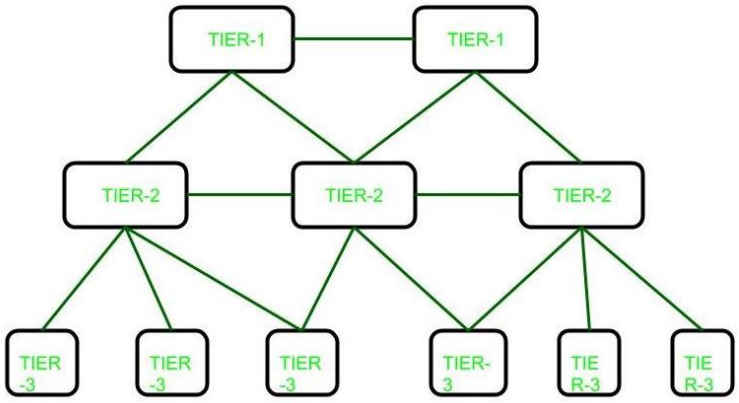
**Interspace:** Interspace is a software program that allows multiple users to communicate online with real-time audio, video and text chat in dynamic 3D environments. The Interspace is a vision of what the Internet will become in future.

**Evolution of Internet**

In the 1960s a research project was commissioned by Advanced Research Projects Agency Network (ARPANET) in the U.S. Department of Defence to connect the academic and research institutions located at different places for scientific collaborations. The first message was communicated between the University of California, Los Angeles (UCLA) and Stanford Research Institute (SRI). More and more organisations joined the ARPANET, as well as many independent smaller networks were formed. Few of the milestones in the journey of evolution of computer networks are depicted in the timeline below.



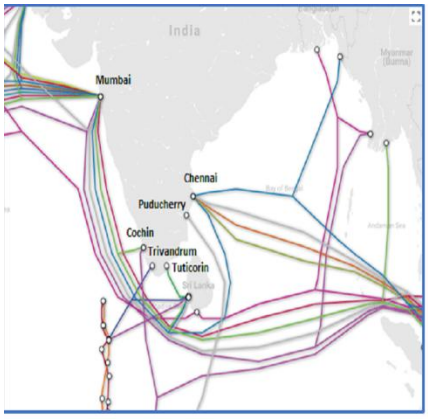

**Networks comprising the of Internet**




Internet comprises of different kinds of networks interconnected together. Different categories of networks- **Tier 1**, **Tier 2** and **Tier 3** networks are interconnected by fibre optic links. Fiber optic is the fastest mode of data communication. Tier 1 networks are global long-distance networks; Tier 2 are multi-city networks and Tier 3 are your local ISPs such as cable TV networks etc. from whom you can purchase your broadband/internet packages. Usually there is an overlap between Tier 2 and Tier 3 networks, which are usually your telecom service providers.

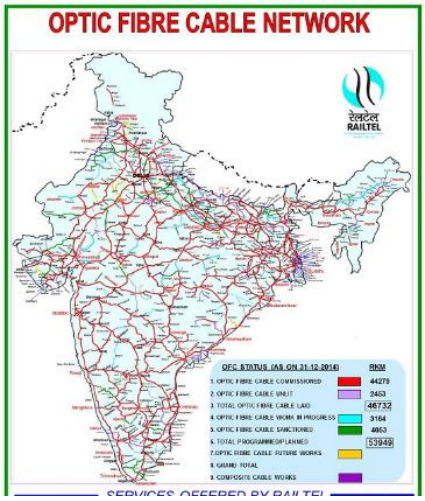
**How is India connected to the Internet?**

India has undersea fibre optic landing stations at Mumbai, Chennai, Cochin, Trivandrum, Tuticorin, Puducherry, Digha, Vishakhapatnam.




| Distribution of Cable Landing Stations and providers that own them | Mumbai | Chennai | Cochin | Trivandrum | Tuticorine | Puduchery | Digha | Visakhapatnam |
|---|---|---|---|---|---|---|---|---|
| Tata Communications | 3 | 1 | 1 | | | | | |
| Airtel | 1 | 2 | | | | | | |
| Reliance Jio | 1 | 1 | | | | | | 1 |
| Global Cloud Xchange | 1 | | | 1 | | | | |
| Vodafone | 1 | | | | | | | |
| Sify | 1 | | | | | | | |
| BSNL | | | | | 1 | | 1 | |
| IOX | | | | | | 1 | | |

CLS Operational  CLS Under Construction




In addition, there are fibre optic networks within the country set up by different agencies such as RailTel, National Fibre Optic Network, BSNL etc.

When you make a data connection on your mobile/broadband, the ISP will make the connection with the nearest fibre optic network within India and transfer that request over the internal fibre optic network in the country. The request is then passed through one of the fibre optic landing stations at Mumbai, Chennai, Cochin, Trivandrum, Tuticorin, Puducherry, Digha or Vishakhapatnam which is then transmitted by undersea fibre optic cable to other parts of the world where the remote website is situated.

**Different ways of sending data over a network**

1. **Circuit switching:** Circuit switching network establishes a fixed bandwidth circuit (channel) between two nodes before the users may communicate, as if the nodes were physically connected with an electrical circuit. Each circuit so established cannot be used by other callers until the circuit has been released. The wired landline / Public switched Telephone Network (PSTN) is an example of circuit switched network. Voice calls on 2G and 3G mobile networks are circuit switched.
2. **Message switching:** In Message switching, the messages are routed in their entirety, one hop/one node at a time. Message switching was the precursor of packet switching. Nowadays, message switching systems are mostly implemented over packet-switched or circuit-switched data networks. E-mail is example of a message switching system.
3. **Packet switching:** In packet switching the message to be sent over a network is broken down into small pieces called packets which are typically 200 bytes on an average. Each packet contains a header which stores its sequence number in the original message as well as the destination address. Each packet is routed through the network independently and may follow different paths. When all the packets arrive at the destination, the message is reconstructed using the information in all the headers. Many packets from different users can be shared on a line as and when packets arrive at an intermediate node, so it is more efficient. EDGE and GPRS on 2G networks use packet switching, 3G uses packet switching for data,4G uses packet switching for voice as well as data, 5G is fully packet switched.

**Protocols Used for Data Communication**

1. **Transmission Control Protocol/Internet Protocol (TCP/IP): TCP**/IP is a networking protocol that allows two computers to communicate. A message sent over TCP/IP is broken down into packets of roughly 200 bytes.

   **The TCP protocol part** maintains end-to-end communication between the sender and the receiver. TCP takes care of breaking the packets and giving the packets a sequence number at the sending end. At the receiving end TCP makes sure that all the packets have arrived and then assembles the message by ordering the packets into sequence.

   The **IP protocol part** takes each packet and routes each packet independently of other packets of the same message. The advantage is that since packets are small, they can be forwarded immediately to the next node in the network. Many packets of different users can be sent simultaneously and this increases the speed and efficiency of the network as a whole.

2. **Hyper Text Transfer Protocol (HTTP):** HTTP is a client-server protocol, in which requests are initiated by the recipient/client, usually the Web browser and the web server gives/serves a response to the client in the form of a html file/resource/any document. HTTP is the foundation of data communication on the World Wide Web. HTTP was initiated by Tim Berners-Lee at CERN in 1989.

3. **File Transfer Protocol (FTP)**: FTP is a standard network protocol used for the transfer of computer files between a client and server on a computer network.

4. **Point-to-Point Protocol (PPP):** PPP is a communications protocol between two nodes on a network which are directly connected to each other without any other device in between them. PPP can provide connection authentication, transmission encryption, and compression. PPP is used to establish connection between a landline DSL (Digital Subscriber Line) router and the ISP, to establish connection over a fibre optic link, etc.

5. **Telnet:** Telnet is an application protocol used on the Internet or local area network to provide a bidirectional interactive text-oriented communication facility using a virtual terminal connection.

6. **General Packet Radio Service (GPRS):** GPRS is a packet oriented mobile data standard on the 2G and 3G cellular communication network's global system for mobile communications (GSM)

7. **Wireless local loop (WLL):** WLL is the use of a wireless communications link for delivering plain old telephone service (POTS) or Internet access to telecommunications customers.

8. **Voice over Internet Protocol (VoIP) / IP Telephony:** VoIP is a protocol/method of using the internet / IP protocol for making voice calls ***as well as video calls.***

9. **Video Conferencing/ Videotelephony/ video teleconference**
   It refers to technologies for the reception and transmission of audio-video signals by users in different locations, for communication between people in real time. Some of the software used for video conferencing are Zoom, Google Meet, Cisco Webex etc. Many protocols are used in video conferencing:

| Protocol | Primary Role | Special Features & Usage | Examples (Apps/Hardware/Software) |
|---|---|---|---|
| **VoIP** | Core technology | Transmits voice and video over IP networks; foundation of modern communication. | Skype, WhatsApp, Microsoft Teams, Cisco IP Phones |
| **SIP** | Signaling | Manages call setup, modification, and termination; widely adopted in VoIP/video conferencing. | Zoom, Cisco Webex, Avaya systems, Asterisk PBX |
| **H.323** | Multimedia | Legacy protocol suite for video | Polycom video conferencing |

| Protocol | Primary Role | Special Features & Usage | Examples (Apps/Hardware/Software) |
|---|---|---|---|
| | communication suite | conferencing; comprehensive but complex. | systems, Cisco TelePresence, Lifesize endpoints |
| **RTP** | Media transport | Real-time delivery of audio and video; ensures synchronization. | Used in Zoom, Teams, Webex, and SIP-based phones |
| **RTCP** | Control protocol | Provides feedback on quality, jitter, and packet loss; helps maintain performance. | Integrated into RTP-based apps like Skype, Teams, and WebRTC |
| **WebRTC** | Browser-based RTC | Enables peer-to-peer video/audio directly in browsers; secure with DTLS/SRTP. | Google Meet, WhatsApp Web, Discord, Firefox/Chrome browsers |
| **UDP** | Transport layer | Fast, low latency transport; commonly used for real-time video. | Backbone transport in Zoom, Teams, and gaming voice/video chats |
| **TCP** | Transport layer | Reliable, ordered delivery; used when data integrity is critical. | File transfer during calls in Skype, fallback transport in WebRTC |
| **QUIC** | Modern transport | Combines speed of UDP with reliability of TCP; optimized for streaming. | Google Meet, YouTube streaming, Chrome browser communications |

**How protocols map directly to real-world platforms**:

- **VoIP** is the umbrella technology.
- **SIP/H.323** are the signalling frameworks.
- **RTP/RTCP** handle the actual media streams.
- **WebRTC** powers browser-based video calls.
- **UDP/TCP/QUIC** are the transport layers underneath.

**Internet Protocol (IP) address**

Whenever a device makes a connection with an Internet Service Provider (ISP), the ISP will assign a unique number to that particular connection. This unique number which identifies each device connected to the network is known as the IP Address.

If a company sets up its own network which is not connected to Internet, then it can assign each device a **fixed IP address.** Another way of allocating IP address is to let the job be done by a special machine/computer known as **DHCP (Dynamic Host Control Protocol)** server. A DHCP server has a pool of IP addresses with it. Whenever a system/device comes online then the DHCP server can allocate a unique IP address from its pool temporarily to that machine for a fixed amount of time. This method of obtaining an IP address from a DHCP server is known as **dynamic IP address**. Dynamic IP address is valid only for a specific amount of time.

Normally when we use a data connection on mobile/broadband router, the DHCP server of the ISP allocates a dynamic IP address to our machine to identify it over the Internet for a particular session only.

The current version of IP Address is version 4(IPv4), which is a 32-bit address written as groups of four decimal numbers ranging from 0-255 and separated by dots. Eg. 192.168.1.1, 127.0.0.1 etc.

The next generation of IP Address is version 6 (IPv6) which has 128-bit numbers to identify a very large number of devices connected to the internet. It is represented by 8 groups of hexadecimal digits (0 to 9 and A to F) separated by colons. Eg. *2001:0db8:0000:0000:0000:8a2e:0370:7334*

**World Wide Web (WWW) / The Web**

The World Wide Web is an information storage system in which different kinds of resources such as files/images/music files / video files can be stored in an interlinked form. The protocol used on the web is the Hyper Text Transfer Protocol (HTTP).

**Web Browser**
The web browser is the software application used to access resources on the internet. E.g. Chrome, Firefox, Internet Explorer etc.

**Hyper Text Markup Language (HTML)**
HTML is the language used to write the documents that are hosted on the Internet. The documents written in HTML are also known as **web pages / html pages**. The individual web pages/ resources are linked together by clickable links known as hyperlinks.

**Static / Dynamic web pages**
**Static web pages** are written in HTML language only. The content of the web page is fixed and it will always display the same content each time it is opened.

In **dynamic web page** in addition to HTML language additional scripting language is used inside the web page, which can change what is displayed finally in the web browser. The scripting language is of two types - client-side scripting and server-side scripting.

**Client-side scripting**
If the scripting language is embedded into the pages that are delivered to the user and the script is executed in the *web browser of the user* requesting the web page, then it is known as client-side scripting. Examples of client-side scripting language are - JavaScript, VBScript, CSS, AJAX, jQuery, PHP etc.

Client-side scripting being used on any web page can be seen by right clicking any web page in a browser and clicking the 'View Source' or similar options.

**Server-side scripting**
In server-side scripting, request from the user's web browser is caught by the webserver. The webserver then parses the query and then constructs a new webpage at that instant and then sends it back to the user's web browser. Here all the scripting code is present on the webserver and the user does not need to know which/what code was executed at the server, only the processed output is presented to the web browser. Examples of server-side scripting language are - PHP, ASP.NET, C++, Java, Python etc.

Nowadays all websites usually use server-side scripting for displaying web pages. We can identify visually that server-side scripting is being done by a web server by looking at the address. If it contains anything containing .jsp or .asp or any long sequence of characters then it is usually served by a server-side script. We cannot see the code of a server-side script from a client web browser.

**Extensible Markup Language (XML)**
XML is a markup language that defines a set of rules for encoding documents in a format that is both human-readable and machine-readable. It is a text-oriented data in Unicode. XML provides a software- and hardware-independent way of storing, transporting, and sharing data.

**Website**
A website is a collection of web pages that is logically organized under a common name, called the **domain name or the website address**. E.g. www.wikipedia.org, www.youtube.com etc.

**Web Server**
A Web Server is a combination of hardware and software that is used to host a Web site. A web server can satisfy the requests coming from a web browser to display the web pages that have been hosted by the web server. Examples of Web Server are - Apache (Open Source), nginx (open source), IIS (By Microsoft), etc.

**Web Hosting**
Web Hosting is a service provided by companies in which space and facilities on a webserver are leased out to many users desirous of hosting their own website. E.g. hostgator.com, godaddy.com etc.

**Uniform Resource Locator (URL)**
A URL refers to the location of a resource (file/image/audio/video) on the network/internet. It also specifies the protocol/mechanism to access that particular resource. URLs are also known as web address and they are displayed in the address bar of the web browser.
E.g. http://www.example.com/dir1/dir2/index.html
http - is the protocol used
www.example.com - is the domain name
/dir1/dir2 shows the location of the resource within the webserver
index.html - is the name of the resource/file that we wish to access

**Electronic mail / Email**
Email is a method of exchanging messages on the Internet. Several companies provide free email services on the internet and provide web interfaces to access the emails. Eg. www.gmail.com, www.outlook.com, ymail etc.

**Email client**
An email client is software which we can use to access our emails which are stored on an email server on the internet. E.g. Microsoft Outlook, Mozilla Thunderbird, etc.
The protocols used for sending / receiving emails are:

1. **Post Office Protocol version 3 (POP3)** - This protocol is used to receive mails and it downloads all the messages on the email server and deletes them from the email server.
2. **Internet Message Access Protocol (IMAP)** - This protocol is also used to receive mails. It has the additional feature that it allows us to leave back the copy of the emails on the email server after downloading all the messages on our computer.

3. **Simple Mail Transfer Protocol (SMTP)** - It is the protocol used *by email clients* to only **send emails**. SMTP protocols can be used *by email servers* to **send as well as receive** emails from *other email servers*.

**Chat**

A chat is a location on the internet/websites where multiple people can talk/write messages to each other. A chat can also have features that allow sending a private message (PM) or a direct message (DM) directly to another person in the chat room directly. Chatting with unknown persons on the internet poses a security risk and caution must be maintained to not reveal any personal information during a chat.

**Browser Settings**

We access the Internet using a web browser. Setting in browser can affect the safety and security of the device which we are using. The browser can also be a window for hackers to gain access to our device and misuse the files/photos/information present in our device.

Commonly used browser settings for safety and security are:

1. **Default Search Engine**- This option allows us to use the search engine of our choice. Choosing a proper search engine is needed as all the search queries are recorded by the search engine provider.
2. **Tracking protection/Do Not Track request**- This option can be used to prevent trackers/advertising agencies which can track and record our browsing behaviour across multiple sites and build up a user profile.
3. **Cookies and Site Data**: Cookies are small text files that websites can use to store information about the pages and selections you have made on a website. Cookies can also be persistent that may be present for a long time and record browsing behaviour across different web sites (third party cookies). So usually it is a good practice to clear cookies and allow cookies on only the sites that you trust.
4. **Permissions for Location/Camera/Microphone**: Allow only trusted websites access to your location/camera/microphone as it can be misused.
5. **Add-Ons / plugins / Extensions:** They are used to add some additional feature to your web-browser. But the add-on/plugin/extension that is added to your web-browser can either read a single web page/particular web site or all the web pages of the web browser and so should be installed from only trusted sources. Remove/uninstall add-ons that you don't need.
6. **Updates:** Update your browser regularly to remove the security holes in it.

**Web 2.0**: It refers to websites that emphasize user-generated content, ease of use, participatory culture and interoperability for end users.

**Mobile Communications Technology**

1. **1G:** 1G refers to the first generation of wireless cellular technology (mobile telecommunications). They used analog signals and were introduced in the 1980s until they were replaced by 2G digital telecommunications in the mid-1990s. In 1G the voice signals were modulated to a higher frequency typically 150MHz and up and then transmitted. Almost all 1G operations have stopped now.

2. **2G:** 2G stands for second-generation cellular network. 2G comprised of two different kind of technologies- **GSM and CDMA.** The primary benefits of 2G networks over their predecessors were:
    a. Digitally encrypted phone conversations, at least between the mobile phone and the cellular base station but not necessarily in the rest of the network.
    b. Significantly more efficient use of the radio frequency spectrum enabling more users per frequency band.
    c. Data services for mobile, starting with SMS text messages.

3. **Global System for Mobile Communications (GSM):** GSM is a standard developed by the European Telecommunications Standards Institute (ETSI) to describe the protocols for second-generation (2G) digital cellular networks used by mobile devices such as mobile phones and tablets. It was first deployed in Finland in December 1991. By the mid-2010s, it became a global standard for mobile communications achieving over 90% market share.

GSM divides the entire spectrum available to a telecom service provider and divides it into roughly 124 to 299 channels each for uplink (sending) and downlink (receiving). When a subscriber places a call, the call is digitized and digital packets containing voice data are multiplexed in time over a particular channel, i.e. Time Division Multiplexing (TDM) of different calls at different points in time is done and so efficiency is improved as a whole for all calls made.

4. **Code-division multiple access (CDMA):** CDMA is a communication technologies where several mobile phones can send information simultaneously over a range of frequencies using spread spectrum technology and a special coding scheme which prevents interference between calls of two mobile devices.

   In CDMA the entire spectrum available to a telecom service provider is used by each CDMA phone. But while modulating the voice signal over the carrier each phone uses a different code to modulate over different region of the spectrum.

   Although among 2G technologies CDMA was not widely used but the concept of CDMA specifically Wideband CDMA has been used in the 3G generation of mobile technology.

5. **General Packet Radio Service (GPRS):** GPRS is a packet oriented mobile data standard on the 2G and 3G cellular communication network's global system for mobile communications (GSM)

6. **Enhanced Data rates for GSM Evolution (EDGE):** EDGE is a digital mobile phone technology that allows improved data transmission rates in 2G and 2.5G mobile networks.

7. **3G:** 3G stands for the third generation of wireless mobile telecommunications technology. It is the upgrade for 2G and 2.5G GPRS networks which permits faster data transfer. 3G mobile telephony was relatively slow to be adopted globally as 3G does not use the same radio frequencies as 2G. Mobile operators must build entirely new networks and license entirely new frequencies to achieve high data transmission rates.

   Several different technologies are branded under 3G, which includes:
   - **UMTS (Universal Mobile Telecommunications System)** which further is divided into
     - **W-CDMA (Wideband Code Division Multiple Access)** most widely used in the world including India
     - **Time Division Synchronous Code Division Multiple Access (TD-SCDMA)** used primarily in China
     - **Evolved High Speed Packet Access, or HSPA+**
   - **CDMA2000** system used especially in North America and South Korea

   From 3G onwards the standards for 3G / 4G / 5G has been set by the **3GPP (3rd Generation Partnership Project)** which is a consortium of standards organization which includes **Telecommunications Standards Development Society of India (TSDSI).**

8. **4G:** 4G is the fourth generation of broadband cellular network technology. 4G is fully digital packet switched network and supports speeds of up to 100Mbps.

9. **5G:** 5G is the fifth generation technology standard for broadband cellular networks, which started operations in 2019. 5G networks have greater bandwidth and support speeds of up to 10Gbps.

10. **Mobile Processors:** A Mobile processor is a CPU chip designed for laptops, tablets, smartphones and other portable devices. Mobile processors use lower voltages and are designed to run cooler than the CPUs in desktop computers and do not need fans to cool them. They typically have more "sleep modes" that enable sections of the chip to be throttled down or turned off when not used. The most widely used mobile processors are ARM chips. E.g. Qualcomm Snapdragon , Apple Bionic , Samsung Exynos etc.

**Network Security:** Network security is concerned with protection of our device as well as data from illegitimate access or misuse.

**Malware (Malicious software):** Malware is any software developed with an intention to damage hardware devices, steal data, or cause any other trouble to the user. Viruses, Worms, Ransomware, Trojans, and Spyware are some of the kinds of malware.

1. **Virus:** A virus is a piece of software code created to perform malicious activities and hamper resources of a computer system like CPU time, memory, personal files, or sensitive information. a computer virus infects other computer systems that it comes into contact with by copying or inserting its code into the computer programs or software (executable files). A virus remains dormant on a system and is activated as soon as the infected file is opened (executed) by a user. Some of the most common intentions or motives behind viruses include stealing passwords or data, corrupting files, spamming the user's email contacts, and even taking control of the user's machine. Some well-known viruses include CryptoLocker, ILOVEYOU, MyDoom, Sasser and Netsky, Slammer, Stuxnet, etc.
2. **Worms:** Worms are also malicious software but they are standalone programs that are capable of working on its own. Whereas a virus needs human trigger for replication (i.e. when a user pens/executes the infected file), a worm replicates on its own and can spread to other computers through the network. Some prominent examples of worms include Storm Worm, Sobig, MSBlast, Code Red, Nimda, Morris Worm, etc.
3. **Ransomware:** It is a type of malware that targets user data. It either blocks the user from accessing their own data or threatens to publish the personal data online and demands ransom payment against the same. Example of ransomware is WannaCry which infected around 200,000 computers across 150 countries in May 2017.
4. **Trojan:** A Trojan is a malware, that looks like a legitimate software and once it tricks a user into installing it, it acts pretty much like a virus or worm. However, a Trojan does not self-replicate or infect other files, it spreads through user interaction such as opening an email attachment or downloading and executing a file from the Internet. Some Trojans create backdoors to give malicious users access to the system.
5. **Spyware:** It is a type of malware that spies on a person or an organisation by gathering information about them, without the knowledge of the user. It records and sends the collected information to an external entity without consent or knowledge of the user.
6. **Adware:** An Adware is a malware that is created to generate revenue for its developer. An adware displays online advertisements using pop-ups, web pages, or installation screens.
7. **Key loggers:** A key logger makes logs of daily keyboard usage and sends it to an external entity. In this way, very sensitive and personal information like passwords, emails, private conversations, etc. can be revealed to an external entity without the knowledge of the user.

**Modes of Malware distribution**

Malware is distributed by many ways. Some of them are:

1. When you download something from the internet it may contain malware.
2. Clicking on any spam email may install malware in your system
3. Using removable media such as pen drives can copy the malware from one system to another.
4. Worms can exploit weakness in the system to copy itself over the network/internet.

**How to detect malware in your system?**

The following signs usually show up when a system is infected by malware:

- Frequent pop-up windows prompting you to visit some website and/or download some software;
- Changes to the default homepage of your web browser;
- Mass emails being sent from your email account;
- Unusually slow computer with frequent crashes;
- Unknown programs startup as you turn on your computer;
- Programs opening and closing automatically;
- Sudden lack of storage space, random messages, sounds, or music start to appear;
- Programs or files appear or disappear without your knowledge.

**Preventive measures against malware**

- Using antivirus, anti-malware, and other related software and updating them on a regular basis.
- Configure your browser security settings
- Always check for a lock button in the address bar while making payments.
- Never use pirated on unlicensed software. Instead go for Free and Open Source Software (FOSS).
- Applying software updates and patches released by its manufacturers.
- Taking a regular backup of important data.
- Enforcing firewall protection in the network.
- Avoid entering sensitive (passwords, pins) or personal information on unknown or public computers.
- Avoid entering sensitive data on an unknown network (like Wi-Fi in a public place) using your own computer.
- Avoid clicking on links or downloading attachments from unsolicited emails.
- Scan any removable storage device with an antivirus software before transferring data to and from it.
- Never share your online account or banking password/pins with anyone.
- Remove all the programs that you don't recognise from your system.
- Do not install an anti-spyware or antivirus program presented to you in a pop-up or ad.
- Use the pop-up windows 'X' icon located on the top-right of the popup to close the ad instead of clicking on the 'close' button in the pop-up. If you notice an installation has been started, cancel immediately to avoid further damage.

**Antivirus:** Antivirus is software that detects and removes viruses as well as other malware from a system.

**Methods of Malware Identification used by Antivirus**
**(A) Signature-based detection:** An antivirus works with the help of a signature database known as "Virus Definition File (VDF)" which consists of virus signatures and must be updated regularly.
**(B) Sandbox detection:** In this method, a new application or file is executed in a virtual environment (sandbox) and its behavioural fingerprint is observed for a possible malware.
**(C) Data mining techniques:** This method employs various data mining and machine learning techniques to classify the behaviour of a file as either benign or malicious.
**(D) Heuristics:** Here, the source code of a suspected program is compared to the code of viruses present in the heuristic database. If the majority of the source code matches with any code in the heuristic database, then the code is flagged as a possible threat.
**(E) Real-time protection:** Here the antivirus program always runs in the background and checks the running processes in the computer's memory for presence of any virus/malware.

**Spam:** Spam is any unsolicited email/message that repeatedly sends advertisement links and invitation emails to a large number of users.

**Hyper Text Transfer Protocol Secure (HTTPS):** HTTPS sends encrypted information over the internet. A website is a secure site if it begins with https:// and the browser shows a lock symbol in front of it to say that the site has valid security credentials.

**Firewall:** A Firewall is a software or hardware or a combination of both that continuously monitors and controls the incoming and outgoing traffic. It blocks any unauthorised communication or any communication that is not permitted in its rule book.

**Cookies:** A computer cookie is a small file, which is stored by a website on the client's computer. Cookies are used by the websites to store browsing information/security credentials/selections made by the user. Third party cookies are cookies that are persistent across different websites and can track user behaviour across different websites and are a security concern.

**Hackers and Crackers:** Hackers and crackers are people having a thorough knowledge of the computer systems, system software (operating system), computer networks, and programming. They use this knowledge to find loopholes and vulnerabilities in computer systems or computer networks and gain access to unauthorised information. Depending on the intent, there are different types of hackers-

1. **White Hat/Ethical Hacker:** They use their knowledge to find and help in fixing the security flaws in the system

2. **Black Hats/Crackers:** They use their knowledge unethically to break the law and disrupt security by exploiting the flaws and loopholes in a system.

3. **Grey Hats:** This class of hackers are neutral, they hack systems by exploiting its vulnerabilities, but they don't do so for monetary or political gains. They hack systems for the fun of it.

**Network Security Threats**

1. **Denial of Service (DoS):** Denial of Service (DoS) is a situation in which an attacker (Hacker) limits or stops an authorised user to access a service, device, or any such resource by overloading that resource with illegitimate requests. For example flooding a website with many fake requests will slow down a website.

   A subcategory of DoS attack is Distributed Denial of Service (DDoS) in which compromised computer (Zombies) systems distributed across the globe are used to generate fake requests.

2. **Network Intrusion:** Network Intrusion refers to any unauthorised activity on a computer network. Some examples of Network Intrusion are DoS, Trojans, Worms. Some other examples are:
   a. **Asymmetric Routing:** Here the attacker tends to avoid detection by sending the intrusion packets through multiple paths, thereby bypassing the network intrusion sensors.
   b. **Buffer Overflow Attacks:** In this attack, the attacker overwrites certain memory areas of the computers within the network with code (set of commands) that will be executed later when the buffer overflow (programming error) occurs. Once the malicious code is executed, an attacker can initiate a DoS attack or gain access to the network.
   c. **Traffic Flooding:** This method involves flooding the network intrusion detection system with message packets. This huge load leaves the network detection system incapable of monitoring the packets adequately. The hacker takes advantage of this congested and chaotic network environment to sneak into the system undetected.

3. **Snooping:** In this attack, the hacker taps or listens to a channel of communication by picking all of the traffic passing through it. Once the network packets are analysed by the snooping device or software, it reproduces the exact traffic packets and places them back in the channel, as if nothing has happened.

4. **Eavesdropping:** Eavesdropping is an unauthorised real-time interception or monitoring of private communication between two entities over a network. Eavesdropping happens in real-time whereas snooping is not real time.

**Intellectual Property Right (IPR)**

Intellectual Property refers to the inventions, literary and artistic expressions, designs and symbols, names and logos. The ownership of such concepts lies with the creator, or the holder of the intellectual property. This enables the creator or copyright owner to earn recognition or financial benefit by using their creation or invention. Intellectual Property is legally protected through copyrights, patents, trademarks etc.

**(A) Copyright -** Copyright grants legal rights to creators for their original works like writing, photograph, audio recordings, video, sculptures, architectural works, computer software, and other creative works like literary and artistic work. Copyrights are automatically granted to creators and authors.

**(B) Patent-** A patent is usually granted for inventions. Unlike copyright, the inventor needs to apply (file) for patenting the invention. When a patent is granted, the owner gets an exclusive right to prevent others from using, selling, or distributing the protected invention. A patent protects an invention for 20 years, after which it can be freely used.

**(C) Trademark-** Trademark includes any visual symbol, word, name, design, slogan, label, etc., that distinguishes the brand or commercial enterprise, from other brands or commercial enterprises. For example, no company other than Nike can use the Nike brand to sell shoes or clothes. It also prevents others from using a confusingly similar mark, including words or phrases. For example, confusing brands like "Nikke" cannot be used to sell shoes by some other company. However, it may be possible to apply for the Nike trademark for unrelated goods like notebooks.

**Violation of IPR**

Violation of intellectual property right may happen in one of the following ways:

**(A) Plagiarism-** Presenting someone else's idea or work as one's own idea or work is called plagiarism.

**(B) Copyright Infringement-** Copyright infringement is when we use other person's work without obtaining their permission to use.

**(C) Trademark Infringement-** Trademark Infringement means unauthorised use of other's trademark on products and services.

**Cyber Crime**

Cybercrime, or computer-oriented crime, is a crime that involves a computer and a network. The computer may have been used while committing a crime or it may be the target.

**Types of Cyber Crime:**

1. **Hacking:**
   Hacking is the act of unauthorised access to a computer, computer network or any digital system. Hackers usually have technical expertise of the hardware and software. They look for bugs to exploit and break into the system.

   *Ethical hackers/White hat hackers* use their skills to find out the vulnerability in a system and suggest improvements to the system where as *black hat hackers/crackers* use their skills for malicious intent to break into a system.
2. **Phishing:**
   Phishing is the fraudulent attempt to obtain sensitive information such as usernames, passwords and credit card details by disguising oneself as a trustworthy entity in an electronic communication. It is typically carried out by email spoofing (sending fake emails) or instant messaging. Users are lured by communications purporting to be from trusted parties such as social web sites, auction sites, banks, online payment processors or IT administrators. It often directs users to enter personal information at a fake website which matches the look and feel of the legitimate site.
3. **Illegal Downloads:**
   Illegal downloading refers to obtaining files for which you don't have the right to use or download from Internet. It is downloading a paid digital item without making any payment and using an illegal way to download it. Eg. Downloading a movie/music/software/ebook without making payment to the publisher.
4. **Child Pornography:**
   It is publishing and transmitting any obscene material about children in electronic form. It is punishable under Information Technology Act 2000 and Indian Penal Code 1860 leading to punishment of up to imprisonment of 7-10 years and fines up to 10 lakhs.

5. **Credit Card / Debit Card / Bank Fraud** – Here credit/debit/bank information is obtained by fraudulent means and used to carry out illegal financial transactions.
6. **Non-delivery of Goods/Service Fraud**- Here user is encouraged to buy and pay on a website but the goods are not delivered.
7. **Identity Theft**- In this kind of fraud another user's email, social media, e-commerce details are accessed by illegal means and personal data may be leaked online, or some unwanted messages are posted by an imposter etc.
8. **Auction Fraud**: Here people are encouraged to participate in an online auction for a non-existent or sub-standard product.
9. **Cyber Stalking**: Posting negative / fake / false /derogatory messages about somebody all over the internet / social media platforms in multiple places wherever that person has an account is called Cyber Stalking.

**Protection against Cyber Threats:**

- Use different unique IDs for each website
- Use strong passwords and do not repeat passwords
- Restrict giving out personal information on websites
- Don't post personally identifiable information such as mobile number, address, workplace etc. on social media posts.
- Using combination of methods such as secure/encrypted methods, Biometric protection methods (such as fingerprint, iris scan, voice recognition etc.) to protect digital data.
- Ensure safety of all user ids, passwords, financial information such as bank account number, credit/debit card, biometric data etc.

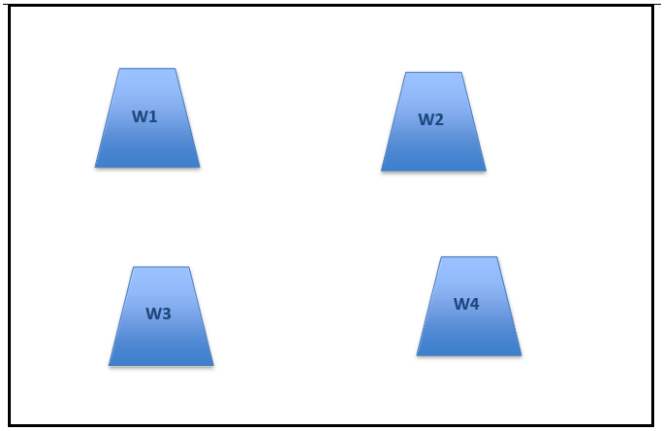
**Information Technology Act 2000 (ITA-2000 or IT act):**

It is the primary law in India dealing with cyber law, cybercrime and electronic commerce. Some of the major sections are:

| SECTION | OFFENCE | PENALTY |
|---|---|---|
| 66A | Sending of offensive messages | |
| 67A | Publishing images containing sexual acts | Imprisonment up to 7 years and/or fine up to Rs. 10,00,000. |
| 67B | Publishing child porn or predating children online | Imprisonment up to 5 years and/or fine up to Rs. 10,00,000 on 1$^{st}$ conviction. Imprisonment up to 7 years and/or fine up to Rs. 10,00,000 on 2$^{nd}$ conviction. |
| 67C | Failure to maintain records | Imprisonment up to 3 years and/or fine up to Rs. 2,00,000. |
| 68 | Failure/Refusal to comply with orders | Imprisonment up to 7 years and/or possible fine. |
| 69 | Failure/Refusal to decrypt data | Imprisonment up to 3 years and/or fine up to Rs. 1,00,000. |
| 70 | Securing access or attempting to secure access to protected system | Imprisonment up to 10 years and/or fine. |
| 71 | Misrepresentation | Imprisonment up to 3 years and/or fine up to Rs. 1,00,000. |

**NETWORKING QUESTION**

Q] A company Mega Enterprises has 4 wings of buildings as shown in the diagram:




Center to center distances between various Buildings:

W3 to W1 - 50m

W1 to W2 - 60m

W2 to W4 - 25m

W4 to W3 - 170m

W3 to W2 - 125m

W1 to w4 - 90m

Number of computers in each of the wing:

W1 - 30

W2 - 15

W3 - 150

W4 - 25

Computers in each wing are networked but wings are not networked. The company has now decided to connect the wings also.

i. Suggest a most suitable cable layout for the above connections.

ii. Suggest the most appropriate topology of the connection between the wings.

iii. The company wants internet accessibility in all the wings. Suggest a suitable technology.

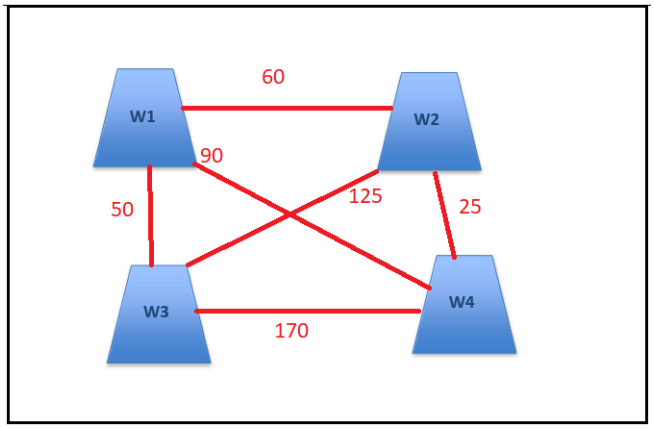
iv. Suggest the placement of the following devices with justification if the company wants minimized network traffic

a) Repeater b) Hub / switch

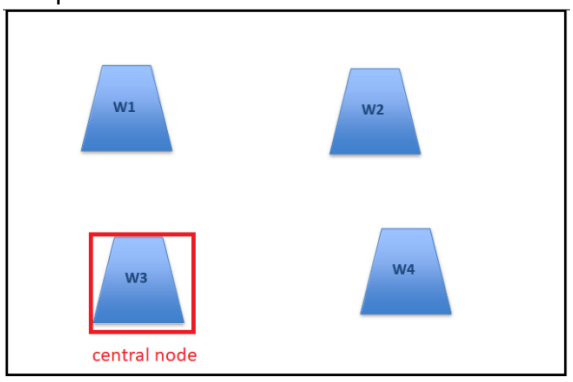
v. The company is planning to link its head office situated in New Delhi with the offices in hilly areas. Suggest a way to connect it economically.

**Steps For Cable Layout:**

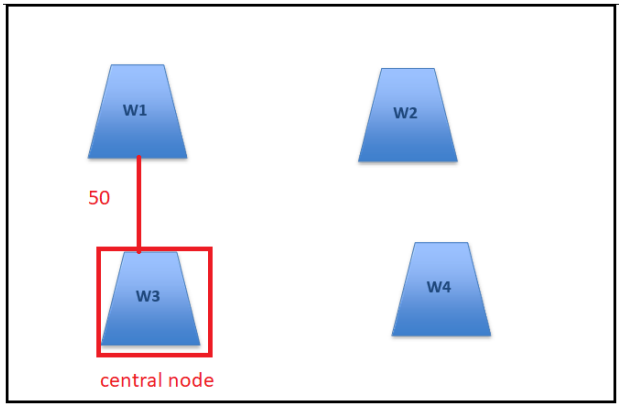
1. First draw the rough diagram showing the buildings and the distances



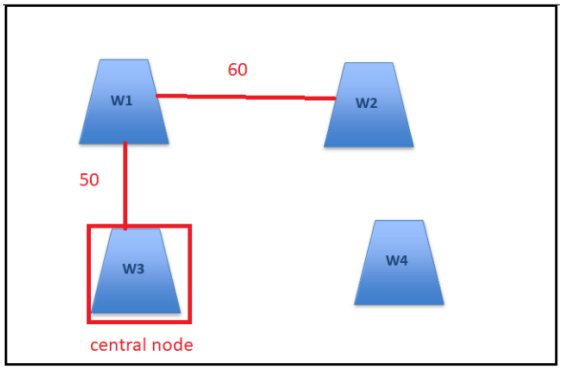

2. Then take the building with the highest number of computers as the central node. Here W3 has 150 computers. So it will house the server of the company and it becomes the central node.



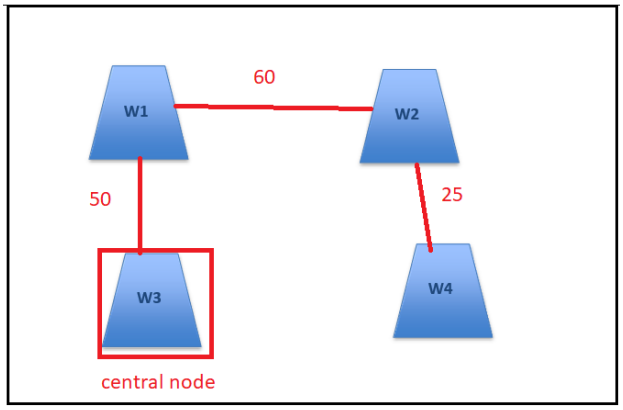

3. Then connect to W3 the building with the smallest distance from W3. Here W1 is 50 meters from W3. So first connect W1 to W3.




4. Then add the building with the shortest distance to W3 either directly or through W1. Here W2 can be directly connected to W3 by
   a. a 125 meter wire OR
   b. it can be connected to W1 by a 60 meter wire
   Since in the second case (60 +50=110) is less than 125m the second option is chosen

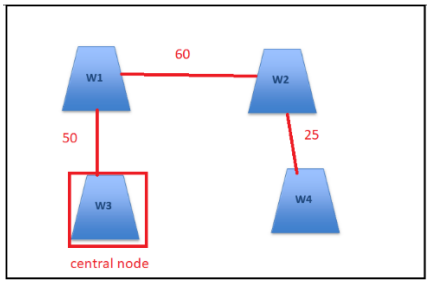


5. Then add the final building W4 so that the distance of W4 from W3 is the least either directly or indirectly through any other building.




Answers]

(I) Cable Layout is as shown below:




(ii) Hybrid topology

(iii) A fibre optic connection can be used along with a gateway to connect to the central building W3 and the internet connection can be shared with all the computers of the company.

(iv) a) A repeater is used whenever the cable length of any section is more than 100 meters. In our design the cable lengths are less than 100 meters, so a repeater is not needed.

(b) A hub/switch will need to be placed within each building. Also compared to a hub a switch is more preferable to reduce network traffic.

(v) If mobile network of high speed 3G/4G is available in the hilly area, then it can be used to connect to the office in Delhi. If mobile network is not available, then VSAT terminal can be used to connect to a satellite network and connect it to Delhi office.

## 9. STRUCTURED QUERY LANGUAGE (SQL)

**Database**: A database is an organised collection of interrelated data.

**DBMS**: The software used to manage databases is called Database Management System (DBMS).

**Relational Database:** A database in which the data is stored in the form of relations or tables is called a Relational Database. A relational database is a collection of one or more tables.

**RDBMS:** A DBMS used to manage Relational Databases is called Relational Database Management System (RDBMS).

**Benefits of using DBMS:**

1. Redundancy can be controlled.
2. Inconsistency can be avoided.
3. Data can be shared.
4. Security restrictions can be applied.

**Relation/Table:** A table/relation is a two-dimensional representation of data arranged in rows and columns. Columns of a table also known as **fields** or **attributes**. Rows of a table are also known as **records** or **tuples**.

**Domain:** A domain refers to the set of permissible values that an attribute (or column) in a table can hold. It defines the data type, constraints, and rules for the values stored in a specific column, ensuring data integrity and consistency. For example, consider the age column in a table employee, the domain for the age column will be the positive integers.

**Primary Key:** The group of one or more columns used to uniquely identify each row of a relation is called its Primary key. A table can have only one Primary key.

**Candidate key:** A column or a group of columns which can be used as a primary key of a relation is called a candidate key. There can be many candidate keys for a table.

**Alternate key:** A candidate key of a table which is not made its primary key is called its Alternate key.

**Degree of a Table:** The number of columns in the table is called the degree of a table.

**Cardinality of a Table**: The number of rows in a table is called its cardinality.

**SQL(Structured Query Language):** SQL is the language used to manipulate and manage databases and tables within an RDBMS.
The SQL Language is comprised of the following kinds of statements:

(a) **DDL (Data Definition Language):** It is a category of SQL commands that are used to create, destroy, or restructure databases and tables. Eg. CREATE, DROP, ALTER.
(b) **DML (Data Manipulation Language):** It is a category of SQL commands that are used to manipulate data within tables. Eg. INSERT, UPDATE, DELETE.
(c) **DQL (Data Query Language):** It is a category of SQL commands that are used only to view the data / information present in a table / database. Eg. SELECT command comes under DQL
(d) **DCL (Data Control Language):** It is a category of SQL commands that control access to the databases and tables. Eg. GRANT, REVOKE.

**MySQL**

MySQL is an open source relational database management system. MySQL was created by the company MySQL AB, founded by David Axmark, Allan Larsson and Michael Widenius. MySQL was bought by Sun Microsystems which is now owned by Oracle Corporation.

**Basic MySQL Commands:**

1. **show databases;**
   It shows all the databases present in mysql

2. **create database** *databasename*;
   It creates a new database named *databasename* in mysql.

3. **use** *databasename*;
   The current database on which we are working is changed to *databasename.*

4. **show tables;**
   It shows the list of all tables present in the current database.

5. **drop database** *databasename;*
   This command removes the *databasename* from mysql.

**SQL Data Types:**

| S.No. | Data Type | Explanation | Example |
|---|---|---|---|
| 1. | CHAR | Stores character data. All data have exactly the same size specified in the definition even if user is entering less number of characters in input. | NAME CHAR(20) |
| 2. | VARCHAR | Stores variable length character. It stores only the number of characters that the user has specified. | ADDRESS VARCHAR(20) |
| 3. | FLOAT, REAL, DOUBLE | Stores number having decimal value. Precision after the decimal point is not guaranteed. | Salesamt FLOAT,<br>Cost REAL,<br>Totalsales DOUBLE, |
| 4. | DEC, DECIMAL, NUMERIC | Stores numbers having decimal value. Precision of the numbers after decimal point is guaranteed | Int_rate DEC(5,2),<br>Int_rate DECIMAL(5,2),<br>Int_rate NUMERIC(5,2),<br>{ 5 is the maximum digits of the number and 2 is the number of digits after decimal point DEC(5,2) will store maximum number as 999.99 ie max 5 digits out of which 2 digits can be after decimal point} |
| 5. | INT | Stores integer values | Age int, |
| 6. | DATE | Stores the date. Standard dates are expressed in “yyyy-mm-dd” format in SQL. | Dateofbirth DATE, |
| 7. | TIME | Stores time expressed in “HH:MM:SS” | Starttime TIME, |

**SQL COMMANDS:**

1. **CREATE TABLE:**

   It is used to create a new table in RDBMS.

   Syntax:

```
CREATE TABLE   <tablename>
(  <column_name>          <data_type>                  <column_constraint>,
   <column_name>          <data_type>          <column_constraint>,
  ....
  ....
  <column_name>          <data_type>          <column_constraint>,
<table_constraint>   ( <column_name 1>, <column_name 2>,  ....  ,  <column_name n >          )
);
```

   Eg.

```
CREATE TABLE student
( rollno int,
  name char(20),
  address varchar(30)  );
```

   **CONSTRAINT:** A constraint is a condition that is applied to a column or set of columns that help to ensure that valid data only is entered in that column or set of columns.

   Constraints can be of the following types:

   1. **PRIMARY KEY:** It sets a column or group of columns as the primary key of the table. NULLs and duplicate values are not accepted in the column(s).
   2. **UNIQUE:** It ensures that duplicate values are not entered in the column or group of columns. There can be many unique constraints on a table.
   3. **NULL:** When NULL is applied to a column, the column can contain NULL/empty values.
   4. **NOT NULL:** It is also applied to a single column. The column cannot contain NULL values.
   5. **DEFAULT:** It is used to specify a default value if the user has not entered the value for a column.

Examples of use of constraints:

1. 
```
create  table employee
( ecode          int              primary key,
  ename          char(20)         not null,
  aadharno       varchar(30)      unique,
  grade          char(2)          default  "E1",
  grosspay       decimal,
  netpay         decimal
);
```

2. 
```
create table student
(
  regno          int              unique,
  class          char(3)          not null,
  section        char             not null          default  'A',
   rollno        int              not null,
   name          varchar(30)       not null,
   percent       float            ,
   hometown      varchar(30)      null,
   primary key  (class , section, rollno)
);
```

3. **DESCRIBE *tablename /* DESC *tablename;***
   The describe / desc command can be used to display the details of all the columns present in a table. This is particularly useful if we are working on a database which someone else has created or when working on a RDBMS and we wish to quickly see the details of the columns present in a table.
   Eg. describe student;
   *o/p:*

| Field | Type | Null | Key | Default | Extra |
|---|---|---|---|---|---|
| regno | int(11) | YES | UNI | NULL | |
| class | char(3) | NO | PRI | NULL | |
| section | char(1) | NO | PRI | A | |
| rollno | int(11) | NO | PRI | NULL | |
| name | varchar(30) | NO | | NULL | |
| percent | int(11) | YES | | NULL | |
| hometown | varchar(30) | YES | | NULL | |

4. **INSERT statement :** Insert statement is used for adding rows to a table. The following rules must be followed while writing insert statement:
   (a) Any data that is a number(int, float, dec,numeric) is entered without quotes and any data that is not a number (char, varchar, date, time) must be enclosed either in single quotes or double quotes.
   (b) If we don't write the column names after the table name then the list of values specified after the values must match exactly the count of columns of the table.
   (c) If we don't want to enter any value for a column write the word NULL without quotes while specifying the values.
   (d) If any column is having a NULL constraint or a DEFAULT constraint, then that column can be omitted from the list of column names that is written after the table name.
   (e) If any values is tried to enter for a column that fails the constraint for that column then an error is given and the row is not entered in the table.

   Eg.
   1. #column names not specified after table name
      insert into student values
      (100, 'I', 'A', 1, 'aa1', 60, null );

   2. #column names specified after table name
      insert into student(regno, class, rollno, name, percent) values
      (101, 'I', 2, 'aaa2', 75 );

   3. #entering a row that fails the constraint will give an error, here name not null
      insert into student(regno, class, section, rollno) values
      (999, 'X', 'A', 1);

Assume that the following Student table is created in the database with the following values:

**Student**

| regno | class | section | rollno | name | percent | hometown |
|---|---|---|---|---|---|---|
| 100 | I | A | 1 | aaa1 | 60 | mumbai |
| 101 | I | A | 2 | aaa2 | 75 | delhi |
| 102 | I | B | 1 | bbb1 | 90 | chennai |
| 104 | I | B | 2 | bbb2 | 95 | kolkatta |
| 105 | I | B | 3 | bbb3 | 97 | kolkatta |
| 150 | II | A | 1 | ccc1 | 55 | mumbai |
| 151 | II | A | 2 | ccc2 | 60 | mumbai |
| 152 | II | A | 3 | ccc3 | 62 | chennai |
| 154 | II | B | 1 | ddd1 | 75 | chennai |
| 155 | II | B | 2 | ddd2 | 80 | chandigarh |
| 801 | IX | A | 1 | abc | 75 | madurai |
| 802 | IX | A | 2 | amp | 82 | nasik |
| 803 | IX | A | 3 | bac | 67 | nasik |
| 807 | IX | B | 1 | cba | 77 | nagpur |
| 808 | IX | B | 2 | aab | 82 | delhi |
| 809 | IX | B | 3 | ccm | 99 | dehradun |
| 850 | X | A | 1 | mac | 82 | dubai |
| 852 | X | A | 2 | amc | 79 | nagpur |
| 853 | X | B | 1 | baa | 83 | kolkatta |
| 854 | X | B | 2 | abb | 84 | chennai |

**4. SELECT Command:**

The select command is used to retrieve rows from a table.

**Syntax:**

**Select [* or <column name1>, <column name2> … <column name n> ] from <table name>**

Eg.

**(a) To display all the rows of the table student**

*Select * from student;*

[ * means all columns of the table]

o/p:

| regno | class | section | rollno | name | percent | hometown |
|---|---|---|---|---|---|---|
| 100 | I | A | 1 | aaa1 | 60 | mumbai |
| 101 | I | A | 2 | aaa2 | 75 | delhi |
| ……. | | | | | | |
| 854 | X | B | 2 | abb | 84 | chennai |

**(b) To display the columns class, section, rollno and name of students of all the classes**

Select class, section, rollno, name from student;

o/p:

| class | section | rollno | name |
|---|---|---|---|
| **I** | **A** | **1** | **aaa1** |
| **I** | **A** | **2** | **aaa2** |
| ……. | | | |
| X | B | 2 | abb |

**(c) To display the columns class, section, rollno and name and rename (use column alias) the 'name' column as 'Full Name' of students of all the classes**

Select class, section, rollno, name **as 'Full Name'** from student;

o/p:

| class | section | rollno | Full Name |
|---|---|---|---|
| **I** | **A** | **1** | **aaa1** |
| **I** | **A** | **2** | **aaa2** |
| ……. | | | |
| X | B | 2 | abb |

**(d) To display name and percent * 2 and use column alias as 'Twice percent'**

Select name, **percent * 2** ***as 'Twice Percent'*** from student;

o/p:

| name | Twice Percent |
|---|---|
| aaa1 | 120 |
| aaa2 | 150 |
| … | |
| … | |
| baa | 166 |
| abb | 168 |

**(e) To display the hometowns of the students**

select hometown from student;

o/p:

| hometown |
|---|
| mumbai |
| delhi |
| chennai |
| kolkatta |
| kolkatta |
| mumbai |
| mumbai |
| chennai |
| chennai |
| chandigarh |
| madurai |
| nasik |
| nasik |
| nagpur |
| delhi |
| dehradun |
| dubai |
| nagpur |
| kolkatta |
| chennai |

The rows are repeated with values. In order to omit the repeated values, we specify the word distinct keyword after the select keyword.

**(f) To display the different class and sections present in the table.**

select distinct class, section from student;

o/p:

| class | section |
|---|---|
| I | A |
| I | B |
| II | A |
| II | B |
| IX | A |
| IX | B |
| X | A |
| X | B |

The duplicate / repeating class and sections are removed using the distinct keyword.

**5. Where clause:**

The where clause is used to retrieve specified / some rows of the table based on the conditions specified. If any data values are specified in where clause that is a number ( int, float, decimal), then such numbers are not enclosed in quotes. Any data value that is not a number (char, varchar, date) is enclosed in quotes (either single or double quotes).

The where clause can accept the following operators:

(i) Relational Operators : = (equals to ) , <> (not equal to) , != (not equals to ), > , <, >= , <=.
(ii) Logical Operators: AND, OR, NOT
(iii) Condition based on range: BETWEEN , NOT BETWEEN
(iv) Condition based on list: IN, NOT IN
(v) Condition based on pattern match: LIKE, NOT LIKE
(vi) Searching for presence / absence of NULLs: IS NULL , IS NOT NULL

**Syntax:**
**select <select clause >**
**from <table name 1> [, <table name 2>]**
**where <condition>**

**(a) To see the data of class 'IX' students only:**

select * from student where class = 'IX';
o/p:

| regno | class | section | rollno | name | percent | hometown |
|---|---|---|---|---|---|---|
| 801 | IX | A | 1 | abc | 75 | madurai |
| 802 | IX | A | 2 | amp | 82 | nasik |
| 803 | IX | A | 3 | bac | 67 | nasik |
| 807 | IX | B | 1 | cba | 77 | nagpur |
| 808 | IX | B | 2 | aab | 82 | delhi |
| 809 | IX | B | 3 | ccm | 99 | dehradun |

**(b) To display the names of class X A students:**

select name from student where class = 'X' and section = 'A';
o/p:

| name |
|---|
| mac |
| amc |

AND – both the conditions must be true for the row to come in output
OR – any one of the condition must be true for the row to come in output
NOT – Reverses the condition. True becomes false and False becomes True.

**(c) To display the student names from classes IX as well as student names from class X.**

select name from student where class = 'IX' or class = 'X';
o/p:

| name |
|---|
| abc |
| amp |
| bac |
| cba |
| aab |
| ccm |
| mac |
| amc |
| baa |
| abb |

**(d) To display the names of students who are not from class I:**

select name from student where NOT (class = 'I');

o/p:

| name |
|---|
| ccc1 |
| ccc2 |
| ccc3 |
| ddd1 |
| ddd2 |
| abc |
| amp |
| bac |
| cba |
| aab |
| ccm |
| mac |
| amc |
| baa |
| abb |

**(e) To display the names of students of class IX who have percentages in the range 80 to 90 (including both 80 and 90)**

select name from student where class = 'IX' and ( percent >=80 and percent <=90);

o/p:

| name |
|---|
| amp |
| aab |

**BETWEEN operator :** It can be used to find the range of values that lie between two values (including both the upper value and the lower value). For eg. The above query can be written as:

select name from student where class = 'IX' and ( percent between 80 and 90);

(f) **To display the names and percent of students of class 'IX' who are not scoring between the range 70 to 100.**

select name from student where class = 'IX' and ( percent **not between** 70 and 100);

o/p:

| name | percent |
|---|---|
| bac | 67 |

**NOT BETWEEN:** It can be used to find the column values that lie outside a range of values.

(g) **IN operator:** It can be used to check if column values match any one of the values that have been specified in a list.

**To find the names of students of class IX having hometown as either nasik, Nagpur or delhi.**
select name from student where class = 'IX' and hometown in ('nasik', 'nagpur', 'delhi');

| name |
|---|
| amp |
| bac |
| cba |
| aab |

The same query can be written using OR operator as :
select name from student where class = 'IX' and
(hometown = 'nasik' or hometown = 'nagpur'or hometown = 'delhi');

(h) **NOT IN operator:** The NOT in operator finds rows that do not match in the list.

For eg. **To find the names of students of class IX who are not having hometowns as either 'Nasik' or 'Nagpur'.**
select name from student where class = 'IX' and hometown not in ('nasik', 'nagpur');
o/p:

| **name** |
|---|
| abc |
| aab |
| ccm |

(i) **LIKE Operator:** The LIKE operator is used for pattern matching for comparisons on character strings using patterns. Patterns are described using two special wildcard characters:
1. % (percent) - % matches zero or more characters
2. _ (underscore) - _ matches exactly one character

Eg.

1. **To find the names of students having exactly three characters:**
select name from student where name like '_ _ _';
o/p:

| **name** |
|---|
| abc |
| amp |
| bac |
| cba |
| aab |
| ccm |
| mac |
| amc |
| baa |
| abb |

2. **To find the names of students of class X whose names are starting with a:**
select name from student where class = 'X' and name like 'a%';
o/p:

| name |
|---|
| amc |
| abb |

3. **To find the names of students whose names are ending with b**
   select name from student where name like '%b';
   o/p:

| name |
|---|
| aab |
| abb |

4. **To find the names of students of class 'IX', having the letter 'b' as the second character in their names:**
   select name from student where class = 'IX' and name like '_b%';
   o/p:

| name |
|---|
| abc |
| cba |

5. **To display the names of students having two consecutive a's in their names:**
   select name from student where name like '%aa%';
   o/p:

| name |
|---|
| aaa1 |
| aaa2 |
| aab |
| baa |

(j) **NOT LIKE operator:** The NOT LIKE operator is used to match rows that do not match the specified pattern of characters.
For eg.

1. **To find the names of students of class IX who do not have the letter 'a' in their names.**
   select name from student where class = 'IX' and name not like '%a%';
   o/p:

| name |
|---|
| ccm |

2. **To find the names of students of class I whose names do not begin with the letter 'a':**
   select name from student where class = 'I' and name not like 'a%';
   o/p:

| name |
|---|
| bbb1 |
| bbb2 |
| bbb3 |

**(k) Searching for NULL – IS NULL:**
To find the class, section, rollno, names of the students who have left their hometown as blank or not mentioned their hometown:
select class,section, rollno, name from student **where hometown is null;**

**(l) Searching for absence of NULL – IS NOT NULL:**
To find the class, section, rollno, names of the students who have filled in their hometown:
select class,section, rollno, name from student **where hometown is NOT null;**

6. **ORDER BY clause:** The order by clause allows sorting of the query results (ASC –ascending order or DESC – descending order) based on values contained in one or more columns. The data in the table is not sorted only the displayed results are sorted. If sort order (either ASC or DESC) is not specified in front of column name, then it is assumed to ASC.

**Syntax:**
**select** <columns>
**from** <tables>
**where** <condition>
**order by** <column name 1> [ASC / DESC] [, <column name2> [ASC / DESC] , …. <column name n> [ASC / DESC]]

Eg.

**(a) To display the names and percentages of the students of class 'X' in the decreasing order of their percentages:**

```
select name, percent from student where class = 'X'
order by percent desc;
```

o/p:

| name | percent |
|---|---|
| abb | 84 |
| baa | 83 |
| mac | 82 |
| amc | 79 |

(b) To display the class, section , rollno ,names and percentages of the students in order of the classes and within each class section wise and in the descending order of their percentages.

```
select class, section,rollno, name, percent
from student
order by class, section, percent desc;
```

o/p:

| class | section | rollno | name | percent |
|---|---|---|---|---|
| I | A | 2 | aaa2 | 75 |
| I | A | 1 | aaa1 | 60 |
| I | B | 3 | bbb3 | 97 |
| I | B | 2 | bbb2 | 95 |
| I | B | 1 | bbb1 | 90 |
| II | A | 3 | ccc3 | 62 |
| II | A | 2 | ccc2 | 60 |
| II | A | 1 | ccc1 | 55 |
| II | B | 2 | ddd2 | 80 |
| II | B | 1 | ddd1 | 75 |
| IX | A | 2 | amp | 82 |
| IX | A | 1 | abc | 75 |
| IX | A | 3 | bac | 67 |
| IX | B | 3 | ccm | 99 |
| IX | B | 2 | aab | 82 |
| IX | B | 1 | cba | 77 |
| X | A | 1 | mac | 82 |
| X | A | 2 | amc | 79 |
| X | B | 2 | abb | 84 |
| X | B | 1 | baa | 83 |

(c) To display the class, section, name of the students in class wise order based on descending order of their percentages.
select class, section, name, percent from student
order by class asc, percent desc;

o/p:

| class | section | name | percent |
|---|---|---|---|
| I | B | bbb3 | 97 |
| I | B | bbb2 | 95 |
| I | B | bbb1 | 90 |
| I | A | aaa2 | 75 |
| I | A | aaa1 | 60 |
| II | B | ddd2 | 80 |
| II | B | ddd1 | 75 |
| II | A | ccc3 | 62 |
| II | A | ccc2 | 60 |
| II | A | ccc1 | 55 |
| IX | B | ccm | 99 |
| IX | A | amp | 82 |
| IX | B | aab | 82 |
| IX | B | cba | 77 |
| IX | A | abc | 75 |
| IX | A | bac | 67 |
| X | B | abb | 84 |
| X | B | baa | 83 |
| X | A | mac | 82 |
| X | A | amc | 79 |

7. **AGGREGATE FUNCTIONS:** Aggregate functions work on multiple rows and give one value as the answer. The aggregate functions are MAX(), MIN(), AVG(), SUM(), COUNT()

(a) **MAX():** It is used to find the highest value of any column or expression based on a column.

Eg. 1. **To display the highest percent among class 'X' students.**
select max(percent) from student where class = 'X';
o/p:

| max(percent) |
|---|
| 84 |

Eg. 2. **To find the highest selling price of an item**
select max(cost + cost * margin / 100) from item;
// max can work with expressions also

(b) **MIN():** It is used to find the lowest value of any column or an expression based on a column.
Eg. 1. **To display the lowest percent among class 'IX' students.**
select min(percent) from student where class = 'IX';

| min(percent) |
|---|
| 67 |

(c) **AVG():** It is used to find the average value of any column or an expression based on a column. AVG() does not count columns having NULL values while computing the average

Eg. For the following sales table:

**sales**

| orderno | sales_amt |
|---|---|
| 1 | 100 |
| 2 | NULL |
| 3 | 150 |

select avg(sales_amt) from sales;

o/p:

| avg(sales_amt) |
|---|
| 125 |

**The NULL sales_amt is not counted towards the calculation of average. The avg is calculated as: (100 + 150) / 2 = 125**

(d) **SUM():** It is used to find the total value of any column or an expression based on a column.

For eg. In **the previous sales table, for calculating the total sales the query is:**

select sum(sales_amt) from sales;

o/p:

| sum(sales_amt) |
|---|
| 250 |

(e) **COUNT():** It is used to count the number of values in a column. COUNT() takes one argument which can be any column name, an expression based on a column, or an asterisk(*).

**(i) If the argument is an asterisk(*) then it counts the number of rows.**
**(ii) If the argument is based on a column or expression, then it counts the number of non-NULL values in that particular column or expression.**
**(iii) If the argument is based on a column or expression and we want to count the repeating values only once then the keyword DISTINCT should be used inside the count function and before the column name.**

Eg. For the following sales table

**sales**

| orderno | sales_amt |
|---|---|
| 1 | 100 |
| 2 | NULL |
| 3 | 150 |

Eg. 1. **To find the total number of rows of the table, the query is:**
select count(*) from sales;
o/p:

| count(*) |
|---|
| 3 |

2. **To find the number of valid sales , the query is:**
select count(sales_amt) from sales;
o/p:

| count(sales_amt) |
|---|
| 2 |

In this example **the NULL sales_amt is not counted by the count function.**

3. For the following table orders to count how many different cities have ordered, the query is:

**orders**

| o_no | city |
|---|---|
| 1 | delhi |
| 2 | mumbai |
| 3 | delhi |

**select COUNT( DISTINCT city) from orders;**
o/p:

| COUNT(DISTINCT city) |
|---|
| 2 |

The city 'delhi' is repeated twice in the city column and is counted only once and the total count of distinct/different cities is only 2.

8. **GROUP BY:** It is used to group all the records of a table into different groups based on one or more columns and then find the sum, min, max, avg or count of each of the group individually.

   Whenever GROUP BY is used the fields of coming after the SELECT clause can only be the columns appearing after the GROUP BY or an aggregate function (ie, either SUM, MIN, MAX, AVG , COUNT)

   Eg. 1. **To find the highest percent of each class, the query will be:**
   *select class, max(percent)*
   *from student*
   *group by class;*
   o/p:

| class | max(percent) |
|---|---|
| I | 97 |
| II | 80 |
| IX | 99 |
| X | 84 |

   [ **Group is form on the basis of column coming after the group by statement - here column class**

   All the class I students form the first group,
   class II students the second group,
   class IX students the third group,
   class X students the fourth group.

   The aggregate function max works on each group separately and finds the max(percent) within
   - subgroup of Class I
   - subgroup of Class II
   - subgroup of Class IX
   - subgroup of Class X
   ]

   2. **Groups can be formed on the basis of more than one column also.** For ex. if we want the highest percent as well as the lowest percentage within each class and within each class section wise, then the query will be:
   *select class, section, max(percent), min(percent)*
   *from student*
   *group by class, section;*
   o/p:

| class | section | max(percent) | min(percent) |
|---|---|---|---|
| I | A | 75 | 60 |
| I | B | 97 | 90 |
| II | A | 62 | 55 |
| II | B | 80 | 75 |
| IX | A | 82 | 67 |
| IX | B | 99 | 77 |
| X | A | 82 | 79 |
| X | B | 84 | 83 |

9. **HAVING clause:** It is used to place some condition on individual groups.

HAVING is used always with a GROUP BY clause. HAVING always has an AGGREGATE function (i.e. MIN, MAX, AVG, COUNT, SUM ) used with it. Whenever HAVING is used there must be a GROUP BY clause and the select statement must have a column named in the GROUP BY clause or an AGGREGATE function used in the select clause.

Eg. 1. **Find the classes and the number of students in that class, which are having the number of students in that class more than 4.**

```
select class, count(*)
from student
group by class
having count(*) > 4;
```

o/p:

| class | count(*) |
|---|---|
| I | 5 |
| II | 5 |
| IX | 6 |

2. **For the table shown below find the cities, the total order amount, the number of orders and the average order amount where the total sales for that city is more than 10000**

sales

| orderno | city | order_amt |
|---|---|---|
| 1 | mumbai | 5000 |
| 2 | delhi | 2000 |
| 3 | chennai | 12000 |
| 4 | mumbai | 7000 |
| 5 | delhi | 1000 |
| 6 | bangalore | 7000 |

```
select city, sum(order_amt), count(*), avg(order_amt)
from sales
group by city
having sum(order_amt) > 10000;
```

o/p:

| city | sum(order_amt) | count(*) | avg(order_amt) |
|---|---|---|---|
| mumbai | 12000 | 2 | 6000 |
| chennai | 12000 | 1 | 12000 |

10. **UPDATE:** It is used to change the values of a row in a table. If the UPDATE statement is used without a WHERE clause then all the rows of the table are updated. If the WHERE clause is specified then only the rows that match the where clause are updated.
For eg. consider the account table shown below:

**account**

| accno | custname | balance |
|---|---|---|
| 101 | abc | 500 |
| 102 | def | 1000 |
| 103 | ghi | 2000 |

(a) To increase the balance amount by 500 for all customers the query will be:
**Update account**
**set balance = balance + 200;**

After executing this command the balance amount for all rows is increased by 500.

(b) To change the custname of accno 103 to 'xyz' as well as change the balance amount to 10000, the update command is:
**Update account**
**set custname = 'xyz', balance = 10000**
**where accno = 103;**

If multiple columns are to be updated then the set clause can have multiple column names separated by commas.

(c) To make/set to null a column value, the null keyword is used without the quotes.
Update account
**set balance = null**
where accno = 102;

This command sets the balance to NULL for account number 102.

11. **DELETE:** It is used to delete entire rows from the table. The WHERE clause is used to select the rows that are to be deleted. If the where clause is omitted then all the rows from the table are deleted.
Eg. In the above account table to delete the entry for account number 103, the command is:
**delete from account**
**where accno = 103;**

12. **DROP TABLE:** It is used to remove a table from the database.
Eg. To remove table employee from the database the command is:
**drop table employee;**

Difference between delete command and drop command is that delete is used to delete rows from a table whereas drop command, drops the table entirely from the database.

13. **ALTER TABLE:** It can be used to add, remove and modify columns of a table. It can also be used to add and remove constraints.
Eg. Consider a table employee which was created using the following create table statement:

```
create table employee
(
empno int  primary key,
name varchar(20),
dept varchar(10),
address varchar(100),
 hometown varchar(50)
);
```

(a) To remove the primary key constraint from table employee
**alter table employee drop primary key;**

(b) To add a new primary key to the table employee:
**alter table employee add primary key (name, dept);**

(c) To remove a single column hometown
**alter table employee drop hometown;**

(d) To remove two columns hometown as well as address:
**alter table employee drop hometown, drop address;**

(e) To modify dept column so that it does not accept null values
**alter table employee modify name varchar(20) not null;**

(f) To modify two columns - dept column dept to set it to varchar(50) and set name to not null:
**alter table employee modify dept varchar(50), modify name varchar(20) not null;**

(g) To add a new column salary float to the employee table:
**alter table employee add salary float;**

(h) To add two columns salary float and number of days worked int to the table employee:
**alter table employee add salary float, add days_worked int;**

14. **CARTESIAN PRODUCT or CROSS JOIN OF TABLES:**
Cartesian product of two tables is a mathematical concept in which each row of one table is paired with each row of the other table.
Consider the two tables as below:

**TableA**

| C1 | C2 | C3 |
|---|---|---|
| a | b | c |
| d | e | f |

**TableB**

| C4 | C5 |
|---|---|
| G | H |
| I | J |
| K | L |

Mathematically Cartesian product is denoted by :

**TableA x TableB**

| C1 | C2 | C3 | C4 | C5 |
|---|---|---|---|---|
| a | b | c | G | H |
| a | b | c | I | J |
| a | b | c | K | L |
| d | e | f | G | H |
| d | e | f | I | J |
| d | e | f | K | L |

The **degree (number of columns)** of the cross product is the sum of the number of columns of both the tables.
The **cardinality (number of rows)** of a cross product is the multiplication of the number of rows of both the tables.

In SQL the cross product of two or more tables is always formed when more than one table is specified after the FROM keyword. The tables must be separated by commas.

For eg. **cross product of TableA and TableB** is formed by the SQL:
**select * from TableA, TableB;**

15. **EQUIJOIN of TABLES:** It is used to obtain meaningful data from the Cartesian product / Cross Join of two or more tables by specifying the condition of equality between the two tables.

Consider the following two tables:

customers

| custno | custname |
|---|---|
| 101 | abc |
| 102 | def |
| 103 | ghi |

orders

| orderno | custno | order_amt |
|---|---|---|
| 1 | 101 | 5000 |
| 2 | 103 | 2000 |

The following query creates the cross join of customers and orders tables.
**select * from customers, orders;**

customers x orders

| custno | custname | orderno | custno | order_amt |
|---|---|---|---|---|
| 101 | abc | 1 | 101 | 5000 |
| 101 | abc | 2 | 103 | 2000 |
| 102 | def | 1 | 101 | 5000 |
| 102 | def | 2 | 103 | 2000 |
| 103 | ghi | 1 | 101 | 5000 |
| 103 | ghi | 2 | 103 | 2000 |

Even though the cross product gives all the combinations of rows present in both the tables, the rows that contain meaningful data are those rows for which the custno in both the tables match.

Equijoin is used to match this common column by specifying a where clause for the above query and using the condition of equality of the two common columns as :

*select * from customers, orders* ***where customers.custno = orders.custno;***

o/p:

| custno | custname | orderno | custno | order_amt |
|---|---|---|---|---|
| 101 | abc | 1 | 101 | 5000 |
| 103 | ghi | 2 | 103 | 2000 |

**Whenever a table join is used**

**(i) Equijoin must be specified in the where clause and any additional condition must come after the AND specified after the equijoin condition.**

**(ii) Any column can be specified by TABLENAME.COLUMNNAME i.e. a column name can be preceded by the Table name and a dot(.)**

**(iii) If the common column between the two tables is used in where or select clause then it must be preceded by the Table name and a dot.**

**(iv) If a Table Alias is used in the FROM clause then it renames the Table to the Alias. Then only the Alias name and a dot must be used, the original table name and a dot must NOT be used anywhere in the SQL query.**

**Join operator**

Another way of writing the equijoin is by using the join operator
The command for equijoin –
*select * from customers, orders* ***where customers.custno = orders.custno;***

can be written as-
*select * from customers* ***join*** *orders* ***on*** *customers.custno = orders.custno;*

in both the cases the output is same and the common column *custno* is repeated twice from both the tables as shown below-

***o/p:***

| custno | custname | orderno | custno | order_amt |
|---|---|---|---|---|
| 101 | abc | 1 | 101 | 5000 |
| 103 | ghi | 2 | 103 | 2000 |

**Natural Join**

The natural join operator automatically searches for the common column names in both the tables and performs an exact match on the values present in the common columns. In the natural join the where clause for equality or the on condition is not needed to be explicitly written as it is done by the *RDBMS* system implicitly.

The above query using natural join is-

*select * from customers natural join orders;*

*o/p:*

| custno | custname | orderno | order_amt |
|---|---|---|---|
| 101 | abc | 1 | 5000 |
| 103 | ghi | 2 | 2000 |

In a natural join the common column is present only once in the output and it need not be prefixed by the table name as in equijoin.

**Join Examples**

Consider the following two tables:

customers

| custno | custname | custcity |
|---|---|---|
| 101 | abc | mumbai |
| 102 | def | delhi |
| 103 | ghi | mumbai |
| 104 | jkl | bangalore |

orders

| orderno | custno | order_amt |
|---|---|---|
| 1 | 101 | 6000 |
| 2 | 103 | 2000 |
| 3 | 102 | 7000 |
| 4 | 104 | 8000 |

1. To display the custno, custname and their respective order amounts, the query can be written as:

```
select custno, custname, order_amt
from customers natural join orders;

select customers.custno, custname, order_amt
from customers, orders
where customers.custno = orders.custno;

select customers.custno, custname, order_amt
from customers join  orders on customers.custno = orders.custno;
```

o/p:

| custno | custname | order_amt |
|---|---|---|
| 101 | abc | 6000 |
| 102 | def | 7000 |
| 103 | ghi | 2000 |
| 104 | jkl | 8000 |

2. To display the customer names and order amounts of customers living in delhi:

```
select custname, order_amt
from customers natural join orders
where custcity = 'delhi';

select custname, order_amt
from customers  join orders on customers.custno = orders.custno
where custcity = 'delhi';

select custname, order_amt
from customers  as A, orders  as B
where  A.custno  =  B.custno and custcity = 'delhi';
```

o/p:

| custname | order_amt |
|---|---|
| def | 7000 |

**Table alias A** is used for customer and **Table alias B** is used for orders table. **Only alias names A and B must be used wherever table names are to be used.**

3. To display the customer names and order amounts of those customers from Mumbai who have order amount greater than 5000.

```
select custname, order_amt
from customers natural join orders
where custcity = 'mumbai' and order_amt > 5000;

select custname, order_amt
from customers A join orders B on A.custno = B.custno
where custcity = 'mumbai' and order_amt > 5000;

select custname, order_amt
from customers A, orders B
where A.custno = B.custno and custcity = 'mumbai' and order_amt > 5000;
```

o/p:

| custname | order_amt |
|---|---|
| abc | 6000 |

**16. FOREIGN KEY:**

Consider the following two tables - customer and ord tables storing the customer information and the orders information in them.

**customer table**

| custno | custname |
|---|---|
| 101 | abc |
| 102 | def |
| 103 | ghi |

**ord table**

| orderno | custno | orderamt |
|---|---|---|
| 1 | 101 | 2000 |
| 2 | 103 | 5000 |
| 3 | 101 | 7000 |

The primary key for the customer table is custno and the primary key for the ord table is orderno.

In the ord table the column custno is used to signify that the particular order is placed by which customer and it links the ord table to the customer table. We wish to ensure that whenever a row is entered in the ord table then the custno must always be present in the customer table i.e. a command such as-

*insert into ord values(4, 999, 1200)*

should not be permitted as the custno 999 is not present in the customer table.

In order to enforce this constraint the ord table must have a foreign key constraint which says that the data in the custno column must be a valid value in the customer table.

*A **foreign key** is a column/set of columns in one table (here ord table) which references a primary key of another table (here customer table).*

The foreign key links two tables and ensures that whenever data is added to one table, the data entered in the foreign key of one table must be present as the primary key in the other table.

The foreign key is created during the create table statement as-

```
create table customer
( custno int primary key,
  custname varchar(30)
);
create table ord
( orderno int primary key,
  custno int ,
  orderamt float,
  foreign key (custno) references customer(custno) );
```

## INTERFACING PYTHON WITH MYSQL

In order to access the MySQL database from within python the **mysql.connector** library needs to be imported into our python program.

Assume that the following student table is present in the database for the purpose of the database connectivity programs:

```
create table  student
( regno int primary key,
class char(3),
rollno int,
name varchar(30) not null
);


insert into student values (100,'I',1,'aaa1');
insert into student values (101,'I',2,'aaa2');
insert into student values (150,'II',1,'ccc1');
insert into student values (151,'II',2,'ccc2');
insert into student values (152,'II',3,'ccc3');
```

**Program 1: MySQL connectivity**

```
# 1 MySQL connectivity
import mysql.connector

mycon=mysql.connector.connect(host='localhost', user='student', password='kvo',database='practice')

if mycon.is_connected() == False:
   print('Error connecting to database')
   exit

mycur=mycon.cursor()
mycur.execute('select * from student;')

print('Rows in student table are:')
for x in mycur:       #iterating through the cursor object gives a tuple object
   print('Row Number', mycur.rowcount)
   print(x)

mycur.close()
mycon.close()

o/p:
Rows in student table are:
Row Number 1
(100, 'I', 1, 'aaa1')
Row Number 2
(101, 'I', 2, 'aaa2')
Row Number 3
(150, 'II', 1, 'ccc1')
Row Number 4
(151, 'II', 2, 'ccc2')
Row Number 5
(152, 'II', 3, 'ccc3')
```

1. The command -
   mycon=mysql.connector.connect(host='localhost', user='student', password='kvo',database='practice')
   is used to obtain the connection object (accepted in mycon variable) that maintains database connectivity between our python program and the database. The connection parameters are passed to the connect method to establish connectivity.
2. The command-
   mycon.is_connected()
   can be used to check whether the database connectivity is established or not.
3. The command-
   mycur=mycon.cursor()
   is used to get a cursor object (accepted in mycur variable) from the connection object mycon. Cursor object is used to execute SQL statements on the remote database from within python application.
4. The command-
   mycur.execute('select * from student;')
   is used to execute SQL statements such as SELECT, INSERT, UPDATE and DELETE by passing a string that contains the SQL statement that we wish to execute.
5. For a SELECT query ONLY, the cursor object (mycur) functions as an iterable object which can be iterated upon using a for loop to get ONE row at a time. The row is returned back as a tuple object.
6. The command-
   mycur.rowcount
   is used to return back the TOTAL number of rows fetched by the cursor object from the remote database TILL NOW.
7. The *mycur.close()* - command will close the cursor object and resets all results, and ensures that the cursor object has no reference to its original connection object.
8. The *mycon.close()* - command will close the database connection.

**Program 2: fetchone command**

```
#1_1 fetchone command
import mysql.connector

mycon=mysql.connector.connect(host='localhost', user='student', password='kvo',database='practice')
if mycon.is_connected() == False:
   print('Error connecting to database')
   exit

mycur=mycon.cursor()
mycur.execute('select * from student;')

print('Rows in student table are:')
row = mycur.fetchone()   #fetchone returns a tuple object
while row:  #row is None at the end of resultset
   print('Row Number', mycur.rowcount)
   print(row)
   row = mycur.fetchone()

mycur.close()
mycon.close()
```

*o/p:*

```
Rows in student table are:
Row Number 1
(100, 'I', 1, 'aaa1')
Row Number 2
(101, 'I', 2, 'aaa2')
Row Number 3
(150, 'II', 1, 'ccc1')
Row Number 4
(151, 'II', 2, 'ccc2')
```

```
Row Number 5
(152, 'II', 3, 'ccc3')
```

1. Similar to previous program the command - **mycur.fetchone()** - will return back one row as a tuple object (which is accepted in the variable row)
2. When all the results/rows from the resultset are retrieved the fetchone() command will return **None** and this property can be used to exit the while loop.

**Program 3: fetchall command**

```
#1_3 fetchall command
import mysql.connector

mycon=mysql.connector.connect(host='localhost', user='student', password='kvo',database='practice')

if mycon.is_connected() == False:
   print('Error connecting to database')
   exit

mycur=mycon.cursor()
mycur.execute('select * from student;')

print('Rows in student table are:')
row = mycur.fetchall()   #fetchall returns a list of tuples
print(row)

mycur.close()
mycon.close()

o/p:
Rows in student table are:
[(100, 'I', 1, 'aaa1'), (101, 'I', 2, 'aaa2'), (150, 'II', 1, 'ccc1'), (151, 'II', 2, 'ccc2'), (152, 'II', 3, 'ccc3')]
```

1. The fetchall() command is used to fetch all the rows in ONE go from the resultset of the SELECT command.
2. If there are many rows that are returned by the SELECT statement then there is a chance that the system memory may get full. So it should be used with caution when the number of rows returned is high.

**Program 4: retrieving selected rows of table**

```
# 1_4 retrieving selected rows of table
import mysql.connector

mycon=mysql.connector.connect(host='localhost', user='student',
                    password='kvo',database='practice')
if mycon.is_connected() == False:
   print('Error connecting to database')
   exit

rno=int(input('Enter the registration number to search:'))
#sqlstr = """select * from student where regno = %i;""" %(rno,)
sqlstr = """select * from student where regno = {};""" .format(rno)

mycur=mycon.cursor()
mycur.execute(sqlstr)
rows=mycur.fetchall()
if mycur.rowcount==0:
   print('Reg no', rno,  'not present')
else:
```

```
    print('Details are:')
#   print(rows)
    for i in rows:
        print(i)

mycur.close()
mycon.close()
```

*o/p1:*
*Enter the registration number to search:150*
*Details are:*
*(150, 'II', 1, 'ccc1')*

*o/p2:*
*Enter the registration number to search:999*
*Reg no 999 not present*

1. When we want to retrieve selected rows of a table then we need to use the WHERE clause in the SELECT query.
2. To add the condition to the WHERE clause, there are two methods to construct the query string that is passed to the execute() method-

   **a) Method 1:**

   sqlstr = """select * from student where regno = %i;""" %(rno,)

   Here we use C-language style string formatting:
   - %i is used to replace it with integer value,
   - %f is used to replace with float value,
   - %s is used to replace with string value.

   Usually we use triple quoted strings for the query string and if the data value that is to be replaced in the WHERE clause is a char/varchar/date kind of SQL data, then inside the query string also we enclose the data in either single (') or double quotes(").

   After the triple quoted string in the query string a % symbol followed by a tuple object is passed to perform the replacement within the query string with the value contained in a variable.

   **b) Method 2**

   sqlstr = """select * from student where regno = {};""" .format(rno)

   Here also we use triple quoted string for the query string but we use {} to perform the text replacement using the format() method. The format method will contain as many parameters as there are replacements within the query string.

3. We use the fetchall() method to retrieve the query results. If no rows are matching the criteria then the rowcount is 0 otherwise we can display the results returned by the fetchall() method.

**Program 5: Adding a row to a table**

```
# Practical 18 Adding a row to a table
import mysql.connector

mycon=mysql.connector.connect(host='localhost', user='student',
                    password='kvo',database='practice')

print('Enter the details of the new student to be added:')
rno=int(input('Enter the registration number:'))
cls=input('Enter the class:')
rlnum=int(input('Enter the roll number:'))
nm=input('Enter the name:')

#sqlstr = "insert into student values( %i , '%s', %i, '%s');" %(rno, cls,rlnum,nm )
sqlstr = "insert into student values( {} , '{}', {}, '{}');" .format(rno, cls,rlnum,nm )

if mycon.is_connected() == False:
   print('Error connecting to database')
   exit
mycur=mycon.cursor()
mycur.execute(sqlstr)
mycon.commit()   #commit the transaction

mycur.execute('select * from student;')
print('Rows in student table are:')
for x in mycur:
   print(x)

mycur.close()
mycon.close()
```

*o/p:*

```
Enter the details of the new student to be added:
Enter the registration number:222
Enter the class:V
Enter the roll number:3
Enter the name:pqr

Rows in student table are:
(100, 'I', 1, 'aaa1')
(101, 'I', 2, 'aaa2')
(150, 'II', 1, 'ccc1')
(151, 'II', 2, 'ccc2')
(152, 'II', 3, 'ccc3')
(222, 'V', 3, 'pqr')
```

1. The procedure for performing insert / update / delete is similar. We create the correct query string by using appropriate text replacement and then perform the execute() method against that particular INSERT/UPDATE/DELETE command.
2. The only additional command is the - mycon.commit() - command which is used to perform commit on the database to make the changes to the database permanent.

**Program 6: Deleting a row from a table**

```
# 3 Deleting a row from a table
import mysql.connector

mycon=mysql.connector.connect(host='localhost', user='student',
                    password='kvo',database='practice')
rno=int(input('Enter the registration number of the student to be deleted:'))

sqlstr= "delete from student where regno = %i ;" %(rno,)

if mycon.is_connected() == False:
   print('Error connecting to database')
   exit
mycur=mycon.cursor()
mycur.execute(sqlstr)
mycon.commit()   #commit the transaction
mycur.execute('select * from student;')

print('Rows in student table are:')
for x in mycur:
   print(x)

mycon.close()
```

*o/p:*
*Enter the registration number of the student to be deleted:152*
*Rows in student table are:*
*(100, 'I', 1, 'aaa1')*
*(101, 'I', 2, 'aaa2')*
*(150, 'II', 1, 'ccc1')*
*(151, 'II', 2, 'ccc2')*
*(222, 'V', 3, 'pqr')*

**Program 7: Updating a row of a table**

```
# 4 Updating a row of a table
import mysql.connector

mycon=mysql.connector.connect(host='localhost', user='student',
                    password='kvo',database='practice')

if mycon.is_connected() == False:
   print('Error connecting to database')
   exit

mycur=mycon.cursor(buffered=True)   #to reuse the cursor object use buffered =True

rno=int(input('Enter the registration number of the student to be updated:'))
sqlstr= "select * from student where regno = {} ; ".format(rno)

mycur.execute(sqlstr)

if mycur.rowcount == 0:
   print("Registration number", rno, "does not exist in database")
else:
   print("Enter the new details for student with reg no", rno)
   cls=input('Enter the class:')
   rlnum=int(input('Enter the roll number:'))
   nm=input('Enter the name:')
```

```
    sqlstr= "update student set class= '{}'  , rollno={} ,name= '{}'  where regno= {} ;".format(cls,rlnum,nm,rno)

    #print(sqlstr)
    mycur.execute(sqlstr)
    mycon.commit()   #commit the transaction

    mycur.execute('select * from student;')

    print('Rows in student table are:')
    for x in mycur:
        print(x)

mycon.close()
```

*o/p:*
*Enter the registration number of the student to be updated:151*
*Enter the new details for student with reg no 151*
*Enter the class:XII*
*Enter the roll number:888*
*Enter the name:xyz*

*Rows in student table are:*
*(100, 'I', 1, 'aaa1')*
*(101, 'I', 2, 'aaa2')*
*(150, 'II', 1, 'ccc1')*
*(151, 'XII', 888, 'xyz')*
*(222, 'V', 3, 'pqr')*

If the same cursor object is to be reused in the program then add the parameter buffered=True when creating the cursor object.

***NOTE:***
Whenever you are passing arguments from a user and concatenating it to a string to form the query string within the program, some check on the input must be performed so that the user may not pass bad characters/commands to the database - such as quotation marks, %, _ etc characters which have special meaning in SQL or even other SQL commands that may delete the data. Such attempts are known as **'SQL injection attacks'.**